TKR COLLEGE OF ENGINEERING AND TECHNOLOGY
AN AUTONOMOUS INSTITUTION
Accredited by NBA and NAAC with 'A' Grade.
(Sponsored by TKR Educational Society, Approved by AICTE, Affiliated to JNTU H)
Medbowli, Meerpet, Balapur, Hyderabad, Telangana – 500 097
Phone: 9100377790, email: info@tkrcet.ac.in, web site: www.tkrcet.ac.in

AI BASED PERSONAL FINANCE MANAGER

FINJARVIS: a web platform that categorises spending, forecasts the next month, scores financial health and answers questions from the user's own data

A project report submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

MEKALA KALYAN	**23K91A05F0**
RAJANALA SAI HEMANTH	**23K91A05K3**
MIDDALA SURYA TEJA REDDY	**23K91A05F2**
MANGA AKASH	**23K91A05E5**

Batch C11 (CSE-C)

Under the esteemed guidance of

N PADMAVATHI

[DESIGNATION]

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

TKR COLLEGE OF ENGINEERING & TECHNOLOGY (AN AUTONOMOUS INSTITUTION)

Medbowli, Meerpet, Balapur, Hyderabad, Telangana - 500 097

Academic year [ACADEMIC YEAR]

CERTIFICATE

TKR COLLEGE OF ENGINEERING AND TECHNOLOGY
AN AUTONOMOUS INSTITUTION
Accredited by NBA and NAAC with 'A' ' Grade.
(Sponsored by TKR Educational Society, Approved by AICTE, Affiliated to JNTU H)
Medbowli, Meerpet, Balapur, Hyderabad, Telangana – 500 097
Phone: 9100377790, email: info@tkrcet.ac.in, web site: www.tkrcet.ac.in

This is to certify that the project report entitled **"AI Based Personal Finance Manager"** is a bonafide record of the work carried out by **MEKALA KALYAN** (23K91A05F0), **RAJANALA SAI HEMANTH** (23K91A05K3), **MIDDALA SURYA TEJA REDDY** (23K91A05F2), **MANGA AKASH** (23K91A05E5), in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, during the academic year [ACADEMIC YEAR], under my supervision.

Project Guide	**Head of the Department**	**Principal**
N Padmavathi	[HOD NAME]	[PRINCIPAL NAME]
[DESIGNATION]	Department of Computer Science and Engineering	TKR College of Engineering & Technology

External Examiner: [NAME AND SIGNATURE]

Place: [PLACE]

Date: [DATE]

DECLARATION

We hereby declare that the project report entitled **"AI Based Personal Finance Manager"**, submitted to the Department of Computer Science and Engineering, TKR College of Engineering & Technology, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of original work carried out by us under the guidance of N Padmavathi, [DESIGNATION]. The results embodied in this report have not been submitted to any other university or institution for the award of any degree or diploma.

MEKALA KALYAN 23K91A05F0

RAJANALA SAI HEMANTH 23K91A05K3

MIDDALA SURYA TEJA REDDY 23K91A05F2

MANGA AKASH 23K91A05E5

Place: [PLACE]

Date: [DATE]

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, N Padmavathi, [DESIGNATION], for valuable guidance, encouragement and constructive suggestions throughout this project.

We thank [HOD NAME], Head of the Department of Computer Science and Engineering, for providing the facilities and support needed to complete this work, and [PRINCIPAL NAME], Principal, TKR College of Engineering & Technology, for the encouragement given to us.

We thank the project coordinators and all faculty and staff members of the department for their help, and our families and friends for their constant support.

We also acknowledge R. Kharat, S. Solapurkar, A. Tikkas, M. Uttarwar and A. Yadav, whose multi-model personal finance assistant is the registered base paper of this project, and the other authors whose work is listed in the references. We are grateful to the maintainers of the open-source software used here: FastAPI, SQLAlchemy, SQLite, scikit-learn, pandas, ReportLab, OpenPyXL, React, Vite, Recharts, pytest, matplotlib and Playwright.

MEKALA KALYAN

RAJANALA SAI HEMANTH

MIDDALA SURYA TEJA REDDY

MANGA AKASH

Managing personal money is hard when income, recurring bills and investments are scattered across many digital transactions, and most tools only record what already happened. This project builds FINJARVIS, a web application in which a user signs up, records or imports transactions, and gets an automatic category for each one, a forecast of next month's spending, monthly budgets with alerts at 75, 90 and 100 per cent, savings goals, a manual investment portfolio, a 0-100 financial health score and an assistant that answers questions from the user's own data. The categoriser is a TF-IDF and logistic-regression model trained on a 51-description seed list, the forecast is a least-squares linear trend over monthly totals, and the health score is a weighted sum of six explainable components. The back end is FastAPI over SQLAlchemy and SQLite and the front end is React; the original Streamlit pages run on the same Python code. The registered base paper combines BERT, Q-learning, LSTM and Isolation Forest; this work implements the first two tasks in a much simpler form and does not implement anomaly detection or reinforcement learning.

We measured it with its own tools. All 38 unit tests and 49 end-to-end smoke checks pass. On a 198-description probe set the categoriser scores 71.7 % (macro-F1 0.761), but 100 % on seed-like wording against 18.2 % on merchants it has not seen, where it falls back to the Transport class. On the demo account the linear trend has a one-step MAPE of 28.4 % over three test months, worse than a plain mean (21.4 %), and a simulation shows it only beats the mean after about seven months of history. Typical API responses take 7 to 130 ms on a laptop-class CPU. All data are fictional or synthetic and no study with real users was done.

Keywords: personal finance automation, expense categorisation, expense forecasting, financial health score, budgeting assistance, smart alerts, data visualisation, FastAPI, linear regression, TF-IDF.

TABLE OF CONTENTS

CERTIFICATE i
DECLARATION ii
ACKNOWLEDGEMENT iii
TABLE OF CONTENTS v
LIST OF FIGURES viii
LIST OF TABLES x
LIST OF ABBREVIATIONS xi
CHAPTER 1: INTRODUCTION 1
1.1 Introduction to the Project Domain 1
1.2 Background and Motivation 1
1.3 Problem Statement 2
1.4 Need for the Proposed System 3
1.5 Project Objectives 3
1.6 Scope of the Project 4
1.7 Project Contributions 4
1.8 Limitations and Assumptions 5
1.9 Organization of the Report 6
CHAPTER 2: LITERATURE SURVEY 7
2.1 Overview 7
2.2 Base Paper Analysis 7
2.3 Expense Categorisation 9
2.4 Expense Forecasting 10
2.5 Anomaly Detection and Budget Recommendation 10
2.6 Recent Personal-Finance Systems 10
2.7 Datasets and Algorithms Used in Related Work and in This Project 11
2.8 Strengths and Limitations of Existing Research 12
2.9 Research Gap 13
CHAPTER 3: FEASIBILITY STUDY 14
3.1 Technical Feasibility 14
3.2 Economic Feasibility 14
3.3 Operational Feasibility 15
3.4 Schedule Feasibility 15
3.5 Risk Analysis 15
3.6 Advantages and Constraints 16
CHAPTER 4: SYSTEM REQUIREMENTS AND ANALYSIS 17
4.1 Existing System 17
4.2 Limitations of the Existing System 17
4.3 Proposed System 17
4.4 Advantages of the Proposed System 18
4.5 Data Requirements 18
4.6 Functional Requirements 19
4.7 Non-Functional Requirements 19
4.8 Hardware Requirements 20
4.9 Software Requirements 20
4.10 User Roles and System Constraints 21
CHAPTER 5: SYSTEM DESIGN 22
5.1 Design goals and architecture 22
5.2 High-level workflow 23
5.3 Data flow diagrams 24
5.4 UML design 27
5.5 Database design 29
5.6 Core algorithm design 30
5.7 Module design 32
5.8 API design 32
5.9 User interface design 34
5.10 Design decisions and trade-offs 34

CHAPTER 6: METHODOLOGY AND SOFTWARE IMPLEMENTATION....36
6.1 Overall methodology....36
6.2 Dataset....36
6.3 Preprocessing....37
6.4 Algorithm choice and justification....38
6.5 Mathematical formulation....38
6.6 Model architecture and training....40
6.7 Evaluation protocol....41
6.8 Application architecture....42
6.9 API design and key code....42
6.10 Security and error handling....43
6.11 Installation and execution....44
CHAPTER 7: SYSTEM TESTING....46
7.1 Testing strategy....46
7.2 Unit testing....46
7.3 Integration and functional testing....47
7.4 Security testing....48
7.5 Validation and edge cases....48
7.6 Model validation as testing....49
7.7 Performance testing....49
7.8 User acceptance testing....50
7.9 Limits of the testing done....51
CHAPTER 8: RESULTS AND OUTPUT SCREENS....52
8.1 Experimental set-up....52
8.2 Output screens....52
8.3 Expense categoriser....55
8.4 Spending forecast....58
8.5 Financial health score....60
8.6 AI assistant....61
8.7 Import pipeline, API latency and tests....62
8.8 Comparison with the base paper....63
8.9 Sample inputs and outputs....64
8.10 Success and failure analysis....64
8.11 Limitations of the evaluation....65
8.12 Objectives versus outcomes....66
8.13 Summary of findings....66
CHAPTER 9: CONCLUSION....68
9.1 Summary....68
9.2 Objectives achieved....68
9.3 Key findings....68
9.4 Module-by-module conclusions....69
9.5 Practical applications....70
9.6 Closing remark....70
9.7 Known limitations....70
CHAPTER 10: FUTURE SCOPE....71
10.1 Improvements to the existing system....71
10.2 Additional features....71
10.3 Scalability and deployment....71
10.4 Evaluation plan for the next version....72
10.5 Order of work....72
10.6 Risks and dependencies....72
10.7 Further research....72
REFERENCES....73
APPENDIX A: IMPORTANT CODE....75
A.1 Expense categoriser (ml/expense_classifier.py)....75
A.2 Spending forecast (ml/spending_prediction.py)....75
A.3 Financial health score (ml/financial_health.py)....76
A.4 Budget alerts (services/budget_service.py)....76

A.5 Tokens and the authentication dependency (backend/main.py) 76
APPENDIX B: INSTALLATION AND USER GUIDE 78
B.1 Requirements 78
B.2 Installation and start with run.bat 78
B.3 Manual start and settings 78
B.4 Using the web app 79
B.5 Tests and experiments 79
B.6 Troubleshooting 80
APPENDIX C: ADDITIONAL SCREENS 81
C.1 Light theme and sign-up pages 81
C.2 Additional result charts 82

LIST OF FIGURES

Figure 1: The 51-example seed list on which the shipped categoriser is trained (examples per category). 11
Figure 2: Layered architecture of FINJARVIS: React single-page app, Vite proxy, FastAPI application with 33 routes, service layer, ML modules, SQLAlchemy and SQLite. 22
Figure 3: High-level workflow: record or import a transaction, categorise it, update budgets, forecast, score and answer questions. 23
Figure 4: Data flow diagram, level 0: FINJARVIS as one process between the user, the optional LLM provider and the market-data source. 24
Figure 5: Data flow diagram, level 1: the main processes and the six data stores. 25
Figure 6: Data flow diagram, level 2: adding a transaction (validation, categorisation, learning, storage). 26
Figure 7: UML class diagram: the six ORM classes, the service functions and the three ML modules. 27
Figure 8: Sequence diagram of the main flow: login, add a transaction with auto-categorisation, then read the dashboard. 28
Figure 9: Use-case diagram: the registered user, the demo visitor and the optional external services. 28
Figure 10: Entity-relationship diagram of the six tables. 29
Figure 11: Flowchart of the core algorithms: categoriser, forecast, health score and budget alerts. 30
Figure 12: Categoriser pipeline: description, TF-IDF, logistic regression, label, optional correction and retraining. 31
Figure 13: Financial health score model: six weighted components, clipped sum, rating band. 31
Figure 14: The whole training set of the categoriser: 51 descriptions in 11 categories. 37
Figure 15: Landing page (dark theme): the hero card shows the demo account for October 2026 with health score 90 (Excellent), income INR 65K, spent INR 30.1K and 54 % saved; the strip below lists what the product does. 53
Figure 16: Login page: username or e-mail, password, a Try the demo account button and the demo credentials. 53
Figure 17: Dashboard for October 2026: balance INR 1,98,382, income INR 65,000, expenses INR 30,080, savings INR 34,920 (53.7 %), investments INR 84,250, net worth INR 2,82,632, average budget usage 44 %. The health card shows 90 (Excellent), component scores 100, 100, 100, 100, 80 and 24, and names Goal Progress as the area to focus on. 53
Figure 18: Dashboard charts: income against expenses, spending by category (led by Rent), the six-month trend and the allocation of the four holdings. 53
Figure 19: Add a transaction: type, date, amount, description, a category that defaults to Auto-detect (AI), payment method and notes; a manual category that differs teaches the model. 53
Figure 20: View and manage: 68 transactions, income INR 3,90,000 and expenses INR 1,91,618 for the six months, with filters, search, sort and edit and delete icons. 53
Figure 21: Import from CSV or Excel: the preview reports Row 6 as skipped and lists the four valid rows with categories the AI filled in; nothing is saved until the preview is confirmed. 54
Figure 22: Analytics for 24 July to 22 October 2026: income INR 1,95,000, expenses INR 1,14,194, savings rate 41.4 %, daily average INR 1,359 and largest expense INR 18,000 (rent); the category chart is led by Rent, Investments and Shopping. 54
Figure 23: Spending trend and prediction: the expense curve for May to October 2026 and the prediction card: next month INR 39,471 from six months, recent three-month average INR 36,592, trend Increasing. ..54
Figure 24: Budgets for October 2026: Food 15 %, Transport 9 %, Entertainment INR 2,200 of 2,500 (88 %, Getting close), Shopping 45 % and Bills 65 %, each with a progress bar; warnings are promised at 75, 90 and 100 %. 54
Figure 25: Financial goals: Emergency Fund 32 %, New Laptop 19 % and Vacation 20 %, each with the suggested monthly saving and an update field. 54
Figure 26: Investments: invested INR 77,860, value INR 84,250, profit INR 6,390 (8.21 %) over four holdings, with an allocation donut and profit and loss by asset. 54
Figure 27: AI assistant: two questions answered from the user's own data (Rent at INR 108,000 is the top category; health 90.4, Excellent, weakest area goal progress). A banner notes that free-form answers are off because no AI_API_KEY is set. 55
Figure 28: Financial reports: month and year selectors and three downloads, a PDF summary, an Excel workbook and a CSV of every transaction. 55
Figure 29: Notifications: one unread alert, Your Entertainment budget is 75% used, created when spending crossed the 75 % threshold. 55

Figure 30: Profile: name, e-mail, currency INR, monthly income INR 65,000, savings target INR 20,000, risk preference Moderate and a card that loads the fictional demo data. 55
Figure 31: Accuracy of the three models on each probe tier; the shipped model falls from near 100 % to 18 % on unseen merchants. 56
Figure 32: Confusion matrix on the probe set (rows are true categories, 18 descriptions each); the Transport column collects the unseen-merchant errors. 57
Figure 33: (a) Tier-C accuracy against the number of corrections; (b) accuracy on the same merchant written three other ways, before and after one correction. 58
Figure 34: Actual monthly expenses against the four one-step-ahead forecasts for months 4 to 6 of the demo account. 59
Figure 35: Simulated MAPE against months of history at 10 % noise; the dotted line marks the application's three-month minimum. 60
Figure 36: Health-score sensitivity to each input, changed one at a time from the baseline profile. 61
Figure 37: Additional screen: landing page, light theme, with the same demo card (health score 90, income INR 65K, spent INR 30.1K, saved 54 %). 81
Figure 38: Additional screen: dashboard, light theme: the same eight cards and the health score card as in the dark theme. 81
Figure 39: Additional screen: sign-up page with the password-rule chips (8+ characters, upper-case, lower-case, a digit, a special character) that turn on as the password satisfies each rule. 81
Figure 40: Precision, recall and F1 per category on the probe set; Transport recalls everything but is right on only 27 % of what it labels. 82
Figure 41: Accuracy on kept predictions and share of items kept as the minimum probability rises (computed offline; the application applies no threshold). 82
Figure 42: The complete training set of the categoriser: 51 descriptions, 3 to 6 per category. 82
Figure 43: Simulated MAPE of the four forecasting methods against noise (2,000 synthetic users per point). 83
Figure 44: Ratings of 5,000 random profiles under the health score. 83
Figure 45: Outcome of the 26 test questions for the assistant's rule layer. 83
Figure 46: Median and 95th-percentile latency of every measured endpoint. 83

LIST OF TABLES

Table 1: Abbreviations used in this report xi
Table 2: Objectives of the project and their outcome 3
Table 3: Relation of FINJARVIS to the base paper 9
Table 4: Algorithms used in FINJARVIS and where each comes from 11
Table 5: Comparison of the surveyed work 12
Table 6: Technical feasibility: components and the evidence for each 14
Table 7: Cost items 14
Table 8: Project phases 15
Table 9: Project risks and mitigation 15
Table 10: Advantages and constraints of the project 16
Table 11: Modules of the proposed system 18
Table 12: Advantages and the evidence for each 18
Table 13: Data requirements 18
Table 14: Functional requirements and how each is met 19
Table 15: Non-functional requirements, measured values and status 19
Table 16: Hardware used for development and measurement 20
Table 17: Software used 20
Table 18: User roles 21
Table 19: The six tables and their columns 29
Table 20: Modules, sizes and responsibilities. 32
Table 21: The 33 routes of the FINJARVIS API. 33
Table 22: Pages, routes and the endpoints they use. 34
Table 23: Principal design decisions 34
Table 24: Data sets used by the product and by the measurements. 36
Table 25: Algorithms chosen and the alternatives not used. 38
Table 26: Weights, neutral values and rating bands of the health score. 40
Table 27: Training configuration of the categoriser. 40
Table 28: Software requirements. 44
Table 29: Test levels, tools and results. 46
Table 30: Unit tests by file. 47
Table 31: Smoke-test checks by feature. 47
Table 32: Security checks that were run. 48
Table 33: Edge cases and observed behaviour 48
Table 34: API latency (milliseconds). 50
Table 35: Categoriser on the 198-description probe set 55
Table 36: Per-category scores on the probe set (18 descriptions per category). 57
Table 37: One-step-ahead forecasts for August, September and October 2026 (INR) and their errors. 58
Table 38: Simulated one-step-ahead MAPE (%) against noise (12-month histories). 59
Table 39: Components of the demo account's score (October 2026) 60
Table 40: Sample questions and the assistant's answers (demo account). 61
Table 41: API latency in milliseconds (n = repetitions). 62
Table 42: Base paper against this work. 63
Table 43: Misclassified probe descriptions (first four tier-C errors and both tier-B errors). 64
Table 44: The six project objectives against what was measured. 66
Table 45: Objectives against outcomes 68
Table 46: Environment variables 78
Table 47: Steps for each page 79
Table 48: Common problems 80

LIST OF ABBREVIATIONS

Table 1: Abbreviations used in this report

Abbreviation	Expansion	Abbreviation	Expansion
API	Application Programming Interface	LSTM	Long Short-Term Memory
BERT	Bidirectional Encoder Representations from Transformers	MAE	Mean Absolute Error
CORS	Cross-Origin Resource Sharing	MAPE	Mean Absolute Percentage Error
CRUD	Create, Read, Update, Delete	ML	Machine Learning
CSV	Comma-Separated Values	NFR	Non-Functional Requirement
DFD	Data Flow Diagram	ORM	Object-Relational Mapping
ER	Entity-Relationship	REST	Representational State Transfer
FR	Functional Requirement	RL	Reinforcement Learning
HMAC	Hash-based Message Authentication Code	SMOTE	Synthetic Minority Over-sampling Technique
HTTP	Hypertext Transfer Protocol	SPA	Single-Page Application
INR	Indian Rupee	SQL	Structured Query Language
JSON	JavaScript Object Notation	TF-IDF	Term Frequency - Inverse Document Frequency
LLM	Large Language Model	UI	User Interface
LOO	Leave-One-Out	UML	Unified Modeling Language
LR	Logistic Regression	UPI	Unified Payments Interface

CHAPTER 1: INTRODUCTION

1.1 Introduction to the Project Domain

Almost everyone who earns money in a city today also leaves a trail of small digital transactions: a UPI payment for a meal, a card debit for a bus pass, a standing order for rent, a mutual-fund instalment. The batch introduction records the consequence: individuals now hold several accounts, income sources, recurring bills and investments at once, and manual expense tracking is tedious enough that most users abandon it within weeks. Financial literacy research links the ability to plan, save and respond to unexpected expenses with long-term well-being [19], and a recent user-centred study of a finance app for young adults begins from the same worry, that people lack the skills and the habit to manage money [2]. Software that records spending without effort and turns the record into a forecast and a warning is the subject of this project.

Four ideas from machine learning meet in a personal finance manager: **text classification** of a description such as "Swiggy order", **time-series forecasting** of next month's spending, **anomaly detection** of transactions that do not fit a person's pattern, and **recommendation** of advice that fits the person's income. The registered base paper combines all four in one assistant [1]. This project builds the first two, a rule-based form of the fourth, and a scoring model the base paper does not have; it does not build the third (Section 1.5).

Terms used in this report: a **transaction** is one dated income or expense with a description, amount, category and payment method; a **category** is one of eleven expense labels (Food, Transport, Entertainment, Bills, Shopping, Education, Healthcare, Rent, Travel, Investments, Salary) or "Other"; a **budget** is a monthly limit for one category; a **goal** is a savings target with a deadline; a **holding** is one investment entered by the user; the **health score** is a 0-100 number from six weighted components; the **assistant** answers questions from the user's records.

1.2 Background and Motivation

The batch registers a 2025 paper in the International Journal for Research in Applied Science and Engineering Technology as the base paper: "A Smart AI-Based Personal Finance Assistant: A Multi-Model Approach for Expense Categorization, Budgeting, Forecasting and Anomaly Detection" [1]. Its authors argue that existing tools depend on manual categorisation and static budgeting rules and propose four models in one pipeline: a fine-tuned BERT classifier that sorts transactions into 50 categories (extended from 16, with SMOTE oversampling), a Q-learning agent that recommends a budget level, an LSTM that forecasts savings from twelve months of

history, and an Isolation Forest that flags unusual transactions. It reports an accuracy of 92.3 % (precision 90.8 %, recall 91.5 %, F1 91.1 %) for the classifier, an RMSE of 137.2, MAE of 89.5 and R-squared of 0.63 for the forecaster, and precision 0.93, recall 0.94 and F1 0.94 for the anomaly detector, on data the paper describes as synthetic or collected and labelled by the authors; the data are not released.

The paper is a research prototype with a Streamlit interface; the batch's gap statement is to deliver the multi-model idea as an affordable, customisable web platform. Similar recent work includes an explainable XGBoost and LSTM hybrid [4], an Isolation Forest and K-Means leakage detector [5] and a data-generation framework for finance language models [3] (Chapter 2).

FINJARVIS is a working application, not a notebook: a person signs up, adds transactions by hand or imports a CSV or Excel file, sees each category filled in by a trained model, watches budgets and goals, reads a forecast and a health score, asks questions in plain language and downloads reports. For the user it is a free, inspectable alternative to commercial finance apps; for the project it is a chance to build simple versions of the base paper's modules and measure honestly where they work and where they do not.

1.3 Problem Statement

The batch problem statement lists four difficulties. Manual expense recording is tedious and is quickly abandoned. Existing applications report past spending but do not forecast what is coming. Generic budgeting advice ignores individual income patterns and commitments. Unusual transactions go unnoticed until the statement is reviewed much later. The stated need is a platform that categorises spending automatically, forecasts future expenditure and gives advice grounded in the user's own data.

We restate the problem so that it can be tested. **Build a personal finance platform in which each new transaction is categorised by a trained model, the next month's spending is forecast from the recorded months, budget thresholds raise alerts, a single explainable score summarises financial health, and questions are answered from the user's own records; then measure how each part behaves, including where it fails.** The two quantitative parts can be written down. If the recorded monthly expense totals are y1 ... yn (n at least 3), the forecast for the next month is the value at x = n of the least-squares line through the points (i, yi), clipped at zero. The health score is the weighted sum of six component scores s1 ... s6, each clipped to the range 0 to 100:

$$forecast = max(0,\ a*n + b),\ \ where\ (a, b)\ minimise\ the\ sum\ of\ (yi - a*i - b)\text{^}2 \qquad score = 0.30*s1 + 0.20*s2 + 0.15*s3 + 0.15*s4 + 0.10*s5 + 0.10*s6 \qquad (1)$$

Neither formula needs a large dataset: the project asks how far such simple, inspectable models go. The measurements show that the categoriser is right for 100 % of descriptions that resemble its training phrases but for only 18.2 % of unseen merchants, and that the linear trend is not better than a plain average until about seven months of history exist (Chapter 8).

1.4 Need for the Proposed System

- **Effortless record keeping.** A model suggests the category when a description is typed and learns from a correction.
- **Looking forward, not only back.** A next-month and per-category forecast, checked against budgets, lets a user act before a limit is crossed.
- **Advice from the person's own numbers.** The health score names the weakest of six areas, and the assistant answers from the user's recorded data instead of giving general tips.
- **Alerts at thresholds.** Budget alerts at 75 %, 90 % and 100 % of a limit are written to a notification list while the month is still running.
- **Affordable and open.** Python, FastAPI, SQLite and React are free; the data stays in one local file; nothing is sent to a cloud service unless the user adds an optional language-model key.
- **Evidence, not assertion.** The platform carries its own tests, a probe set and measured response times, so a reader can check what works.

1.5 Project Objectives

The six objectives set for the batch, and the outcome of each, are listed below. The outcome column is the short version; Chapter 8 gives the measurements and Chapter 9 the verdict. Two objectives were not met, and the table says so plainly.

Table 2: Objectives of the project and their outcome

No.	Objective	Outcome (summary)
1	Build secure user accounts with transaction import and manual entry	Achieved: sign-up and login with bcrypt-hashed passwords and signed bearer tokens, per-user data scoping, manual entry and CSV or Excel import with preview; 38 unit tests and 49 smoke checks pass.
2	Categorise expenses automatically from descriptions using text classification	Achieved in a small form: TF-IDF with logistic regression on a 51-example seed list; 71.7 % accuracy on a 198-description probe set but 18.2 % on unseen merchants.
3	Forecast upcoming expenses and savings using time-series models	Partly achieved: a linear trend forecasts next month's expenses (INR 39,471 for the demo account); no ARIMA, LSTM or direct savings forecast; backtest MAPE 28.4 % on three points.

No.	Objective	Outcome (summary)
4	Detect anomalous transactions that deviate from an established pattern	Not achieved: no anomaly detector exists in the product. The nearest feature reports the single largest expense of a period.
5	Generate personalised budget recommendations and alerts on threshold breaches	Partly achieved: user-set budgets with alerts at 75, 90 and 100 %, a six-component health score and rule-based advice; no learned (reinforcement-learning) recommender.
6	Visualise spending, category breakdown and forecasts on an interactive dashboard	Achieved: a React dashboard and analytics page with 13 pages, charts of monthly trend and category split, and a prediction panel; 19 screens captured.

The batch abstract also promises a Flask and MySQL application with loan tracking, and the batch notes mention a loan amortisation tracker and scheduled digest alerts. The shipped system uses FastAPI and SQLite (it began as a Streamlit application); it has no loan table, no amortisation calculator and no timer-driven alerts. These are deviations from the abstract and are repeated in Chapters 9 and 10.

1.6 Scope of the Project

In scope. Accounts with strong-password rules and per-user data; manual and file-based transaction entry; search, filter, edit and delete; a trained expense categoriser with a keyword fallback and learning from corrections; monthly budgets with threshold alerts; savings goals with required monthly saving; a manual investment portfolio with optional live price refresh; analytics (monthly trend, category split, largest expense); a next-month and per-category spending forecast; the six-component health score; a rule-routed assistant with an optional language-model layer; PDF, Excel and CSV reports; notifications; a demo account with fictional data; and the tests, probe set and measurement scripts that produce every number in this report. The web application has 13 pages over 33 REST routes and 6 database tables.

Out of scope. Anomaly or fraud detection; reinforcement-learning budgeting; deep sequence models (BERT, LSTM); loan, debt and tax tools; bank-API or SMS import; multi-currency conversion; multi-user households or sharing; scheduled digests; mobile apps; production hosting, backups and migrations; and any study with real users or real financial records.

1.7 Project Contributions

1. **A complete, runnable finance platform.** One code base covers accounts, entry and import, categorisation, budgets, goals, investments, analytics, forecasting, a health score, an assistant and reports, behind 33 REST routes and a 13-page React console, started with one command.
2. **An explainable health score.** Six weighted components with published weights and bands; a sensitivity study and a 5,000-profile check show that the score is monotone in the savings rate and where it saturates.

3. **A learn-from-correction categoriser.** The model retrains on the user's history plus the correction; a measured test shows that one correction fixes a repeat merchant in every variant tested while barely helping new merchants.
4. **A forecast with a stated floor.** The module refuses to predict with fewer than three months, and a simulation shows that three months is too few for a linear trend to beat a mean.
5. **An honest evaluation.** 38 passing tests, 49 of 49 smoke checks, a 198-description probe set, a rolling-origin backtest, a 1,000-row import test and measured response times, reported together with the defects they exposed.

In brief, the shipped categoriser reaches 71.7 % accuracy on the probe set (macro F1 0.761) against 51.5 % for the keyword table, but 100 % on seed-like text and 18.2 % on unseen merchants, where 48 of the 56 errors are labelled Transport. The demo forecast has a MAPE of 28.4 % against 21.4 % for a plain mean. The base paper's accuracy figures are neither reproduced nor contradicted: its data, labels and model are different. All results come from a very small seed list and synthetic demo data.

1.8 Limitations and Assumptions

- **Tiny training set.** The classifier learns from 51 short phrases, 3 to 6 per category; leave-one-out accuracy on that list is 11.8 %.
- **Synthetic data.** The demo account is fictional and the probe set was written by the team, so it is not an independent benchmark.
- **Simple models.** The forecast is a straight line through monthly totals and the advice is rule-based; there is no anomaly detector, reinforcement learning or deep model.
- **Optional language model untested.** The optional Anthropic layer was never exercised in any measurement.
- **Known defects.** Unseen merchants are silently labelled, mostly as Transport, because the app applies no confidence threshold; imports commit one row at a time (35.2 rows per second); alerts are de-duplicated by message text only; tokens are home-made HMAC bearer tokens with a default development secret.
- **Single machine, single user.** Concurrency, mobile browsers and live price refresh were not tested.
- **Not financial advice.** Advice screens carry a disclaimer.

1.9 Organization of the Report

Chapter 2 surveys the literature and analyses the base paper. Chapter 3 studies feasibility, and Chapter 4 sets out the existing and proposed systems and the requirements. Chapter 5 gives the design: architecture, workflow, data flow and UML diagrams, the database, the modules and the user interface. Chapter 6 describes the methodology and implementation: the data, the categoriser, the forecast and the health score with their mathematics, the API, security and error handling, and how to install and run the system. Chapter 7 reports testing. Chapter 8 presents every output screen, the measured results, the comparison with the base paper and the objectives against outcomes. Chapter 9 concludes and Chapter 10 sets out the future scope. The references and three appendices (important code, installation and user guide, additional screens) follow.

CHAPTER 2: LITERATURE SURVEY

2.1 Overview

The survey has four threads: the registered base paper, a multi-model personal finance assistant; automatic categorisation of transaction text; forecasting of personal expenses, where the question is how much model is worth having with only a few months of history; and the recent personal-finance systems around the base paper. The base paper [1] is analysed in Section 2.2 and every claim about it was checked against the PDF in the batch folder. Claims about the other works are limited to what their own abstracts and text say; where a paper reports no result the tables say so.

The batch literature survey (5.LITERATURE SURVEY.docx) lists five works: the base paper and four papers from 2025 and 2026. A search on 26 October 2026 located two of the four ([4] and [5]), found an IEEE Xplore record for a third whose authors and conference could not be read [6], and found no record of the fourth [7]; the last two are listed as the batch gave them and nothing here depends on their content. The two papers in the batch's base-paper archive [2], [3] were read in full and are used as supporting work.

2.2 Base Paper Analysis

2.2.1 Registered base paper: Kharat et al., IJRASET, 2025

Reference. R. Kharat, S. Solapurkar, A. Tikkas, M. Uttarwar and A. Yadav, "A Smart AI-Based Personal Finance Assistant: A Multi-Model Approach for Expense Categorization, Budgeting, Forecasting and Anomaly Detection," IJRASET, vol. 13, no. XI, pp. 167-180, November 2025, doi 10.22214/ijraset.2025.75019 [1]. The authors are from the Department of Computer Engineering, Pimpri Chinchwad College of Engineering, Pune.

Problem. Users struggle to categorise expenses, plan budgets, forecast savings and spot fraudulent transactions; existing tools rely on manual input and rule-based approaches that are error-prone and not personalised. The paper proposes one assistant that automates all four tasks and adapts to the user's behaviour.

Method. Four independent modules form a pipeline. (1) *Expense categorisation*: transaction descriptions are cleaned, tokenised with the BERT tokenizer and classified by a fine-tuned bert-base-uncased model [8]; the label set is extended from 16 to 50 categories and class imbalance is handled with SMOTE [13]; training uses learning rate 5e-5, batch size 8, 10 epochs, dropout 0.1 and an 80/20 split. (2) *Budget recommendation*: a Q-learning agent [12] with a state made of income, fixed and variable expenses and savings goal, rewards of +1 and -1, alpha 0.1,

gamma 0.95 and epsilon decaying from 0.995 to 0.01 over 300 episodes, producing a "High", "Moderate" or "Low" savings recommendation. (3) *Savings forecasting*: an LSTM [11] with 64 and 32 units trained on 12-month sequences of a synthetic savings series (min-max scaling, trend features, 50 epochs). (4) *Anomaly detection*: an Isolation Forest [10] with 300 trees, contamination 0.015 and savings and expense ratios as features. The modules are served through a Python back end (Flask or Streamlit) and a Streamlit interface.

Evaluation and findings. The classifier is reported at accuracy 92.3 %, precision 90.8 %, recall 91.5 % and F1 91.1 %, better than rule-based keyword matching (a stated +42 % accuracy) and than Random Forest and SVM baselines; the Q-learning reward rises and converges after about 150 to 200 of 300 episodes; the LSTM reaches RMSE 137.2, MAE 89.5 and R-squared 0.63; the Isolation Forest reaches precision 0.93, recall 0.94, F1 0.94 and R-squared 0.87. A comparison table claims improvements of 42 % (categorisation accuracy), 31 % (budget efficiency), 29 % (RMSE reduction against ARIMA) and 27 % (anomaly recall against manual review).

Strengths and limits. The paper sets out a clear modular architecture, states its hyper-parameters, and covers the whole chain from raw text to alerts. For the purpose of this project its limits are that the data are synthetic or collected by the authors and not released, so none of the numbers can be reproduced; that the reinforcement-learning state and action spaces are only sketched and its evaluation is a reward curve rather than a measured saving; that the LSTM errors have no stated units and the forecaster is trained on a synthetic series; that the anomaly detector is scored on injected outliers, which makes a high score easy; and that there is no deployed multi-user application, authentication or test suite. The forecasting section also calls an R-squared of 0.63 "accuracy of 63 %", which is a different quantity.

2.2.2 Supporting papers read in full

To (arXiv:2601.08640, January 2026) [2] designs a mobile finance application, Mabu, for young adults with Alan Cooper's Goal-Directed Design and gamification. It uses interviews, a survey of respondents aged 18 to 30, a closed card sort with 10 participants and think-aloud and usability tests with 6 participants, and it quotes a 2023 UK test in which only 17 % of 18 to 24 year olds showed sufficient financial literacy. The paper contributes a design method and a usability study, not a model; it supports the claim that a finance app must serve personal goals and be easy to use, which shaped the dashboard and goals pages of this project.

Theerthala (arXiv:2509.14180, September 2025) [3] builds a 19,000-sample reasoning dataset that combines financial context with behavioural-finance studies and fine-tunes an 8-

billion-parameter model on it; a blind study with language models as judges finds the model comparable to 14 to 32-billion-parameter baselines on factual accuracy, fluency and personalisation at about 80 % lower cost. It shows why advice has to be grounded in the user's goals and constraints. This project does not use a fine-tuned language model; its assistant computes answers from the user's own records, with an optional external model, and the paper's judgement protocol is not applied.

2.2.3 What FINJARVIS takes from the base paper, and what it does not

Table 3: Relation of FINJARVIS to the base paper

Module	Base paper [1]	FINJARVIS (as built)	Status
Expense categorisation	Fine-tuned BERT, 50 categories, SMOTE, 80/20 split	TF-IDF (1-2-grams) + logistic regression, 11 categories + Other, 51 seed examples, retrain on correction	Same task, much simpler model
Budget recommendation	Q-learning agent, 3 savings levels	User-set monthly budgets, alerts at 75/90/100 %, rule-based advice, 6-component health score	Replaced by rules
Savings forecasting	LSTM on 12-month synthetic savings	Least-squares line through monthly expense totals, minimum 3 months	Expenses, not savings; linear
Anomaly detection	Isolation Forest, contamination 0.015	None (only the largest expense of a period is reported)	Not built
Interface	Streamlit	React + FastAPI, 13 pages, login, reports	Extended
Evidence	Metrics on unreleased data	38 tests, probe set, backtest, simulation, latency	Own measurements

FINJARVIS does not claim to match the base paper's accuracy figures: the data, the label set (11 against 50 categories) and the model all differ. It takes the paper's modular structure and pipeline order (categorise, budget, forecast, advise) and builds the lightest version that runs on an ordinary laptop with a short seed list, then measures how far it goes.

2.3 Expense Categorisation

Classifying transaction descriptions is a short-text classification problem. Classical systems weight words by TF-IDF [15] and feed them to a linear classifier from a standard library [14]; they are fast, need little data and can be inspected word by word. Pre-trained transformers such as BERT [8] read each word in the context of the whole text and are fine-tuned on labelled examples; FinBERT continues the pre-training on financial text [9]. The base paper uses BERT and reports that it beats keyword matching and classical learners [1]. The price is a labelled corpus large enough to fine-tune (the base paper uses 50 categories) and the computing power to train and serve it.

FINJARVIS takes the classical route because no labelled corpus of Indian bank and UPI narrations is available and the product must run on one CPU. It keeps the keyword table of the

first Streamlit version as a fallback. The cost is measured in Chapter 8: 100 % accuracy on text that resembles the seed phrases and 18.2 % on unseen merchants.

2.4 Expense Forecasting

Forecasting a person's monthly spending is a short time series problem: after six months there are six numbers. The forecasting literature compares simple methods (last value, mean, trend, exponential smoothing) with ARIMA-style models and with neural networks [18]. A well-known comparison of statistical and machine-learning forecasts over many series found that simple statistical methods were at least as accurate as complex learners and far cheaper [17]; the same literature warns that percentage errors misbehave when values are small and recommends reporting several error measures [16]. The base paper uses an LSTM [11] on a synthetic savings series and reports an R-squared of 0.63 [1]; FinSenseAI combines XGBoost and LSTM with SHAP explanations but states in its results section that the models are intended to be evaluated and gives no numbers [4].

FINJARVIS fits a straight line to the monthly totals, which is the simplest trend model, and refuses to predict with fewer than three months. Because the literature suggests that simple baselines are strong, the project compares the line with three baselines (mean of all history, mean of the last three months, last value) on the demo account (MAPE 28.4 % for the line, 21.4 % for the mean) and on 2,000 simulated users per setting; the result is given in Chapter 8.

2.5 Anomaly Detection and Budget Recommendation

The Isolation Forest isolates unusual points by random partitioning: points that need few splits get high anomaly scores [10], and no labelled fraud is needed. The base paper sets the expected share of anomalies (contamination) to 0.015 [1]; the leakage paper pairs the forest with K-Means clustering of small recurring purchases [5]. Budget recommendation by Q-learning [12] treats the budget level as an action chosen to maximise a reward, which needs a well-defined reward and many episodes of data. FINJARVIS builds neither: it reports the single largest expense of a period, lets the user set budgets, and raises alerts when a budget reaches 75, 90 and 100 %, with a rule-based suggestion to trim the top categories by 10 to 15 %. These are stated as gaps against objectives 4 and 5.

2.6 Recent Personal-Finance Systems

The remaining batch papers are close in aim. FinSenseAI [4] (IJISRT, July 2026) predicts future expenses with an XGBoost-LSTM ensemble explained by SHAP on a transaction table of unstated size; its results section lists RMSE, MAE, MAPE and R-squared as metrics without

reporting values. The leakage-detection paper [5] (IJRASET, April 2026) uses an Isolation Forest and K-Means to flag unexpected charges and group small purchases on synthetic data, and its abstract gives no metrics. The IEEE title [6] and the 2026 platform paper [7] could not be read or located and are not discussed. Financial-literacy work [19] supplies the motivation: planning and saving depend on seeing one's own money clearly.

2.7 Datasets and Algorithms Used in Related Work and in This Project

FINJARVIS uses no external data set. Its categoriser is trained on a seed list of 51 short descriptions in 11 categories (figure below). It is evaluated on a probe set of 198 descriptions written by the team before scoring, and the forecast on the six months of the fictional demo account and on simulated users. The algorithms used, with the source of each idea, are listed below.

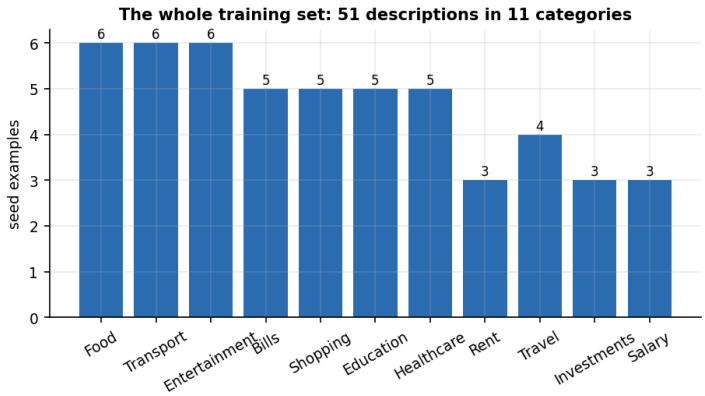


Figure 1: The 51-example seed list on which the shipped categoriser is trained (examples per category).

Table 4: Algorithms used in FINJARVIS and where each comes from

Algorithm	Used for	Source of the idea
TF-IDF word 1-2-grams + multinomial logistic regression	Expense categoriser	Term weighting [15]; scikit-learn [14]
Keyword table (substring match)	Fallback categoriser; comparison baseline	Original Streamlit version of the product
Least-squares line through monthly totals	Next-month and per-category forecast	Trend extrapolation [18]
Mean of history, mean of last 3, last value	Baselines in the forecast study	Simple baselines [18], [17]
Weighted sum of six clipped component scores	Financial health score	Developed in this project; weights chosen by the team
Rule routing over five intents	AI assistant	Developed in this project
bcrypt password hashing; HMAC-SHA256 signed tokens	Accounts and sessions	[20]
Cross-validation style probes (leave-one-out, probe tiers)	Evaluating the categoriser	[23]

2.8 Strengths and Limitations of Existing Research

Table 5: Comparison of the surveyed work

Work	Approach	Data	Reported result	Limitation
Kharat et al. (base paper)	Four modules: BERT expense categoriser (16 -> 50 classes, SMOTE), Q-learning budget recommender, LSTM savings forecaster, Isolation Forest anomaly detector; Streamlit UI	Own synthetic and real-world data labelled into 50 categories; synthetic savings series; synthetic outliers; 80/20 splits	BERT accuracy 92.3 %, precision 90.8 %, recall 91.5 %, F1 91.1 %; LSTM RMSE 137.2, MAE 89.5, R2 0.63; Isolation Forest precision 93 %, recall 94 %, F1 94 %	Synthetic or private data, no public release; reward and state definitions of the RL agent are only sketched; units of the LSTM errors are not stated
To (Mabu app)	Goal-Directed Design plus gamification for a mobile finance app for young adults; interviews, survey, card sorting, think-aloud and usability tests	Survey of respondents aged 18-30; closed card sort with 10 users; usability tests with 6 users	Design and usability study; no forecasting or classification model is evaluated	Small user samples; focuses on engagement and literacy, not on automated categorisation or forecasting
Theerthala	Data-generation framework that combines financial context with behavioural-finance studies to supervise a personal-finance LLM advisor (19k-sample reasoning set, Qwen-3-8B fine-tune)	19k synthetic reasoning samples, held-out split and a blind LLM-jury study	The 8B model is reported comparable to 14-32B baselines on factual accuracy, fluency and personalisation at about 80 % lower cost	Advice generated by a language model; judged by an LLM jury; no user data or transaction pipeline
Dilip and Janaki (FinSenseAI)	XGBoost + LSTM hybrid for expense prediction with SHAP explanations; budgeting recommendations	A real-world transaction table (date, category, amount, type, description); size not stated	The paper lists RMSE, MAE, MAPE and R2 as metrics but its results section states the models are 'intended to be evaluated'; no numeric results are given	No measured results in the text read; no deployed application described
Abiya et al. (leakage detection)	Two unsupervised layers: Isolation Forest flags unusual charges and K-Means groups frequent small purchases; subscription detection and end-of-month balance forecast	Synthetic transaction data (size not stated)	Reported to beat rule-based budgeting at finding hidden patterns; no metrics in the abstract	Synthetic data only; no numeric evaluation in the abstract
Batch survey item 2 (IDCIoT 2025)	AI for personal budget planning and future expense prediction (listed in the batch survey; IEEE Xplore record exists)	Not read	Not read	Not verified here
Batch survey item 3 (2026)	AI platform for expense tracking and budget automation (listed in the batch survey)	Not located	Not located	Not verified here

Strengths. The literature establishes that transaction text can be classified automatically with a pre-trained language model, that unsupervised detectors can flag unusual spending without labelled fraud, that forecasts and recommendations are more useful when explained, that financial advice must be personalised, and that a finance application must be designed around the user's goals.

Limitations. Most sources evaluate on synthetic or private data; two report no numeric result at all; none releases its data; none describes a tested, multi-user web service; and none checks how

a simple trend or a small classifier behaves on inputs it has not seen. Some report percentage improvements over weak baselines (manual review, rule-based matching) rather than over a plain average or a keyword table, and none reports response time, defects or tests of an operating service.

2.9 Research Gap

Every related system either evaluates models on synthetic or unreleased data, or reports no number at all; none of the papers read describes an application that a person can run, log in to and fill with their own transactions, and none reports how a simple trend or categoriser behaves on inputs it has not seen. This project delivers a runnable, affordable web platform with the categorisation, forecasting and advice modules in a simple, inspectable form, and measures where each simple module breaks.

The project's answer has three parts. It builds the platform as one running, tested system with accounts and a demo. It builds the simplest inspectable version of the base paper's categorisation and forecasting modules and a rule-based version of its advice, and says plainly that anomaly detection and reinforcement learning are not built. And it measures the weak points: the categoriser scores 18.2 % on unseen merchants, learning from corrections helps repeat merchants but barely helps new ones, and a linear trend needs about seven months of history to beat a mean.

CHAPTER 3: FEASIBILITY STUDY

3.1 Technical Feasibility

Can the categoriser, forecast, health score, assistant and web console be built with tools that an ordinary desktop supports, and are they fast enough to feel interactive? Each part was built and measured; the table lists the evidence.

Table 6: Technical feasibility: components and the evidence for each

Component	Technology	Evidence that it is feasible
Web API	FastAPI, pydantic validation, one bearer-token dependency	33 routes built; 49 of 49 smoke checks pass; dashboard answers in 67.2 ms (median)
Data store	SQLite file through SQLAlchemy 2 [22]	6 tables, cascade delete, per-user scoping; transaction list 12.7 ms (median)
Categoriser	TF-IDF + logistic regression (scikit-learn) [14]	Trains in 5.2 ms and predicts in 0.84 ms on one CPU thread; 71.7 % probe accuracy
Forecast and health score	NumPy least squares; pure-Python weighted sum	No training; deterministic; 5,000 random profiles scored without error
Passwords and sessions	bcrypt [20]; HMAC-SHA256 tokens	Login takes 305 ms (median) because bcrypt is deliberately slow; tampered tokens rejected in tests
Reports	ReportLab (PDF), OpenPyXL (Excel), CSV	PDF 136 ms, Excel 209 ms, CSV 18 ms (median)
Front end	React 18, TypeScript, Vite, Tailwind CSS, Recharts, TanStack Query	13 pages and 19 screens captured at 1440 x 900
Import	pandas file reading with row-level checks	1,000 rows with 84 bad rows: all 84 skipped, 916 imported and categorised

Three risks were examined. **Data volume**: no corpus of labelled narrations is available, so the model is small and its weakness is measured. **Library drift**: the pickled model was trained under scikit-learn 1.5.0 and loads under 1.9.1 with a warning; retraining takes milliseconds. **Hashing library**: passlib needs bcrypt 4.0.1 (bcrypt 5 breaks it), which is pinned. The system is technically feasible on a single desktop with no GPU and no paid service.

3.2 Economic Feasibility

The whole stack is open source, so licence cost is zero. The remaining costs are the team's time and ordinary hardware.

Table 7: Cost items

Item	Cost for this project	Note
Software licences	None	Python, FastAPI, SQLAlchemy, scikit-learn, React, Vite and Tailwind are free and open source
Cloud or hosting	None	Runs locally; a free tier of a platform host would suffice for a demonstration (a Procfile and railway.json are in the repository, not exercised here)
Data	None	Seed list and demo data are the team's own; no data purchase
Optional language model	Pay-per-use if enabled	Only if the user sets an API key; not used in any measurement
Optional price refresh	None	yfinance reads public prices; needs internet; not exercised here

Item	Cost for this project	Note
Hardware	Existing desktop	Intel Core i5-12500, 15.7 GB, no GPU
Team effort	Four students over the project term	Time cost only

For a user, the platform replaces a commercial subscription with a free application whose data stays in a local file; the cost of a hosted service was not measured. The batch abstract names affordability as a goal, and the stack meets it.

3.3 Operational Feasibility

A user needs only a browser. Setup is one command (run.bat) that builds a virtual environment, installs the dependencies, and starts the API and the web application. A demo account (demo / Demo@1234) is created on first start with fictional data for six months, so every screen has content. Daily use is light: adding a transaction takes 15 ms with automatic categorisation, and the dashboard loads in about 67 ms. Entering transactions by hand remains the main effort; the CSV and Excel import, with a preview that lists skipped rows, removes most of it for users who can export statements.

The risks to operation are those of the design: a user who accepts every automatic category will see unseen merchants labelled Transport, and a user with fewer than three months of history sees no forecast and a message explaining why. The assistant and the health score carry a disclaimer that they are not financial advice.

3.4 Schedule Feasibility

Table 8: Project phases

Phase	Work	Output
1. Study	Read the base paper, survey related work, fix objectives	Batch abstract, survey, objectives
2. Core application	Database, services, categoriser, forecast, health score, Streamlit interface	Original working application (app.py, ui/)
3. Web platform	FastAPI layer over the same services, React console, demo account, run.bat	33 routes, 13 pages, smoke test
4. Evaluation	Probe set, backtests, simulations, import and latency measurements, tests	experiments/metrics.json, api_latency.json
5. Documentation	Diagrams, three decks, chapter documents, report, paper	The submission set

The phases were sequential except that evaluation scripts were written alongside the web platform. Each module is small: the Python code base is about 2,400 lines and the tests run in under twelve seconds.

3.5 Risk Analysis

Table 9: Project risks and mitigation

Risk	Likelihood	Effect	Mitigation in the project

Risk	Likelihood	Effect	Mitigation in the project
Training data too small for the categoriser	Certain	Unseen merchants mislabelled	Measure it (probe tiers), keep a keyword fallback, learn from corrections, document the gap
No real transaction data for evaluation	Certain	Results cannot be generalised	Use fictional demo data and a team-written probe set; label every result as such
Library version drift (scikit-learn, bcrypt)	Observed	Warnings or broken hashing	Pin bcrypt 4.0.1; retrain the model when the pickle is stale
Forecast over-fits a handful of months	Likely	Wrong next-month figure	Minimum of three months; compare with baselines; report MAPE
Optional language model unavailable	Likely	Free-form answers missing	Rule layer answers without it; fixed help text otherwise
SQLite file lost or corrupted	Low	User records lost	Single file, easy to back up; tests use a temp database per test

3.6 Advantages and Constraints

Table 10: Advantages and constraints of the project

Advantages	Constraints
Free, local, inspectable models; every decision can be traced to a rule or a word weight	A 51-example training set limits the categoriser to phrases it has seen
One account, one file, one command to start	Linear trend needs about seven months of history to beat a plain mean
Health score with published weights and a named weakest area	Weights are chosen by the team and not fitted to outcomes
Own tests, probe set and measurements reported with their limits	Probe set is team-written, not an independent benchmark
Optional language-model layer without making it a dependency	Anomaly detection and reinforcement-learning budgeting are not implemented
Responsive React interface, light and dark theme, reports in three formats	Single user per process, SQLite, no migrations, no loan tracker or scheduler

CHAPTER 4: SYSTEM REQUIREMENTS AND ANALYSIS

4.1 Existing System

Before this project the team's starting point was the original FINJARVIS application, a Streamlit program with ten pages (dashboard, transactions, analytics, budgets, goals, investments, assistant, reports, notifications, profile) over the same service layer and SQLite file. It already categorised transactions, forecast the next month and scored financial health, but it had no HTTP interface, so it could be driven only through Streamlit's own pages.

More broadly, personal finance is managed with four kinds of tools. **Paper and spreadsheets** need manual entry and formulas the user writes. **Bank and wallet apps** show a statement and sometimes a category chart, but only for that provider's accounts. **Commercial budgeting apps** add categorisation and budgets, usually behind a subscription with the data held by the vendor. **Research prototypes** such as the base paper [1] combine several models but run as notebooks or Streamlit demonstrations on unreleased data.

4.2 Limitations of the Existing System

- Manual entry and manual categorisation make tracking tedious, and most users stop within weeks.
- Reports describe the past; few tools forecast the next month or check budgets against a forecast.
- Advice is generic and not tied to the user's income, commitments or goals.
- Commercial tools cost money and keep the data on the vendor's servers.
- Research prototypes cannot be run by an ordinary user, have no authentication and report metrics on data that cannot be checked.
- The original Streamlit version has no API and a fixed layout.

4.3 Proposed System

FINJARVIS keeps the service layer and adds a FastAPI application and a React console. A user signs up or logs in, enters or imports transactions, and sees the dashboard. Behind the screens, a categoriser labels each description, budgets raise alerts, a forecast estimates the next month, a health score summarises six components, and the assistant answers from the user's records. The back end holds no business logic of its own: each route calls the same service function that the Streamlit page calls.

Table 11: Modules of the proposed system

Module	File	Responsibility
API	backend/main.py	33 routes, request validation, bearer-token dependency, CORS for the web origin
Authentication	auth/authentication.py, utils/security.py	Sign-up, login by username or e-mail, bcrypt hashing, password rules
Transactions	services/transaction_service.py	Add, edit, delete, search, filter; auto-categorise; learn from correction
Import	services/data_import_service.py	CSV and Excel parsing, row validation, preview and commit
Categoriser	ml/expense_classifier.py	TF-IDF + logistic regression, keyword fallback, retraining
Forecast	ml/spending_prediction.py	Linear trend over monthly totals, per-category forecast, budget-overrun check
Health score	ml/financial_health.py	Six weighted components, bands, weakest-area explanation
Budgets and goals	services/budget_service.py, goal_service.py	Monthly limits, 75/90/100 % alerts, savings progress
Investments	services/investment_service.py	Manual holdings, optional live price refresh
Analytics and assistant	services/analytics_service.py, ai_service.py	Totals, trend, category split; rule-routed answers, insights, optional language model
Reports	services/report_service.py	PDF, Excel and CSV downloads
Database	database/models.py, database.py	6 SQLAlchemy models over one SQLite file
Front end	frontend/src	13 pages, charts, forms, demo-login button

4.4 Advantages of the Proposed System

Table 12: Advantages and the evidence for each

Advantage	Evidence
Automatic categorisation	71.7 % probe accuracy against 51.5 % for the keyword table; 97 % on statement-style strings
Learns from a correction	Repeat-merchant test: 18.2 % before, 100.0 % after one correction (198 tests)
Forward-looking	Forecast panel and per-category forecast; refuses with under 3 months
Explainable health score	Six components with weights; 5,000-profile check shows no violation when the savings rate rises
Safe by default	bcrypt, password rules, signed tokens, per-user scoping; a tampered token and a protected route without a token were rejected in the smoke test
Fast	Dashboard 67 ms, analytics 72 ms (medians, single user)
Free and local	No licence, no cloud, one SQLite file

4.5 Data Requirements

The system stores six kinds of record and needs the following inputs. All are entered by the user or generated by the demo loader; nothing is fetched from a bank.

Table 13: Data requirements

Data	Fields	Source	Notes
User	name, e-mail, username, password hash, currency, monthly income, risk profile, savings target	Sign-up form, profile page	E-mail and username unique
Transaction	date, description, category, amount, type, payment method, notes	Form, CSV or Excel import, demo loader	Category suggested by the model
Budget	category, amount, month, year	Budgets page	One per category and month

Data	Fields	Source	Notes
Goal	name, target amount, current amount, target date, status	Goals page	Status active, completed or abandoned
Investment	name, symbol, type, quantity, purchase price, current price, date	Investments page	Price refresh optional
Notification	message, type, created time, read flag	Written by budget checks	De-duplicated by message text

4.6 Functional Requirements

Table 14: Functional requirements and how each is met

ID	Requirement	Implemented by	Verified by
FR1	Register with name, e-mail, username and a strong password	POST /auth/signup, validators.py	tests/test_auth.py; smoke (weak password rejected)
FR2	Log in by username or e-mail and receive a token	POST /auth/login, authentication.py	test_auth.py; smoke
FR3	Every data route requires a valid token and returns only the caller's data	current_user dependency; user_id on every table	Smoke: 401 without token, new-user isolation
FR4	Add, edit, delete, search and filter transactions	transaction_service.py; /transactions	test_transactions.py; smoke
FR5	Suggest a category from a description and learn from corrections	expense_classifier.py; /transactions/predict-category	test_ml.py; probe set
FR6	Import CSV or Excel with preview and row-level warnings	data_import_service.py; /transactions/import/*	Smoke; 1,000-row import test
FR7	Set monthly budgets and raise alerts at 75, 90 and 100 %	budget_service.py; notifications	test_budget.py; smoke
FR8	Create savings goals and update progress	goal_service.py; /goals	Smoke
FR9	Record investments and refresh prices when online	investment_service.py; /investments	Smoke (refresh not tested)
FR10	Show totals, monthly trend and category split for a period	analytics_service.py; /analytics, /dashboard	Smoke (bad range rejected)
FR11	Forecast next month and per-category spending	spending_prediction.py	test_ml.py; backtest
FR12	Compute a 0-100 health score with its components	financial_health.py	test_financial_health.py; sensitivity study
FR13	Answer questions from the user's data with a disclaimer	ai_service.py; /assistant/ask	26-question test
FR14	Download PDF, Excel and CSV reports	report_service.py; /reports/{kind}	Smoke
FR15	List notifications and mark all read	/notifications	Smoke
FR16	Edit profile (income, currency, risk profile, savings target) and load demo data	/profile, /profile/demo-data	Smoke (duplicate e-mail rejected)

Each requirement is traced to the route or module that meets it and to the test or measurement that checks it. The automated tests (38 in five files: auth 9, budget 6, financial health 6, ML 9, transactions 8) and the 49-check smoke script together exercise FR1 to FR16 except the live price refresh in FR9 and the optional language-model layer in FR13.

4.7 Non-Functional Requirements

Table 15: Non-functional requirements, measured values and status

ID	Category	Requirement	Measured or observed	Status
NFR1	Performance	Common screens load in well under a second	Dashboard 67 ms, analytics 72 ms, transaction list 13 ms (medians, 30 calls)	Met

ID	Category	Requirement	Measured or observed	Status
NFR2	Performance	Bulk import stays usable	Preview of 100 rows 667 ms; commit 35.2 rows/s (one session per row)	Partly met
NFR3	Security	Passwords hashed, weak passwords refused, routes require a token	bcrypt (login 305 ms); password rules tested; 401 without token	Met
NFR4	Security	No default secrets, revocable sessions	Default development secret if APP_SECRET_KEY unset; 7-day tokens with no revocation or refresh	Not met
NFR5	Correctness	Reported numbers equal direct sums of the data	Assistant food total INR 17,365 equals a direct sum; top category matches groupby	Met
NFR6	Reliability	Decline instead of guess	Forecast refuses under 3 months; assistant declines 6 of 6 off-topic questions; categoriser never abstains	Partly met
NFR7	Usability	Clear states, light and dark theme, responsive layout	13 pages; 19 screens; theme toggle; no user study	Met, no user study
NFR8	Portability	Start with one command on Windows	run.bat builds .venv and starts both servers	Met
NFR9	Maintainability	Small, tested code base	About 2,400 lines of Python, 38 tests in 11.5 s	Met
NFR10	Scalability	Serve more than one user at a time	One process and SQLite; not measured	Not evaluated

The status column is deliberately blunt: two requirements are not met or not evaluated and two are met only in part (see Chapters 7 and 8). Met means a measurement or test supports it.

4.8 Hardware Requirements

The system was developed and measured on one desktop; smaller machines were not tested, so no lower bound is claimed. No GPU is needed or used.

Table 16: Hardware used for development and measurement

Item	Configuration
Processor	12th Gen Intel Core i5-12500, 6 cores, 12 threads
Memory	15.7 GB
Graphics	Intel UHD 770 integrated; no discrete GPU, none used
Storage	Local disk; the database is a single file, finjarvis.db
Operating system	Windows 11 Pro

4.9 Software Requirements

Table 17: Software used

Layer	Software and version
Language and runtime	Python 3.11.9 (the code needs 3.11 or newer); Node.js 20 or newer for the front end (24.19.0 used)
Back end	FastAPI 0.110 or newer, uvicorn, SQLAlchemy 2.0, pydantic 2, python-multipart
Data and ML	pandas 2.2, NumPy 1.26, scikit-learn 1.4.2 or newer (the measurements ran under 1.9.1)
Security and reports	passlib with bcrypt 4.0.1, python-dotenv, ReportLab, OpenPyXL
Optional	yfinance (price refresh), anthropic (language-model layer)
Front end	React 18.3, TypeScript 5.9, Vite 8, Tailwind CSS 4, Recharts 3, TanStack Query 5, Axios, react-router 7
Database	SQLite file finjarvis.db (replaceable through FINJARVIS_DB)
Testing and measurement	pytest 8.2, matplotlib, Playwright with Google Chrome for the screenshots

4.10 User Roles and System Constraints

Table 18: User roles

Role	Can do	Cannot do
Visitor (not logged in)	See the landing page and public showcase; sign up or log in; use the demo-login button	Read or write any financial data
User	Manage own transactions, budgets, goals, investments, profile, reports and notifications; ask the assistant	See another user's data (every query is filtered by user_id)
Demo user	Same as a user, on fictional data (demo / Demo@1234)	Nothing is withheld; the data is fictional and can be reset

There is a single privilege level: the system has no administrator role. The constraints below are enforced by the validators and the schemas.

- A username has 3 to 30 letters, digits or underscores; a password has at least 8 characters with an upper-case letter, a lower-case letter, a digit and a special character.
- A transaction needs a valid ISO date, a positive amount and a description; free text is trimmed and capped (255 characters for a description).
- An import file needs the columns date, description, amount, category and type; rows with an unparsable date, a non-numeric or non-positive amount or a blank description are skipped with a warning.
- The forecast needs at least 3 months of expense history.
- A session token is signed with HMAC-SHA256 and lasts 7 days; the development secret must be replaced through APP_SECRET_KEY before real use.
- One backend process serves all requests, and CORS allows only the web origin (port 5411).

CHAPTER 5: SYSTEM DESIGN

This chapter describes the architecture and workflow, the data flow, the UML and database design, the core algorithms, the modules, the API and the user interface. The diagrams were drawn from the code in `backend/`, `services/`, `ml/`, `database/` and `frontend/src/`. FINJARVIS is a single-role, multi-user web application: every registered user sees only their own records, and all state lives in one SQLite file. The design is stated as built, including where it differs from the batch abstract (Flask and MySQL, loan tracking, scheduled alerts).

5.1 Design goals and architecture

Five goals shaped the design. **Every number on screen is computed from the user's own records**: the dashboard, the forecast, the health score and the assistant call the same service functions over the same tables. **The models are simple enough to inspect**: a TF-IDF and logistic-regression categoriser, a least-squares trend and a six-term weighted sum can each be re-computed by hand, which is why none of the heavier models of the base paper [1] was built. **One code path serves two front ends**: the Streamlit pages and the React console call the same `services/` and `ml/` modules, and the FastAPI layer holds no business rules. **Per-user isolation**: every table carries a `user_id` and every service query filters on it. **Zero-configuration deployment**: SQLite, a bundled sample file and a seeded demo account run on a laptop with one launcher script.

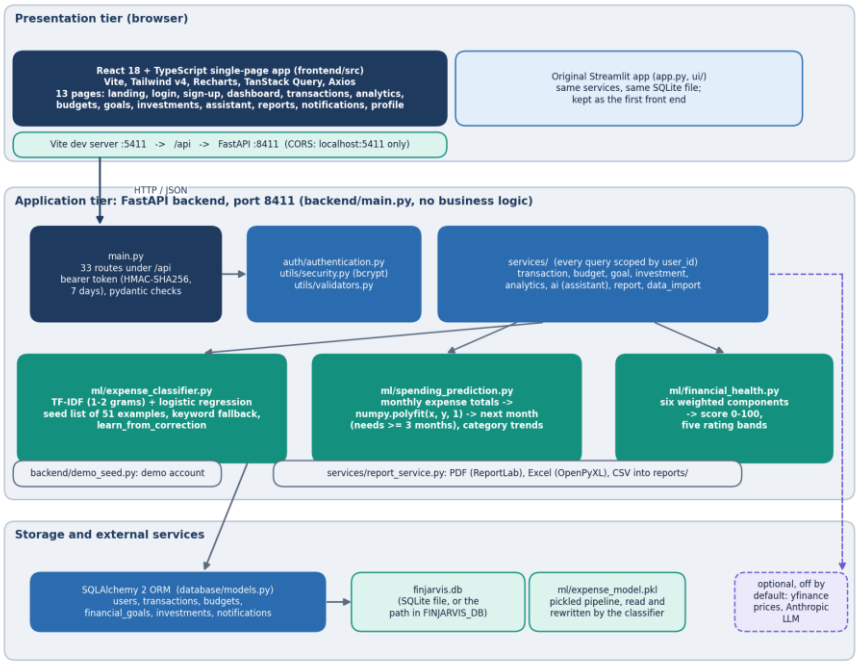


Figure 2: Layered architecture of FINJARVIS: React single-page app, Vite proxy, FastAPI application with 33 routes, service layer, ML modules, SQLAlchemy and SQLite.

The React console on port 5411 reaches the FastAPI application on 127.0.0.1:8411 through the Vite development proxy (`/api` is forwarded), so the browser sees one origin; cross-origin calls are allowed from `localhost:5411` and `127.0.0.1:5411` only. The application exposes 33 routes in `backend/main.py`. Each route validates input with pydantic, resolves the caller through the `current_user` dependency, calls one service or ML function and returns JSON after a `clean` step that converts NumPy and pandas values. The rules live in `services/` and `ml/`, so they can be tested without HTTP. The style is a stateless REST interface [21] over one relational store accessed through an object-relational mapper [22].

The deliberate gaps are worth stating. There is one backend process and one SQLite file, so there is no horizontal scaling. The categoriser is a single pickled file, `ml/expense_model.pkl`, shared by every account: `learn_from_correction` retrains on the seed list plus one user's history and overwrites it, so one user's correction changes every user's predictions (read from the code, not exercised in a multi-user test). Tokens are home-made HMAC bearer tokens with a 7-day lifetime and no revocation, and there is no scheduler: budget alerts are raised when the dashboard or the notification list is requested.

5.2 High-level workflow

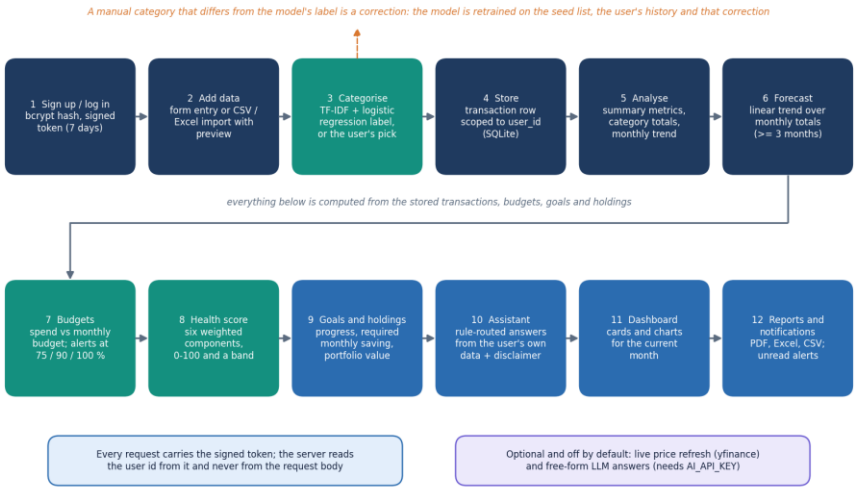


Figure 3: High-level workflow: record or import a transaction, categorise it, update budgets, forecast, score and answer questions.

The workflow has six stages and every stage maps to a route and a page. **Record**: a transaction is typed in or imported from CSV or Excel (preview, then commit). **Categorise**: with 'Auto-detect' the classifier labels the description; a manual category that differs from the model's prediction is stored as chosen and also used to retrain the model, so the classifier is called for every new transaction. **Watch**: budgets are compared with the month's spending and a notification is written when a category crosses 75, 90 or 100 % of its budget. **Forecast**: a

straight line over the monthly expense totals, refused below three months of history. **Score**: the dashboard turns the month's metrics, budgets, holdings and goals into the 0-100 health score. **Ask and export**: the assistant answers from the user's records, and the reports routes produce PDF, Excel or CSV files.

5.3 Data flow diagrams

5.3.1 Level 0: context

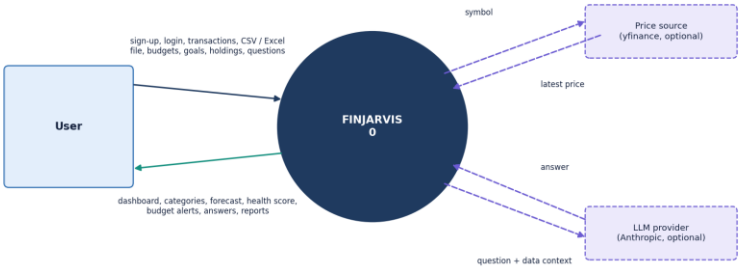


Figure 4: Data flow diagram, level 0: FINJARVIS as one process between the user, the optional LLM provider and the market-data source.

At level 0 FINJARVIS is one process with three external entities. The **user** supplies credentials, transactions, files, budgets, goals, holdings and questions, and receives the dashboard, forecasts, alerts, answers and report files. The **LLM provider** is optional: with `AI_API_KEY` set, a question the rule layer cannot match is sent with a short summary of the user's figures; without a key nothing leaves the machine. The **market-data source** (`yfinance`) is also optional and used only when the user refreshes a holding's price.

5.3.2 Level 1: main processes

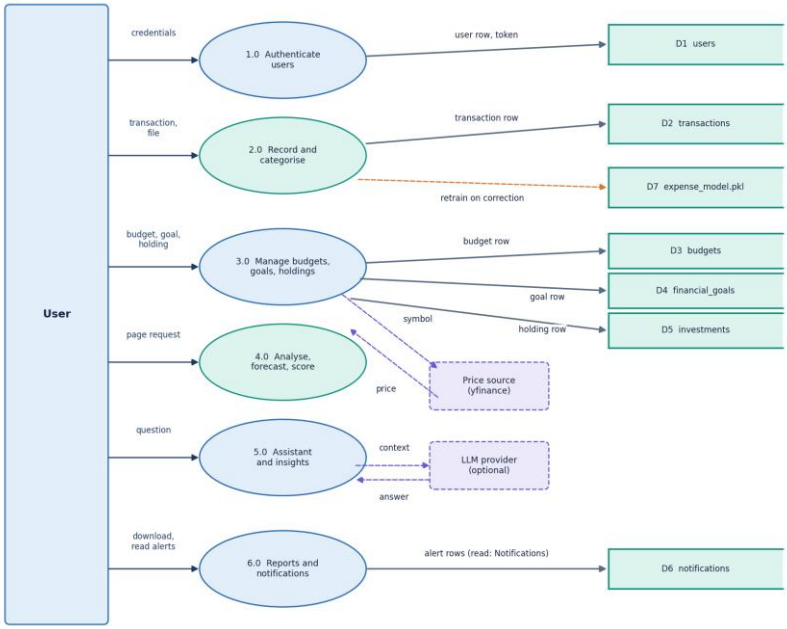


Figure 5: Data flow diagram, level 1: the main processes and the six data stores.

Level 1 shows the processes that read and write the six data stores. Authentication uses the users store; transaction handling writes transactions and calls the categoriser; budget monitoring reads budgets and transactions and writes notifications; analytics reads transactions and holdings and calls the forecast and health modules; the assistant reads the same stores through the analytics functions; reporting writes files only.

5.3.3 Level 2: adding a transaction

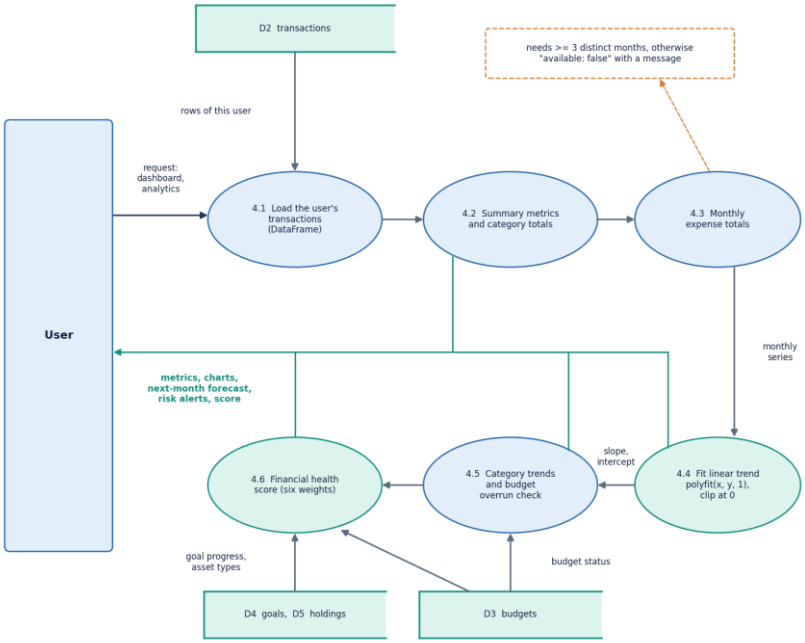


Figure 6: Data flow diagram, level 2: adding a transaction (validation, categorisation, learning, storage).

The transaction process holds the main input rules. The description must not be blank and the amount must be above zero (HTTP 400 otherwise). 'Auto-detect' or an empty category takes the model's label, any other value is kept. If the user's value differs from the model's prediction, the model is retrained on the seed plus the user's history plus the new pair; a retraining failure is swallowed so that it never blocks the insert. The file import follows the same path row by row, except that a bad row is skipped with a warning instead of failing the file.

5.4 UML design

5.4.1 Class diagram

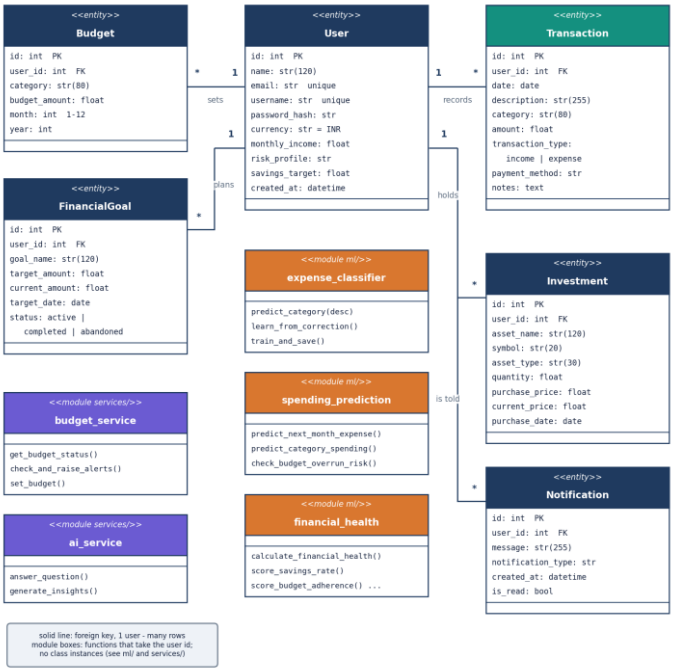


Figure 7: UML class diagram: the six ORM classes, the service functions and the three ML modules.

Six SQLAlchemy classes map to six tables [22]. Behaviour is kept out of the model classes and placed in `services/` as functions that open a session through `get_session`, which commits on success and rolls back on any exception. The only other classes are the pydantic input models of the API (`SignupIn`, `LoginIn`, `TransactionIn`, `BudgetIn`, `GoalIn`, `InvestmentIn`, `AskIn` and others), which carry type and range limits, and the small `AuthResult` dataclass. The ML modules are plain functions; the categoriser holds a scikit-learn `Pipeline` of a `TfidfVectorizer` and a `LogisticRegression`.

5.4.2 Sequence diagram

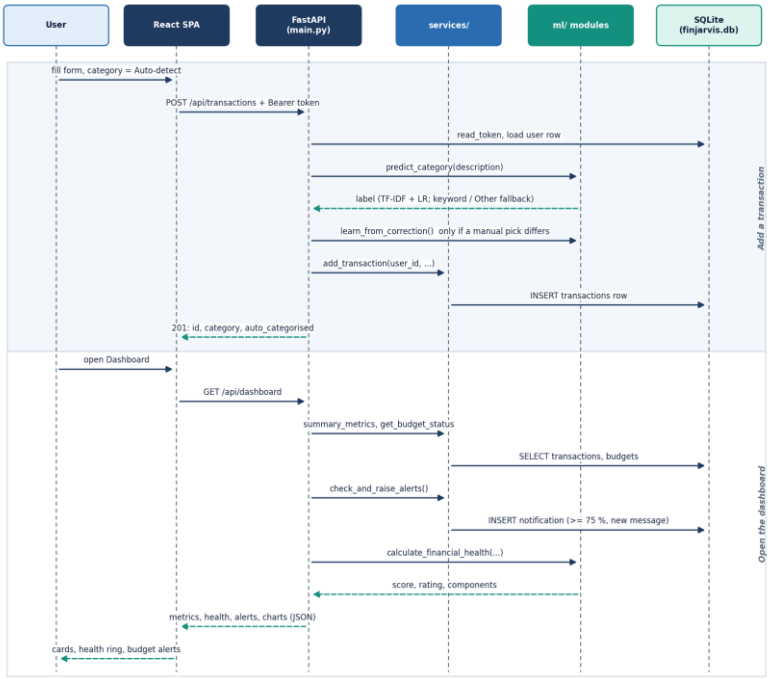


Figure 8: Sequence diagram of the main flow: login, add a transaction with auto-categorisation, then read the dashboard.

The browser posts the login form; the server checks the bcrypt hash and returns a signed token that the console attaches to every later call. A new transaction passes the token check, the field validation, the classifier call and, when the user overrides the label, the retraining attempt. The dashboard request then recomputes the month's metrics, budget status, health score and chart data in one call and raises any budget alert not yet raised.

5.4.3 Use-case diagram

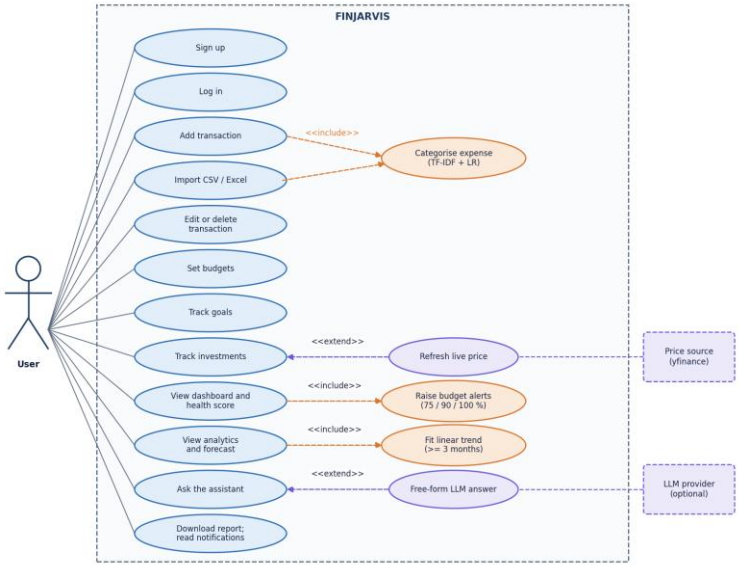


Figure 9: Use-case diagram: the registered user, the demo visitor and the optional external services.

There is one human role, the registered user, and one visitor role: a person without an account can open the landing and login pages and sign in to the fictional demo account. A registered user can manage transactions, import files, set budgets and goals, record holdings, view analytics, ask the assistant, download reports and read notifications. The LLM provider and market-data source are secondary actors. There is no administrator role.

5.5 Database design

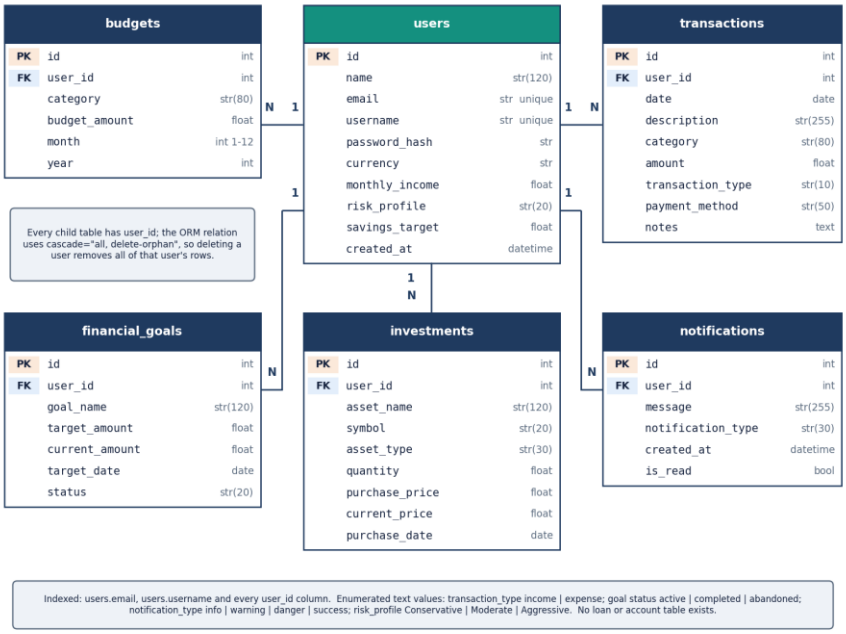


Figure 10: Entity-relationship diagram of the six tables.

The schema has six tables in one SQLite file, `finjarvis.db`, created at start-up by `init_db`; the path can be redirected with `FINJARVIS_DB` (used by the tests and screenshot scripts). A user has many transactions, budgets, goals, investments and notifications; each child row has an indexed, non-null `user_id` foreign key and is removed with its user through the ORM cascade (no route deletes accounts).

Table 19: The six tables and their columns.

Table	Columns	Keys, defaults
users	id, name(120), email(255), username(80), password_hash(255), currency, monthly_income, risk_profile, savings_target, created_at	PK id; unique and indexed email and username; currency INR; risk profile Moderate
transactions	id, user_id, date, description(255), category(80), amount, transaction_type, payment_method, notes	PK id; FK user_id (indexed); type income or expense; payment method Other
budgets	id, user_id, category(80), budget_amount, month, year	PK id; FK user_id (indexed); month 1-12; one row per user, category, month and year is enforced in code (`set_budget` updates the existing row), not by a constraint

Table	Columns	Keys, defaults
financial_goals	id, user_id, goal_name(120), target_amount, current_amount, target_date, status	PK id; FK user_id (indexed); status active, completed or abandoned; current amount 0
investments	id, user_id, asset_name(120), symbol(20), asset_type(30), quantity, purchase_price, current_price, purchase_date	PK id; FK user_id (indexed); asset types Stock, Mutual Fund, ETF, Gold, Fixed Deposit, Other
notifications	id, user_id, message(255), notification_type, created_at, is_read	PK id; FK user_id (indexed); type info, warning, danger or success; unread by default

Three design points deserve a note. Money is stored as floating-point numbers, adequate for a display tool but not for ledger accounting. Monthly queries use string bounds such as `YYYY-MM-31`, which SQLite compares correctly but a stricter database would reject for shorter months. Alerts are de-duplicated by identical message text, so a budget that crosses the same threshold again in a later month raises no new alert.

5.6 Core algorithm design

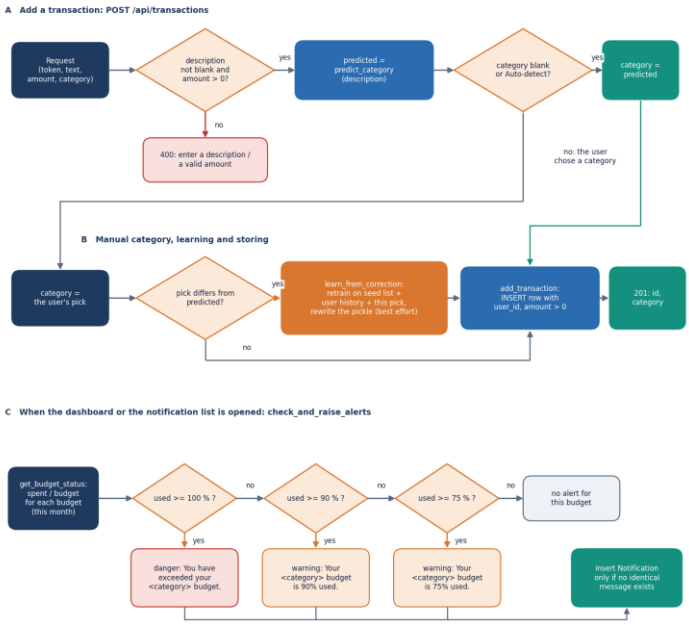


Figure 11: Flowchart of the core algorithms: categoriser, forecast, health score and budget alerts.

Three small algorithms carry the intelligence of the product: the **categoriser** (a description to one of eleven labels), the **forecast** (monthly expense totals to a next-month figure) and the **health score** (the month's records to a 0-100 number and the weakest area). The flowchart shows how routes call them; Chapter 6 gives formulas and parameters.

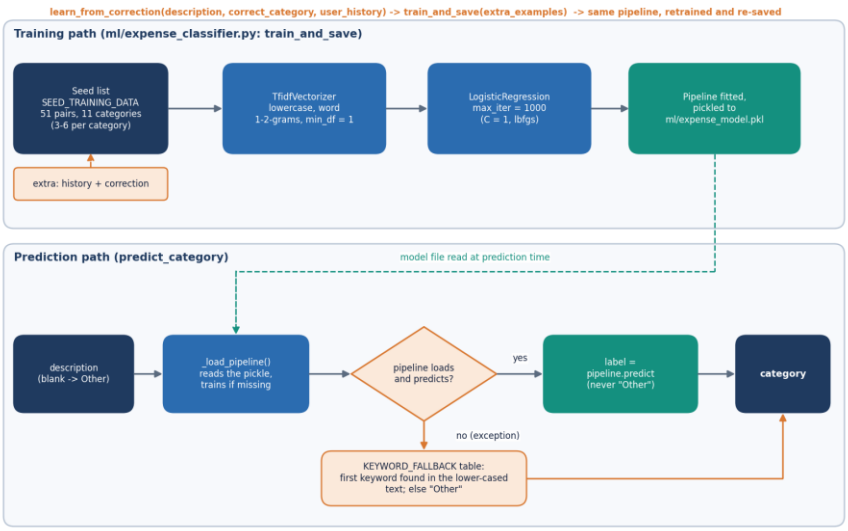


Figure 12: Categoriser pipeline: description, TF-IDF, logistic regression, label, optional correction and retraining.

The categoriser lowercases the text and builds word unigrams and bigrams; TF-IDF weighting [15] gives each term a weight that falls with how many descriptions contain it; a logistic-regression classifier [14] maps the weights to a probability per label and the highest wins. The prediction order is fixed: model, then a keyword table if the model fails, then 'Other'; a model trained on 51 examples never outputs 'Other'. The design choice that matters most is text with no known words: the feature vector is all zeros and the label comes from the intercepts alone. Chapter 8 measures what that costs.

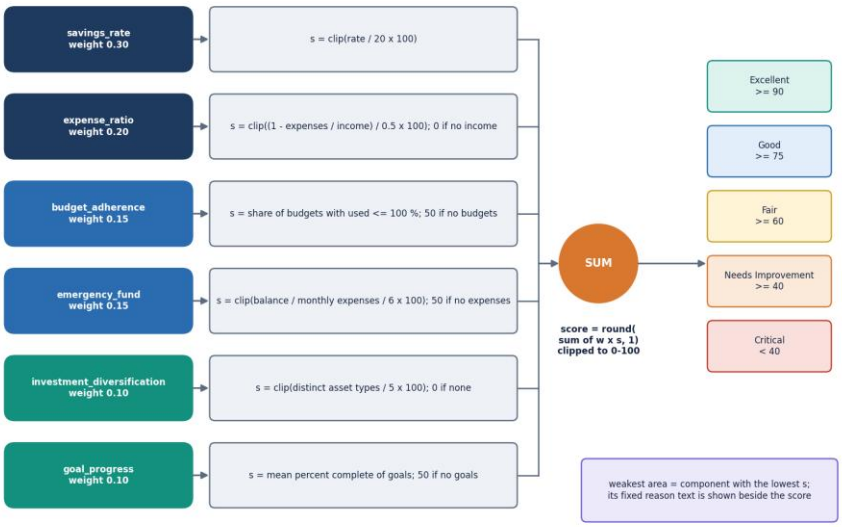


Figure 13: Financial health score model: six weighted components, clipped sum, rating band.

The health score is a weighted sum of six sub-scores, each clipped to 0-100: savings rate (weight 0.30), expense ratio (0.20), budget adherence (0.15), emergency fund (0.15), investment diversification (0.10) and goal progress (0.10). The weights are chosen by the team,

not fitted to data. The result carries a rating band (Excellent from 90, Good from 75, Fair from 60, Needs Improvement from 40, otherwise Critical) and the name and reason of the lowest component, so the number is always explained. The forecast follows the same philosophy: a straight line with an explicit refusal below three months is more honest than a deeper model that would fit noise; Chapter 8 tests that claim and shows the refusal threshold is too low for the method.

5.7 Module design

Each file has one reason to change. Line counts were measured from the repository when this report was built.

Table 20: Modules, sizes and responsibilities.

Module	Lines	Responsibility
`backend/main.py`	666	FastAPI application: 33 routes, token helpers, pydantic input models, `current_user` dependency, JSON clean-up
`backend/demo_seed.py`	138	Demo account and demo-data loader (shifted sample CSV plus synthetic earlier months)
`auth/authentication.py`	82	`signup` and `login` with validation and duplicate checks
`utils/security.py`, `utils/validators.py`	56	bcrypt hashing through passlib; e-mail, username, password-strength and amount validators
`database/models.py`, `database/database.py`	110	Six SQLAlchemy models; SQLite engine, session factory and `get_session` context manager
`services/transaction_service.py`	94	Add, edit, delete and filter transactions, every query scoped by `user_id`
`services/budget_service.py`	96	Budget upsert, usage per category, 75/90/100 % alert notifications
`services/goal_service.py`, `services/investment_service.py`	68	Goals with required monthly saving; holdings, portfolio profit and loss, optional price refresh
`services/analytics_service.py`	100	Monthly metrics, category totals, monthly trend, range overview
`services/ai_service.py`	227	Rule-routed assistant (five handlers), insights, optional LLM layer, disclaimer
`services/data_import_service.py`, `services/report_service.py`	84	CSV/Excel import with row-level warnings; PDF, Excel and CSV reports
`ml/expense_classifier.py`	115	TF-IDF and logistic-regression categoriser, 51-pair seed, keyword fallback, `learn_from_correction`
`ml/spending_prediction.py`	91	Linear-trend forecast of next month's expenses, per-category forecast, budget-overrun check
`ml/financial_health.py`	107	Six sub-scores, weights, rating bands and weakest-area explanation
`scripts/smoke_test.py`	170	49-check end-to-end test against a throw-away database
`scripts/run_experiments.py`, `scripts/measure_api.py`	354	Offline experiments (categoriser, forecast, health score, assistant, import) and API latency
`frontend/src/`	6,959	React 18 and TypeScript console: 13 pages plus a not-found page, UI kit, charts, API client, auth and theme providers
`app.py`, `ui/`	938	The original Streamlit front end over the same services (kept, not extended)
`tests/`	359	38 pytest tests (auth 9, budget 6, financial health 6, ML 9, transactions 8)

5.8 API design

All routes live under `/api`; reads are GET, actions POST, edits PUT and removals DELETE. Every route except `health`, `public/showcase`, `auth/signup` and `auth/login` needs an

`Authorization: Bearer` token. Failures return the FastAPI `detail` string, which the console shows verbatim, so messages are written for the user. 401 means the token is missing, tampered or expired (the console returns to login), 404 means the object does not exist or belongs to another user, 409 a duplicate e-mail or username, 400 a well-formed but unacceptable request, and 422 failed type validation.

Table 21: The 33 routes of the FINJARVIS API.

Method	Path	Auth	Purpose	Errors
GET	/api/health	-	Service status and whether the LLM key is set	-
GET	/api/public/showcase	-	Landing-page figures from the fictional demo account, plus one assistant answer	404 no demo account
POST	/api/auth/signup	-	Validate, hash the password with bcrypt, create the account	400 validation; 409 duplicate
POST	/api/auth/login	-	Username or e-mail plus password; returns a signed token and the user	401
GET	/api/auth/me	token	The signed-in user's profile	401
GET	/api/meta	token	Categories, payment methods, asset types, currencies, risk profiles, goal presets	401
PUT, POST	/api/profile, /api/profile/demo-data	token	Update the profile; load the shifted sample data, budgets, goals and holdings	400; 409; 422
GET	/api/dashboard	token	Month metrics, health score, budget alerts and six chart datasets	401
GET, POST	/api/transactions	token	Filter, sort and total; create ('Auto-detect' uses the classifier, a differing manual category triggers learning)	422 bad date; 400
PUT, DELETE	/api/transactions/{id}	token	Edit selected fields; delete one transaction	400; 404
POST	/api/transactions/predict-category	token	Category the model would assign to a description	401
POST	/api/transactions/import/preview, /commit	token	Parse and clean a CSV/Excel file and label blanks; insert the cleaned rows	400 empty, missing columns, no valid rows
GET	/api/analytics	token	Overview, category split, trend, next-month and category forecasts, overrun risks	400 start after end
GET, POST	/api/budgets	token	Usage per category for a month; create or update (upsert by category, month, year)	422; 400 amount
DELETE	/api/budgets/{id}	token	Delete a budget	404
GET, POST	/api/goals	token	Goals with percent complete and required monthly saving; create a goal	400
PUT, DELETE	/api/goals/{id}	token	Set the saved amount (completes at target); delete a goal	404
GET, POST	/api/investments	token	Holdings and portfolio totals; add a holding	400
POST, DELETE	/api/investments/{id}/refresh, /{id}	token	Try a live price through yfinance (degrades to a message); delete a holding	404
GET, POST	/api/assistant/insights, /ask	token	Rule-based insights plus the disclaimer; answer a question from the user's own data	401
GET	/api/reports/{pdf|excel|csv}	token	Download a monthly report (CSV: all transactions)	400 nothing to export; 500
GET, POST	/api/notifications, /read-all	token	Raise budget alerts, then list notifications and the unread count; mark all read	401

`GET /api/public/showcase` needs no login so that the landing page can show live figures; it reads only the fictional demo account. Import `preview` and `commit` parse the same upload twice, so the user sees the cleaned rows and warnings before deciding.

5.9 User interface design

The console has 13 pages and a not-found page: a public landing page, log-in and sign-up pages, and ten pages behind the login. A route guard sends signed-out visitors to the login page, and a 401 from the API clears the stored token. The interface is dark with a light theme, one gradient accent and a collapsible sidebar; amounts use Indian digit grouping and a compact K, L and Cr notation on charts.

Table 22: Pages, routes and the endpoints they use.

Page	Route	Endpoints used
Landing	/	GET /api/public/showcase
Log in, Sign up	/login, /signup	POST /api/auth/login, POST /api/auth/signup
Dashboard	/app	GET /api/dashboard
Transactions	/app/transactions	GET, POST /api/transactions, PUT, DELETE /api/transactions/{id}, POST predict-category, import preview and commit
Analytics	/app/analytics	GET /api/analytics
Budgets	/app/budgets	GET, POST /api/budgets, DELETE /api/budgets/{id}
Goals	/app/goals	GET, POST /api/goals, PUT, DELETE /api/goals/{id}
Investments	/app/investments	GET, POST /api/investments, POST refresh, DELETE
AI Assistant	/app/assistant	GET /api/assistant/insights, POST /api/assistant/ask
Reports	/app/reports	GET /api/reports/{pdf|excel|csv}
Notifications	/app/notifications	GET /api/notifications, POST read-all
Profile	/app/profile	GET /api/auth/me, PUT /api/profile, POST /api/profile/demo-data

Every page opens with its purpose in one sentence, long actions show a busy state and finish with a toast, money is formatted the same way everywhere, and API errors appear as a sentence next to the control that caused them. Chapter 8 shows the working pages with real data; the screens below illustrate the design itself.

5.10 Design decisions and trade-offs

The table records the main decisions, what each bought and what it cost, so that the limitations in Chapter 9 trace back to a choice.

Table 23: Principal design decisions.

Decision	Reason	Cost
FastAPI, React and SQLite instead of the abstract's Flask and MySQL	The existing service code was Python and database-agnostic; SQLite needs no server; FastAPI gives typed input models and automatic documentation	A deviation from the registered abstract; single-file, single-writer storage; no migrations
Logistic regression on TF-IDF instead of BERT	Trains in milliseconds on a CPU laptop, needs no GPU and no labelled corpus of thousands; predictions are inspectable	A 51-example seed gives 18.2 % accuracy on unseen merchants; no pre-trained language knowledge

Decision	Reason	Cost
Straight-line trend with a three-month minimum	Honest about small histories and trivial to explain	Not better than a plain average below about seven months in the simulation; no seasonality, no uncertainty range
Weighted explainable health score	Every point of the score can be traced to a component and a reason	Weights are chosen, not fitted; savings rate and expense ratio are strongly correlated and carry half the score
Rule-routed assistant with an optional LLM	Answers are computed from the user's own data and work offline	Only five question patterns; 14 of 20 answerable test questions were handled
Single shared model file	Simple persistence, no per-user model management	One user's correction changes every user's predictions
Home-made HMAC token instead of a standard JWT library	No extra dependency; short and auditable	No refresh or revocation; the default secret must be replaced through `APP_SECRET_KEY`
Alerts raised on read, not on a timer	No scheduler process to run or fail	No alert if the user never opens the dashboard or notifications; the abstract's scheduled digests are not provided

Outside the delivered design: no anomaly detector (objective 4), no time-series model beyond the linear trend (objective 3), no reinforcement-learning budget recommender, no loan or debt tracker, no bank or SMS integration and no scheduled digest. Budget advice is rule-based: the assistant suggests trimming the top one or two categories by 10 to 15 % and the budget screen alerts at fixed thresholds.

CHAPTER 6: METHODOLOGY AND SOFTWARE IMPLEMENTATION

6.1 Overall methodology

FINJARVIS treats personal finance as four small problems that share one database: labelling a transaction, watching a budget, forecasting the next month and summarising health. The base paper [1] uses a large model for each (BERT, Q-learning, LSTM, Isolation Forest); this project uses the simplest inspectable model for categorising, forecasting and scoring, a threshold rule for budgets, and omits anomaly detection. The method has seven steps.

1. **Register and authenticate.** A user signs up with a name, e-mail, username and a password that passes five rules; the password is stored as a bcrypt hash and a login returns a signed token valid for seven days.
2. **Record.** Transactions arrive by form or by CSV or Excel import. Each row needs a date, a description, a positive amount and a type; rows that fail are skipped with a warning that names the row.
3. **Categorise.** A TF-IDF and logistic-regression model labels the description; a manual choice that differs from the prediction is stored and used to retrain the model.
4. **Watch.** Monthly budgets per category are compared with the month's expenses; crossing 75, 90 or 100 % writes a notification.
5. **Forecast.** A least-squares line over the monthly expense totals predicts next month's spending and each category's next-month spend; it refuses to answer with fewer than three months of history.
6. **Score.** Six explainable sub-scores are combined with fixed weights into a 0-100 health score with a rating and the weakest area.
7. **Explain and export.** A rule-routed assistant answers questions from the same records, an optional language model handles questions the rules cannot, and PDF, Excel and CSV reports are produced.

6.2 Dataset

FINJARVIS ships **no training corpus beyond a hand-written seed list**: no public dataset, no train-test split inside the application and no licence to cite. Four small data sets are used, all written or generated by the team.

Table 24: Data sets used by the product and by the measurements.

Data set	Size	Used for	Source and caveat

Data set	Size	Used for	Source and caveat
Seed list	51 (description, category) pairs, 11 categories, 3-6 per category	Training the categoriser	Written by the team in `ml/expense_classifier.py`; short English phrases in Indian-merchant style
Sample file	34 rows, Jun-Aug 2026, 10 categories	Demo account and import tests	`data/sample_transactions.csv`; fictional; shifted forward so the newest month is the current month
Demo account	68 transactions, 6 monthly totals (May-Oct 2026), 5 budgets, 3 goals, 4 holdings	Screenshots, forecast backtest, API timing	Sample file plus three synthetic earlier months (seeded generator, about +4 % per month drift); fictional
Probe set	198 descriptions, 11 categories x 18; tiers A, B and C of 66 each	Evaluating the categoriser	`scripts/probe_data.py`, written before scoring and never used for training; **not an independent benchmark**

The seed list is the whole training set of the shipped model; it is balanced only roughly (six examples for Food, Transport and Entertainment, three for Rent, Investments and Salary). The probe set separates three situations: tier A uses wordings close to the seed, tier B uses upper-case statement strings such as `UPI-SWIGGY-8821` that still contain a seed keyword, and tier C uses merchants and phrases the model has never seen. Because the team wrote both seed and probe, tiers A and B are optimistic by construction; tier C is the honest estimate of what a new user would experience.

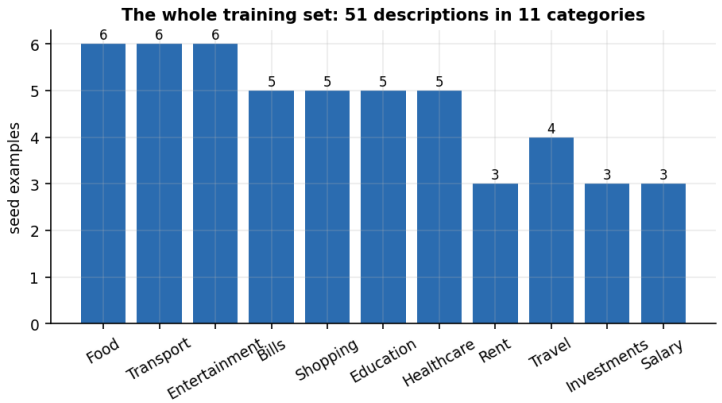


Figure 14: The whole training set of the categoriser: 51 descriptions in 11 categories.

All records in the demo account are fictional. Nothing in the repository, the screenshots or the measurements is a real person's financial data, and no conclusion about real spending behaviour may be drawn from them.

6.3 Preprocessing

Preprocessing is light. For the categoriser the description is stripped of whitespace (an empty one returns 'Other' without calling the model); the `TfidfVectorizer` lowercases it, splits word tokens and builds unigrams and bigrams (`ngram_range=(1, 2)`, `min_df=1`), giving 131 terms

on the 51-example seed. No stop-word list, stemming or spelling correction is applied, so `UPI-SWIGGY-8821` becomes the tokens `upi`, `swiggy` and `8821`, and only `swiggy` is known.

In the import pipeline each row is parsed independently: the date must parse with `pandas.to_datetime`, the amount must be a number above zero, the description non-blank, and the type 'income' or 'expense' (anything else is an expense); a blank category is filled by the categoriser. A failing row is skipped and reported as 'Row n: skipped (invalid date, amount, or missing description)'. Preview and commit parse the same file.

For the forecast the transactions are grouped by calendar month and summed per type; category forecasts pivot the expenses to a month-by-category table with missing months filled with zero. The health score takes the month's income and expenses, the savings rate, the all-time cash balance, the budget statuses, the holdings and the goals; the neutral values for missing data are listed in Section 6.5.3.

6.4 Algorithm choice and justification

Table 25: Algorithms chosen and the alternatives not used.

Task	Chosen	Why	Not used (base paper [1])
Expense categorisation	TF-IDF + multinomial logistic regression [15], [14]	Trains in milliseconds on 51 examples; no GPU; coefficients and probabilities are inspectable; works offline	BERT with 50 categories and SMOTE [8], [13]: needs thousands of labelled rows and a GPU
Spending forecast	Least-squares straight line over monthly totals	With a handful of months a line has two parameters and cannot overfit; the refusal below three months is explicit	LSTM [11]: needs long series; the base paper reports R2 0.63 on a synthetic series
Budget advice	Fixed alert thresholds (75, 90, 100 %) and rule-based suggestions	Transparent; predictable; no reward function to define	Q-learning budget agent [12]
Anomaly detection	Not built	No labelled anomalies and no time to calibrate a contamination rate	Isolation Forest [10]
Health score	Weighted sum of six clipped sub-scores	Every point can be traced to a component; weights sum to one	A learned score would need outcome data that does not exist here

With 51 examples and 131 features a linear classifier is close to the largest model that can be trained without overfitting, and its probabilities could be used for abstention; the product does not, because it always takes the top label (Chapter 8). The straight line for the forecast is likewise a bet on small data: forecasting practice [18], [17] reports that simple methods are hard to beat on short series, and Chapter 8 tests that on this product's own data.

6.5 Mathematical formulation

6.5.1 Expense categoriser

Let a description d be split into terms t (words and word pairs). With n training descriptions and document frequency df(t), scikit-learn's smoothed inverse document frequency and weight are

$$idf(t) = \ln((1 + n) / (1 + df(t))) + 1 \qquad w(t, d) = tf(t, d) \times idf(t) \qquad x_d = w_d / ||w_d||_2 \quad (2)$$

The logistic-regression classifier has one weight vector w_c and intercept b_c per category c in the set of 11 categories. For a description with feature vector x the probability of category c and the predicted label are

$$P(c \mid x) = \exp(w_c \cdot x + b_c) / \sum_k \exp(w_k \cdot x + b_k) \qquad label(d) = \operatorname{argmax}_c P(c \mid x) \quad (3)$$

Training minimises the cross-entropy loss with an L2 penalty on the weights (`LogisticRegression(max_iter=1000)`: penalty strength C = 1 and the lbfgs solver, which are the library defaults). If a description contains no known term then x is the zero vector and P(c | x) depends only on the intercepts; in the shipped model the intercept of Transport (0.321) is the largest, so unseen text is labelled Transport. The retraining on a correction adds the pair (description, corrected label) and the user's whole stored history to the 51 seed pairs and refits from scratch.

6.5.2 Spending forecast

Let y_0 ... y_{n-1} be the monthly expense totals in time order, n at least 3. The slope a and intercept b of the least-squares line, the next-month prediction and the trend label are

$$(a, b) = \operatorname{argmin} \sum_t (y_t - (a\, t + b))^2 \qquad \hat{y}_n = \max(0, a\, n + b) \qquad trend = \begin{cases} \text{increasing} & a > 1 \\ \text{decreasing} & a < -1 \\ \text{stable} & \text{else} \end{cases} \quad (4)$$

The application also reports the mean of the last three months as the 'recent average'. Category forecasts apply the same fit to each category's monthly series. The budget-overrun check flags a category when its predicted spend exceeds its budget for the current month. Forecast quality is reported with the mean absolute error and the mean absolute percentage error [16]:

$$MAE = (1/m) \sum |y_i - \hat{y}_i| \qquad MAPE = (100/m) \sum |y_i - \hat{y}_i| / y_i \quad (5)$$

6.5.3 Financial health score

With sub-scores s_k, each clipped to the range 0 to 100, and weights w_k, the score is H = SUM_k w_k s_k, rounded to one decimal. The sub-scores are defined from the savings rate r (in percent), the monthly income I and expenses E, the cash balance Bal, the budgets, the holdings and the goals:

$$s_save = clip(r / 20 \times 100) \quad s_exp = clip((1 - E / I) / 0.5 \times 100) \quad s_emerg = clip((Bal / E) / 6 \times 100) \quad (6)$$

$$s_budget = 100 \times (\text{budgets at or under 100 \%}) / (\text{all budgets}) \quad s_div = clip(\text{distinct asset types} / 5 \times 100) \quad s_goal = clip(\text{mean percent complete of goals}) \quad (7)$$

Table 26: Weights, neutral values and rating bands of the health score.

Component	Weight	Full marks when	Value when data is missing
Savings rate	0.30	savings rate is 20 % of income or more	0 if income is zero or the rate is negative
Expense ratio	0.20	expenses are half of income or less	0 if income is zero
Budget adherence	0.15	all budgets are at or under 100 %	50 if no budget is set
Emergency fund	0.15	balance covers six months of expenses	50 if there are no expenses
Diversification	0.10	five or more distinct asset types	0 if there are no holdings
Goal progress	0.10	goals are on average complete	50 if no goal is set

The bands are Excellent from 90, Good from 75, Fair from 60, Needs Improvement from 40 and Critical below 40. The lowest component names the 'area to focus on' and supplies the sentence shown beside the score. The budget status itself is computed as percent used = actual spend in the category and month divided by the budget, times 100; the alert thresholds act on that value.

6.5.4 Assistant routing

The assistant is a cascade, not a model. A lowercased question goes to five handlers in order: 'can I save' (the word 'save' and a number), 'reduce expenses', 'health' or 'score', 'where ... spending', and a category name in the question. The first handler that returns text wins. If none does and an `AI_API_KEY` is set, the question and a summary of the user's figures go to a language model; otherwise the fixed help text is returned. Every answer carries the disclaimer.

6.6 Model architecture and training

The shipped categoriser is a scikit-learn `Pipeline` of `tfidf` (`TfidfVectorizer`) and `clf` (`LogisticRegression`), fitted on the 51 seed pairs and pickled to `ml/expense_model.pkl`. Fitting takes 5.2 ms and one prediction 0.84 ms in process (scikit-learn 1.9.1). If the pickle is missing the model is trained on first use; if it cannot be loaded the keyword table (38 keywords) is used, then 'Other'. The repository pickle was written by an older scikit-learn release and loads with an `InconsistentVersionWarning`; retraining removes it. There is no hyper-parameter search or cross-validation, because there is no validation data to drive them.

Table 27: Training configuration of the categoriser.

Item	Value
Vectoriser	`TfidfVectorizer(lowercase=True, ngram_range=(1, 2), min_df=1)`; 131 terms on the seed
Classifier	`LogisticRegression(max_iter=1000)`; default C = 1, lbfgs, multinomial
Training data	51 seed pairs; on a correction: seed + the user's whole history + the corrected pair
Persistence	One pickle file for all users (`ml/expense_model.pkl`, ignored by git)
Fallback	Keyword table, then 'Other'; the model itself never outputs 'Other'
Hardware	Intel Core i5-12500, 15.7 GB RAM, no GPU used; Python 3.11.9

The forecast and the health score have no training stage: each request fits a line with `numpy.polyfit` (the demo account's six months give INR 39,471 against a recent three-month average of INR 36,592), and the weights are constants in `ml/financial_health.py`.

6.7 Evaluation protocol

Because the product has no benchmark data, the evaluation was designed for this project; each design is stated so that the numbers in Chapter 8 can be read correctly.

- **Categoriser.** Accuracy, macro-precision, macro-recall and macro-F1 on the 198-description probe set, reported overall and per tier; compared with the keyword table and with always predicting the most common label (Food). Additional tests: leave-one-out on the 51 seed pairs, accuracy on the 34-row sample file, a confidence-threshold curve, learning from corrections (random subsets of tier C added to the training data, 30 draws), and a repeat-merchant test (one correction, then the same merchant written three other ways).
- **Forecast.** A one-step-ahead rolling-origin backtest on the demo account's six monthly totals (three test points: months 4 to 6), comparing the line with the mean of all history, the mean of the last three months and the last value; plus a synthetic simulation of 2,000 users per setting with 12-month histories (level, linear drift, Gaussian noise and an occasional +50 % spike) to see how the methods behave with more data. The simulation is not real data.
- **Health score.** Sensitivity sweeps from a baseline profile (score 80.7) and 5,000 random profiles to check bounds, rating distribution and monotonicity.
- **Assistant.** 26 questions written by the team (20 answerable from the user's data, 6 off-topic), with two numeric checks against direct sums.
- **Import and API.** A 1,000-row file with 84 injected bad rows; latency of each endpoint on 127.0.0.1 over 30 repetitions (10 for reports and import).
- **Testing.** 38 pytest tests and a 49-check smoke script (Chapter 7).

Two limits apply. The probe set and the seed were written by the same team and the demo account is fictional, so no figure estimates performance on real users; and three test points cannot rank forecasting methods, so the simulation is used to say more. Cross-validation and bootstrap estimators [23] were not applied to the categoriser because 51 examples in 11 classes leave too few per fold; leave-one-out is reported instead and is itself a warning.

6.8 Application architecture

The product runs as two processes. The **backend** is `backend/main.py`, a FastAPI application served by uvicorn on 127.0.0.1:8411. At start-up its lifespan handler calls `init_db()`, which creates the six tables if they do not exist, and `demo_seed.ensure_demo_user()`, which creates the demo account once; a failure to seed is printed and does not stop the server. The **frontend** is a React 18 and TypeScript single-page application built with Vite and served on port 5411; the Vite server proxies `/api` to the backend, so the browser makes only same-origin calls.

The console uses React Router 7 for routes and the sign-in guard, Axios for the API client, TanStack Query for fetching and caching, Recharts for charts, Tailwind CSS 4 with Radix UI for styling and accessible controls, and framer-motion, GSAP and Lenis for motion. The token is kept in local storage under `finjarvis-token`; an Axios interceptor adds it to every request and clears it on a 401. A helper turns any error into a sentence: the API's `detail` text, a field-level message list, or a hint to start the backend when nothing responds. On the server the `clean` helper makes service output JSON-safe (NumPy scalars to numbers, NaN and infinity to null, data frames to records, dates to ISO strings); it is the only translation layer, so the Streamlit pages use the services unchanged.

```
def current_user(authorization: str | None = Header(default=None)) -> SimpleNamespace:
    token = (authorization or "").removeprefix("Bearer ").strip()
    uid = read_token(token) if token else None
    if uid is None:
        raise HTTPException(401, "Your session has expired. Please log in again.")
    with get_session() as session:
        u = session.query(User).filter(User.id == uid).first()
        if not u:
            raise HTTPException(401, "This account no longer exists. Please log in again.")
        return SimpleNamespace(**user_dict(u))
```

Every protected route takes `user=Depends(current_user)` and passes `user.id` to the service layer, which filters every query on it: this dependency is the whole authorisation model (no roles, only ownership).

6.9 API design and key code

The 33 routes are listed in Chapter 5. The create-transaction route is where the categoriser, the learning step and the validation meet:

```
@app.post("/api/transactions", status_code=201)
def create_transaction(body: TransactionIn, user=Depends(current_user)):
    if not body.description.strip():
        raise HTTPException(400, "Please enter a description.")
    if not is_valid_amount(body.amount):
        raise HTTPException(400, "Please enter a valid amount greater than zero.")
    chosen = (body.category or "").strip()
    auto = chosen in ("", "Auto-detect")
    predicted = predict_category(body.description)
    if auto:
        category = predicted
    else:
        category = chosen
        if predicted != category:   # a manual pick that differs from the model is a correction
```

```
            history = [(t["description"], t["category"]) for t in get_transactions(user.id)]
            try:
                learn_from_correction(body.description, category, history)
            except Exception:
                pass                # learning is best-effort, never blocks adding the transaction
    txn_id = add_transaction(user.id, body.date, body.description, category, body.amount,
                             body.transaction_type, body.payment_method, body.notes)
    return {"id": txn_id, "category": category, "auto_categorised": auto}
```

The response tells the console which category was used and whether it was automatic. Retraining reads the user's entire history on every correction, which is cheap for a few hundred rows and would not scale to a large account.

The forecast function returns a structured refusal instead of a number when the history is too short, and the analytics route passes it on unchanged:

```
def predict_next_month_expense(df: pd.DataFrame) -> dict:
    totals = _monthly_totals(df, "expense")
    if len(totals) < MIN_MONTHS_REQUIRED:
        return {"available": False,
                "message": f"Need at least {MIN_MONTHS_REQUIRED} months of transaction "
                           f"history for a reliable prediction (currently have {len(totals)})."}
    y = totals.values.astype(float)
    x = np.arange(len(y))
    slope, intercept = np.polyfit(x, y, 1)
    predicted = max(0.0, slope * len(y) + intercept)
    return {"available": True,
            "predicted_next_month_expense": round(predicted, 2),
            "recent_average": round(float(np.mean(y[-3:])), 2),
            "trend": "increasing" if slope > 1 else ("decreasing" if slope < -1 else "stable"),
            "months_used": len(y)}
```

The health score combines the sub-scores exactly as in Section 6.5.3: the dashboard route builds the six components, takes the weighted sum rounded to one decimal and names `weakest = min(components, key=components.get)`. Budget alerts are raised by `check_and_raise_alerts`, which the dashboard and notification routes call: for each budget of the month it computes the percent used, writes a danger notification at 100 % or more and a warning at 90 % and at 75 %, and skips a message already present. Because the check runs on read, an alert appears the next time the user opens either page.

6.10 Security and error handling

Passwords. A password needs at least eight characters with an upper-case letter, a lower-case letter, a digit and a special character, and is hashed with bcrypt through passlib [20]. Login accepts a username or an e-mail and answers a wrong user and a wrong password with the same message; the sign-up form does reveal a taken e-mail or username (HTTP 409), as registration forms normally do. passlib is incompatible with bcrypt 5, so the requirements pin `bcrypt==4.0.1`.

Tokens. A token is a base64 JSON payload (user id, seven-day expiry) and an HMAC-SHA256 signature made with `APP_SECRET_KEY`, checked with a constant-time comparison; a changed or expired token fails with 401. If the secret is not set the code uses the public development value

`finjarvis-dev-secret`, acceptable for a demonstration and not for a deployment. There is no refresh token and no revocation other than changing the secret, and the token sits in local storage, where an injected script could read it.

Data isolation. Every service function filters on the user id, so another user's transaction, budget, goal or holding answers 404 (the smoke test checks edit and delete). The ORM binds query parameters, so text is never concatenated into SQL, and pydantic models bound the input types.

Error handling. `get_session` commits on success and rolls back on any exception; import row errors become warnings; the assistant degrades from language model to rules to fixed help text; a failed price refresh returns a message; a failed report returns HTTP 500 with the reason; retraining and demo-seeding errors are swallowed on purpose so that a secondary feature cannot take down the primary one.

Known weaknesses (repeated in Chapters 7 and 9): no rate limit or lockout on login, a default secret, tokens that cannot be revoked, a model file shared by all users, CORS and proxy set for localhost only, no HTTPS, and the optional language-model layer sends a summary of the user's figures to a third party when a key is set, which a deployment would have to disclose.

6.11 Installation and execution

Table 28: Software requirements.

Item	Version or note
Python	3.11 or newer (3.11.9 used for the measurements)
Node.js	20 or newer
Backend packages	fastapi, uvicorn, python-multipart, sqlalchemy 2, pandas, numpy, scikit-learn, bcrypt 4.0.1, passlib, python-dotenv, pydantic 2, yfinance, reportlab, openpyxl, httpx, requests
Optional	`anthropic` and an `AI_API_KEY` for the language-model layer; internet access for price refresh
Frontend packages	react 18, react-router 7, @tanstack/react-query, axios, recharts, tailwindcss 4, radix-ui, framer-motion, gsap, lenis, sonner, vite
Test tools	pytest (38 tests), `scripts/smoke_test.py` (49 checks)

1. Open the product folder and double-click `run.bat`. It checks Python and Node, builds `.venv` and installs the backend packages (rebuilding `.venv` if it was copied from another computer), runs `npm install` for the frontend, starts the backend, waits for `/api/health` to answer and starts the frontend.
2. Open http://localhost:5411 and sign in with the demo account: username `demo` and password `Demo@1234` (fictional data), or sign up with a new account; a new account starts empty and the Profile page has a button that loads the demo data into it.
3. To run the pieces by hand: `python -m venv .venv`, `pip install -r backend/requirements.txt`, `python -m uvicorn backend.main:app --host`

`127.0.0.1 --port 8411` from the product folder, and `npm install` then `npm run dev` inside `frontend/`.

4. Copy `.env.example` to `.env` to change the ports, to set `APP_SECRET_KEY` to a long random string, or to add `AI_API_KEY`. Set `FINJARVIS_DB` to use another SQLite file.
5. To check an installation run `python -m pytest tests -q` (38 tests) and `python scripts/smoke_test.py` (49 checks, on a throw-away database and a spare port).

Appendix B repeats these instructions with screens. The first start of `run.bat` takes several minutes because of package downloads, and the Streamlit front end still runs with `streamlit run app.py` on port 8501 against the same database file.

CHAPTER 7: SYSTEM TESTING

7.1 Testing strategy

FINJARVIS was handed over as a working Streamlit application with a small pytest suite and no end-to-end test. The documentation run kept that suite, added a smoke script that walks the web API from sign-up to report download, and added measurement scripts that probe the three algorithms (Chapter 8). The strategy follows the structure of the code. The service layer and the ML modules are plain functions with checkable results, so they are unit-tested against a temporary SQLite file that every test receives through an automatic fixture; the 33 FastAPI routes are tested end to end by a script that starts the real server on a spare port against a throw-away database; and the quality of the models is measured offline on data written for the purpose instead of being asserted.

Table 29: Test levels, tools and results.

Level	Tool and scope	Count	Result
Unit	pytest in `tests/`: password hashing and authentication, transactions and user isolation, budgets and alerts, the health score, the categoriser and the forecast	38 tests	38 passed, 0 failed, 11.5 s
Functional / integration	`scripts/smoke_test.py`: starts the backend, then exercises auth, transactions, import, budgets, goals, investments, analytics, assistant, reports and profile through HTTP	49 checks	49 of 49 passed
Model evaluation	`scripts/run_experiments.py` on the probe set, the demo account, 2,000-user simulations, 5,000 random profiles and 26 assistant questions	6 studies	Results in Chapter 8 (not pass or fail tests)
Performance	`scripts/measure_api.py`: latency of 19 request types on 127.0.0.1	30 or 10 repetitions each	Table in Section 7.7
Screens	Playwright walk of the React console on the demo account (`scripts/capture_screens.py`)	19 screenshots	Captured and inspected by eye; not an automated UI test

Unit tests use tiny, hand-checkable inputs (a four-month series rising by 100 per month, a user with a 20 % savings rate) so that a failure points at one rule; the smoke script uses a brand-new user with exact amounts (a 450 expense that becomes 500, a transport budget of 200 overspent by 220). Neither level uses the demo account's totals as expected values, because those depend on today's date.

7.2 Unit testing

The pytest suite has 38 tests in five files. The fixture in `tests/conftest.py` creates a fresh SQLite file for every test function, rebinds the application's engine and session factory to it, and deletes it afterwards (after disposing the engine, which Windows requires). No test touches the real `finjarvis.db`.

Table 30: Unit tests by file.

File	Tests	What is checked
`test_auth.py`	9	A bcrypt hash differs from the password and verifies only the right one; sign-up succeeds; a weak password, mismatched passwords and a duplicate e-mail are refused; login works with a username and with an e-mail; a wrong password and an unknown user are refused
`test_transactions.py`	8	Add creates a row; a negative amount and an invalid type raise `ValueError`; update; delete and delete of a missing id returns false; **user isolation** (one user's rows are invisible to another); filter by category and type
`test_budget.py`	6	Set and read status; a non-positive amount is rejected; setting again updates the same budget; delete; an overrun raises an alert notification; no alert under the threshold
`test_financial_health.py`	6	Savings-rate and expense-ratio bounds; a healthy profile scores well and a struggling one poorly; the score stays within 0-100; the result carries an explanation and the weakest area
`test_ml.py`	9	'Swiggy order' is Food, 'Uber ride' Transport, 'Netflix subscription' Entertainment; an empty description gives 'Other'; nonsense text returns a string without crashing; the forecast refuses fewer than three months; works with four; never returns a negative value on a falling series; the category forecast declines on an empty frame
Total	**38**	**38 passed**; 16 warnings, all the scikit-learn pickle-version notice

Three of these tests are weaker than they look. The categoriser tests use phrases that are in the seed list, so they confirm that the model reproduces its training data, not that it generalises. The nonsense-text test asserts only that a non-empty string comes back (Chapter 8 shows it is 'Transport'). And no unit test checks the exact value of the health score or the forecast, only their bounds; the experiment scripts cover the numbers.

7.3 Integration and functional testing

`scripts/smoke_test.py` copies nothing from the product's own database: it sets `FINJARVIS_DB` to a temporary file, starts uvicorn on a free port with the same `backend.main:app`, waits for `/api/health` and runs 49 checks with the `requests` library. All 49 passed. The checks are grouped below; the first column is the user-visible feature.

Table 31: Smoke-test checks by feature.

Feature	Checks	Representative assertions
Authentication	5	health endpoint; a protected route without a token answers 401; the demo account logs in; a wrong password is rejected; a weak password is rejected at sign-up (400)
Demo data	4	the demo dashboard has current-month income; the health score is between 0 and 100; all chart datasets are returned; the public showcase works without login
Sign-up and isolation	3	sign-up returns 201; a duplicate answers 409; a new user starts with no transactions
Transactions	10	'Swiggy order' is auto-categorised as Food; income is accepted; amount 0 is rejected (400); search finds the expense; edit; another user's edit and delete answer 404; delete; a second delete answers 404
Import	4	preview of three rows gives 2 valid and 1 warning; a blank category is filled (Transport); commit imports 2; a file with missing columns answers 400 with 'Missing required column(s)'
Budgets and alerts	5	a budget saves (201); spending 220 against 200 shows 100 % or more; an alert notification appears; mark-all-read clears the unread count; delete
Goals	3	17 % complete and a positive required monthly saving; a goal completes at its target; delete

Feature	Checks	Representative assertions
Investments	3	profit and loss of 200 (20 %); refresh with no symbol returns a failure message, not an error; delete
Analytics and assistant	5	the demo user gets a forecast above zero; a one-month user is refused; a reversed date range answers 400; the assistant answers 'biggest spending category' with the disclaimer; insights are returned
Reports	3	PDF starts with `%PDF`, Excel is a zip (`PK`), CSV starts with `id,date`
Profile and security	4	profile update persists; a duplicate e-mail is refused; the demo-data loader inserts transactions; a token with a changed signature answers 401

The script has two known weaknesses: the check 'edit persisted' accepts either of two amounts because list order is not guaranteed, so it would also pass if the edit were lost; and it is a single linear walk, so one early failure can make later checks fail for the same reason.

7.4 Security testing

The security checks that were run are those in the unit and smoke tests above; no penetration test or automated scanner was used. What was verified, and how:

Table 32: Security checks that were run.

Check	Result	Where
Password rules: 'abc' refused; mismatched passwords refused	Passed (HTTP 400)	`test_auth.py`, smoke
Hash is not the plain password and verifies only the right one	Passed	`test_auth.py`
Protected route without a token	401	smoke
Token with a changed signature	401	smoke
Another user edits or deletes a transaction	404; the data is unchanged	smoke, `test_transactions.py` isolation
Duplicate e-mail on sign-up and on profile update	409 on both	smoke
Wrong password versus unknown user	Both refused; the code returns the same message for both (`Invalid username/email or password.`)	`test_auth.py` (refusal), `auth/authentication.py` (message)

Several properties were **not tested** (Chapter 6 explains why each is a weakness): brute-force protection (the code has no rate limit or lockout), token expiry (the seven-day lifetime was not simulated), the default secret (`finjarvis-dev-secret` is used if `APP_SECRET_KEY` is unset, so anyone who knows it can forge a token), SQL-injection payloads (the ORM binds parameters; not probed), cross-site scripting (the token is in local storage), and the shared categoriser file. The product suits a demonstration on a trusted machine and is not hardened.

7.5 Validation and edge cases

The table lists the edge cases exercised by the tests, the smoke script and the experiment scripts; a defect or gap is marked as such.

Table 33: Edge cases and observed behaviour.

Input or situation	Observed behaviour	Verdict

Input or situation	Observed behaviour	Verdict
Transaction with amount 0 or a negative amount	HTTP 400 from the route; `ValueError` from the service	Correct
Empty description sent to the categoriser	Returns 'Other' without calling the model	Correct
Description with no known word (`xyzabc123 qqq`)	Labelled **Transport** (the highest intercept), top probability 0.121; no warning, no abstention	Gap: silent wrong label
Fewer than three months of history	Forecast refuses: 'Need at least 3 months of transaction history for a reliable prediction (currently have n).'	Correct; threshold is low for the method (Chapter 8)
Falling expense series	Prediction is clipped at 0, never negative	Correct
Reversed analytics date range	HTTP 400, "'From' must be on or before 'To'."	Correct
CSV without the required columns	HTTP 400 with the names of the missing columns	Correct
1,000-row import with 84 injected bad rows (dates, amounts, blank descriptions)	84 skipped (exact), 916 imported, all 916 blank categories labelled	Correct
Investment refresh with no symbol or no market data	Returns a failure message and keeps the entered price	Correct
Budget re-crossing a threshold in a later month	No new notification (de-duplicated by message text only)	Gap
Question outside the assistant's patterns	6 of 20 answerable questions fell to the fixed help text; 6 of 6 off-topic questions were declined	Partial (Chapter 8)
'Am I overspending on shopping?'	Answered with the Shopping total, not a budget comparison	Gap
Random financial profiles (5,000)	Score always between 8.7 and 98.2; raising the savings rate by five points never lowered the score (0 violations)	Correct

7.6 Model validation as testing

The three algorithms are tested by measurement; the full results are in Chapter 8 and are summarised here.

- **Categoriser.** Accuracy 71.7 % on the probe set (macro-F1 0.761), but 100 % and 97 % on the seed-like and statement-style tiers and only 18.2 % on unseen merchants; leave-one-out accuracy on the seed is 11.8 %.
- **Forecast.** On the demo account's three test points the line's MAPE was 28.4 % against 21.4 % for the mean of all history; in 2,000-user simulations a plain three-month mean was at least as good as the line in every setting.
- **Health score.** Bounded, monotone in the savings rate on 5,000 random profiles, with a demo-account score of 90.4 (Excellent) whose weakest component is goal progress.
- **Assistant.** 14 of 20 answerable questions answered from the user's data; the two numeric answers checked (Food total INR 17,365; top category Rent) matched direct sums.

These are validation results on data written by the team, not field trials.

7.7 Performance testing

`scripts/measure_api.py` starts the backend on a temporary database that holds the demo account, logs in once, and times each request type 30 times (10 times for the report downloads

and the import preview) with `requests` against `127.0.0.1`. The machine was Intel Core i5-12500 (6 cores, 12 threads), 15.7 GB RAM, no discrete GPU, Windows 11 Pro, Python 3.11.9. Calls to `localhost` added about two seconds each here because of an IPv6 fallback, so every figure uses the numeric address. All figures are single-user, single-process and warm; no load or concurrency test was run.

Table 34: API latency (milliseconds).

Request	Runs	Median	p95	Max
login (bcrypt verify)	30	304.9	367.4	509.0
`GET /health`	30	4.3	19.0	21.3
`GET /dashboard`	30	67.2	129.3	194.6
`GET /transactions`	30	12.7	38.9	96.7
`GET /analytics`	30	72.1	128.5	170.0
`GET /budgets`	30	19.7	51.6	63.7
`GET /goals`	30	7.3	11.6	14.0
`GET /investments`	30	7.0	12.8	17.8
`GET /assistant/insights`	30	35.9	47.9	55.0
`GET /notifications`	30	18.1	33.2	49.3
`GET /meta`	30	7.2	11.8	12.8
`POST /transactions/predict-category`	30	7.9	20.5	30.6
`POST /assistant/ask (analytics)`	30	21.6	47.9	68.7
`POST /assistant/ask (health)`	30	28.4	47.8	86.4
`POST /transactions (auto-categorise)`	30	15.2	20.2	26.8
`GET /reports/csv`	10	17.7	30.5	59.1
`GET /reports/excel`	10	209.4	341.6	1165.2
`GET /reports/pdf`	10	135.5	201.8	211.9
`POST /transactions/import/preview (100 rows)`	10	667.0	933.3	941.0

Every data route answers in under 135 ms at the 95th percentile except four: login (median 305 ms, which is the cost of the bcrypt verification and is intentional), the PDF report (median 136 ms), the Excel report (median 209 ms) and the import preview (median 667 ms for 100 rows, about 6.7 ms per row). The dashboard, which recomputes the metrics, budgets, health score and six chart datasets in one call, takes a median of 67 ms, and a categorisation request takes 7.9 ms. For one user on a local machine every interactive route answers well inside a second.

The import commit is slower: a 1,000-row file took 6.6 s to clean and label and 26.0 s to insert (35.2 rows per second), because `commit_import` opens one database session per row. A monthly statement of a few hundred rows takes seconds.

7.8 User acceptance testing

No user acceptance test was carried out. The product was used only by the project team. There was no questionnaire, no usability session, no task-completion timing and no comparison with another finance application, so no claim about ease of use, satisfaction or financial-behaviour change is made. The Mabu app study [2] shows what such a study looks like

(interviews, survey, card sorting, think-aloud sessions and a usability questionnaire); it is listed as future work in Chapter 10.

The nearest substitute is the screen walk: all 13 pages were opened with the demo account at 1440 x 900 and checked by eye for layout, empty states and obvious errors (the screenshots in Chapter 8). That finds visible defects; it does not show that a person can complete a task.

7.9 Limits of the testing done

The testing supports the statement that the product's rules do what the code says and that its routes work together on one machine. It does not support stronger statements. The limits are:

- **Small, self-written data.** The categoriser was scored on 198 descriptions written by the team that wrote the 51-example seed; the forecast was backtested on six months of fictional data (three test points); the assistant on 26 questions. None is an independent benchmark.
- **No coverage measurement.** The share of lines executed by the tests was not measured; the route layer is reached only through the smoke script.
- **No browser-level tests.** The React console has no automated test; type checking (`tsc -b`) runs in the build.
- **No concurrency, soak or load test.** SQLite serialises writes and the shared model file is rewritten on a correction; neither was exercised with two simultaneous users.
- **No security assessment.** Section 7.4 lists what was and was not checked.
- **Unexercised integrations.** The optional language-model layer and the live price refresh need an API key and internet access and were not run; their failure paths (fall back to rules, return a message) are read from the code, not observed.
- **One machine, one run.** Accuracy figures are deterministic, but latency and import times shift by a few tens of percent between runs.
- **Experiments are not regression tests.** The measurement scripts write JSON; they do not fail when a number gets worse.

In summary, 38 unit tests and 49 end-to-end checks pass and the measurements show where the simple models are weak; the testing cannot show how the product behaves with real users, real statements or a hostile network.

CHAPTER 8: RESULTS AND OUTPUT SCREENS

8.1 Experimental set-up

FINJARVIS has no large training corpus, so this chapter reports what the running application shows (Section 8.2), how its three analytical modules behave when measured (Sections 8.3 to 8.5), how the assistant, the import pipeline and the API perform (Sections 8.6 and 8.7) and how the results compare with the base paper (Section 8.8). Every table and chart is drawn from `experiments/metrics.json` and `experiments/api_latency.json`, written by `scripts/run_experiments.py` and `scripts/measure_api.py`.

8.1.1 The data

The categoriser is trained on 51 short example descriptions in 11 categories that are written into `ml/expense_classifier.py`; no public dataset was used. The application data are the fictional records of the demo account (`demo` / `Demo@1234`): 68 transactions over six months (May to October 2026), five budgets, three savings goals and four investment holdings, built from the 34-row file `data/sample_transactions.csv` (dates shifted to the current month) and three earlier synthetic months (seed 411, drift of about 4 % a month). **All of these data are fictional**: they show that the mechanisms work, not how a real household spends.

8.1.2 The probe set

Because the application has no train and test split of its own, the team wrote a probe set of 198 descriptions before scoring (`scripts/probe_data.py`): 18 for each of the 11 categories, in three tiers of 6 per category. Tier A uses wording close to the seed list, tier B upper-case bank-statement strings that contain a seed keyword (for example `UPI-HOSPITAL-BILL`), and tier C merchants and phrases the model has never seen (for example `BigBasket delivery`, `Jio 599 plan`). The probe set was never used for training. It is **not an independent benchmark**: the same team wrote the seed list and the probe set.

8.1.3 Machine and software

All measurements were made on one machine: an Intel Core i5-12500 (6 cores, 12 threads) with 15.7 GB of memory, an Intel UHD 770 integrated GPU (not used), Windows 11 Pro and Python 3.11.9, with scikit-learn 1.9.1. API timings use the backend on `127.0.0.1`, a throw-away SQLite file with the demo data, one client and a warm server.

8.2 Output screens

The screens below were captured from the running application with Playwright at 1440 by 900 pixels, logged in as the demo account; none is a mock-up. Every amount on them belongs to the

fictional demo records (Section 8.1.1). The light-theme landing page, the sign-up page and the light-theme dashboard are in Appendix C.

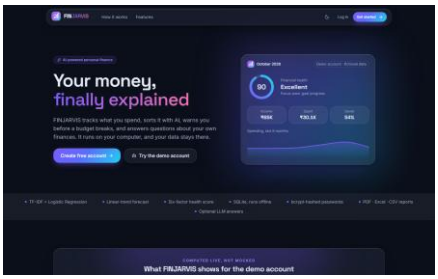


Figure 15: Landing page (dark theme): the hero card shows the demo account for October 2026 with health score 90 (Excellent), income INR 65K, spent INR 30.1K and 54 % saved; the strip below lists what the product does.

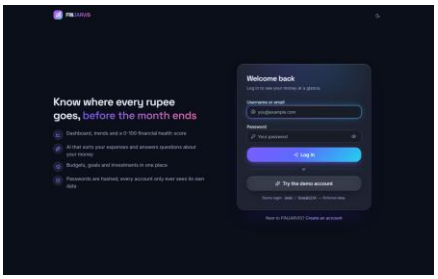


Figure 16: Login page: username or e-mail, password, a Try the demo account button and the demo credentials.

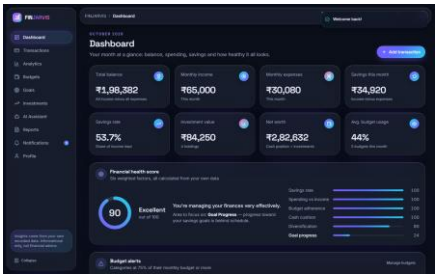


Figure 17: Dashboard for October 2026: balance INR 1,98,382, income INR 65,000, expenses INR 30,080, savings INR 34,920 (53.7 %), investments INR 84,250, net worth INR 2,82,632, average budget usage 44 %. The health card shows 90 (Excellent), component scores 100, 100, 100, 100, 80 and 24, and names Goal Progress as the area to focus on.



Figure 18: Dashboard charts: income against expenses, spending by category (led by Rent), the six-month trend and the allocation of the four holdings.

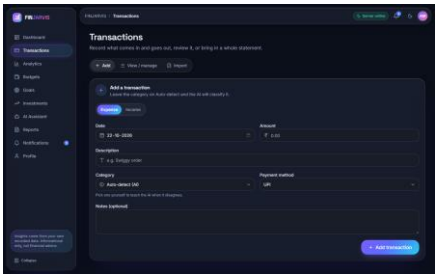


Figure 19: Add a transaction: type, date, amount, description, a category that defaults to Auto-detect (AI), payment method and notes; a manual category that differs teaches the model.

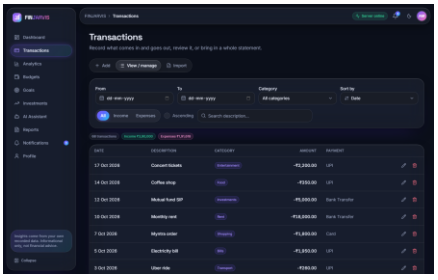


Figure 20: View and manage: 68 transactions, income INR 3,90,000 and expenses INR 1,91,618 for the six months, with filters, search, sort and edit and delete icons.

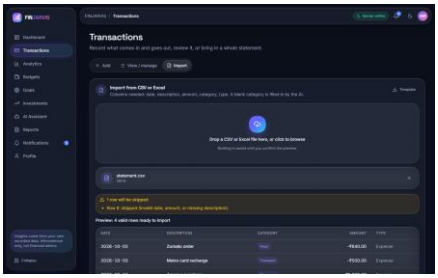


Figure 21: Import from CSV or Excel: the preview reports Row 6 as skipped and lists the four valid rows with categories the AI filled in; nothing is saved until the preview is confirmed.



Figure 22: Analytics for 24 July to 22 October 2026: income INR 1,95,000, expenses INR 1,14,194, savings rate 41.4 %, daily average INR 1,359 and largest expense INR 18,000 (rent); the category chart is led by Rent, Investments and Shopping.



Figure 23: Spending trend and prediction: the expense curve for May to October 2026 and the prediction card: next month INR 39,471 from six months, recent three-month average INR 36,592, trend Increasing.

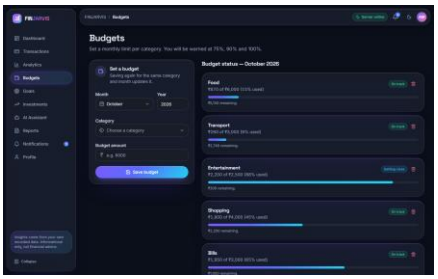


Figure 24: Budgets for October 2026: Food 15 %, Transport 9 %, Entertainment INR 2,200 of 2,500 (88 %, Getting close), Shopping 45 % and Bills 65 %, each with a progress bar; warnings are promised at 75, 90 and 100 %.



Figure 25: Financial goals: Emergency Fund 32 %, New Laptop 19 % and Vacation 20 %, each with the suggested monthly saving and an update field.



Figure 26: Investments: invested INR 77,860, value INR 84,250, profit INR 6,390 (8.21 %) over four holdings, with an allocation donut and profit and loss by asset.

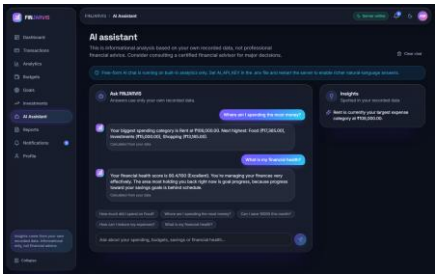


Figure 27: AI assistant: two questions answered from the user's own data (Rent at INR 108,000 is the top category; health 90.4, Excellent, weakest area goal progress). A banner notes that free-form answers are off because no AI_API_KEY is set.

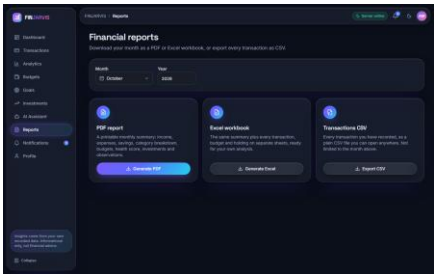


Figure 28: Financial reports: month and year selectors and three downloads, a PDF summary, an Excel workbook and a CSV of every transaction.

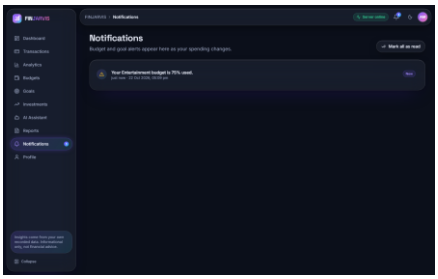


Figure 29: Notifications: one unread alert, Your Entertainment budget is 75% used, created when spending crossed the 75 % threshold.

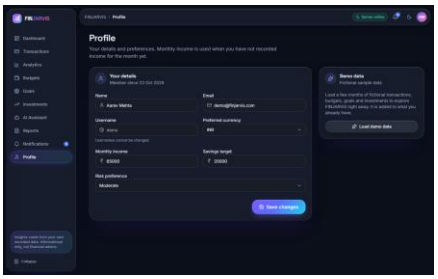


Figure 30: Profile: name, e-mail, currency INR, monthly income INR 65,000, savings target INR 20,000, risk preference Moderate and a card that loads the fictional demo data.

8.3 Expense categoriser

8.3.1 Accuracy on the probe set

The shipped model (TF-IDF with word unigrams and bigrams followed by logistic regression) labels 142 of the 198 probe descriptions correctly, an accuracy of **0.7172** (macro precision 0.8847, macro recall 0.7172, macro F1 **0.7614**). The headline hides a sharp split between the tiers: 100.0 % on wording close to the seed list (66/66), 97.0 % on upper-case statement strings that contain a seed keyword (64/66), and only **18.2 %** on merchants it has never seen (12/66). On the same 198 descriptions the fallback keyword table scores 0.5152 (macro F1 0.6623) and always answering Food scores 0.0909; the learned model beats the table on every tier, but almost entirely in tiers A and B.

Table 35: Categoriser on the 198-description probe set.

Model	Accuracy	Macro P	Macro R	Macro F1	Tier A	Tier B	Tier C
TF-IDF + logistic regression (shipped)	0.7172	0.8847	0.7172	0.7614	1.000	0.970	0.182
Keyword table (fallback)	0.5152	0.9517	0.5152	0.6623	0.712	0.727	0.106

Model	Accuracy	Macro P	Macro R	Macro F1	Tier A	Tier B	Tier C
Majority class (Food)	0.0909	0.0083	0.0909	0.0152	0.091	0.091	0.091

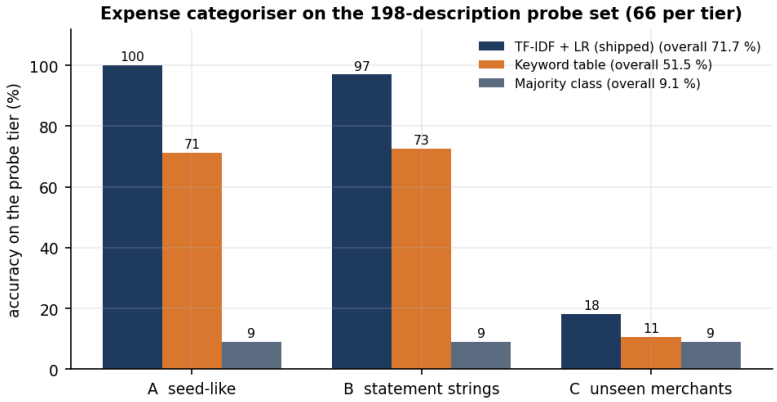


Figure 31: Accuracy of the three models on each probe tier; the shipped model falls from near 100 % to 18 % on unseen merchants.

On the 34-row bundled sample file the model labels all 34 rows as the file does (accuracy 1.000), but its vocabulary overlaps the seed list, so that figure is optimistic. Leave-one-out validation on the 51 seed pairs gives only **11.8 %** (6/51): each seed phrase has three to six siblings that hardly share words with it, so a held-out phrase cannot be recognised. This shows how small the seed list is and agrees with tier C.

8.3.2 Confusion matrix and per-category scores

The confusion matrix of the probe set is shown below. The diagonal holds 142 of 198 items. Almost all of the 56 errors fall in one column: **48 descriptions were labelled Transport**, 54 of the 56 errors belong to tier C and only 2 to tier B (`UPI-AMAZON-PAY-8821` labelled Entertainment, because the seed list has `amazon prime video`, and `UPI-HOSPITAL-BILL` labelled Bills). The cause is mechanical: a description made only of unseen words gives an all-zero TF-IDF vector, so the decision comes from the intercepts alone and Transport has the largest: for the nonsense text `xyzabc123 qqq` the model returns Transport with probability 0.121, against 0.118 for Food and 0.118 for Entertainment. The model never outputs Other, so an unseen merchant is silently given a category.

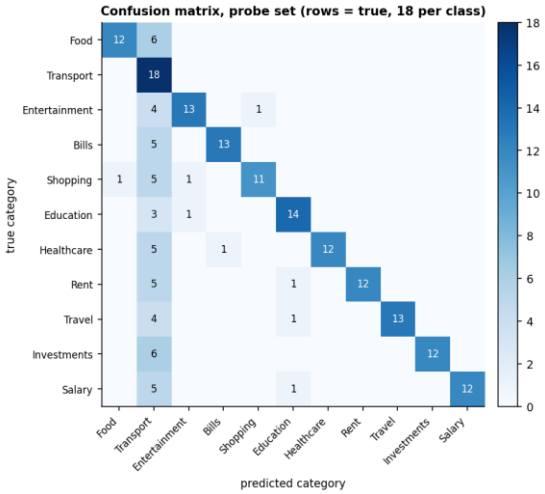


Figure 32: Confusion matrix on the probe set (rows are true categories, 18 descriptions each); the Transport column collects the unseen-merchant errors.

Table 36: Per-category scores on the probe set (18 descriptions per category).

Category	Precision	Recall	F1	Correct
Food	0.923	0.667	0.774	12/18
Transport	0.273	1.000	0.429	18/18
Entertainment	0.867	0.722	0.788	13/18
Bills	0.929	0.722	0.812	13/18
Shopping	0.917	0.611	0.733	11/18
Education	0.824	0.778	0.800	14/18
Healthcare	1.000	0.667	0.800	12/18
Rent	1.000	0.667	0.800	12/18
Travel	1.000	0.722	0.839	13/18
Investments	1.000	0.667	0.800	12/18
Salary	1.000	0.667	0.800	12/18

Precision is 1.000 for Healthcare, Rent, Travel, Investments and Salary; Transport's recall is 1.000 but its precision only **0.273**: it is the dump class.

8.3.3 Confidence and abstaining

The model's probabilities are low in absolute terms (eleven classes, 51 examples) but informative: the mean top probability is 0.197 when the label is right and 0.130 when it is wrong. Withholding predictions below 0.15 would still label 62.1 % of the items at 94.3 % accuracy; at 0.20 it would keep 32.3 % at 98.4 %, and at 0.25 7.6 % at 100 %. **The application applies no such threshold.** Asking the user when the model is unsure would turn the Transport dump into a prompt for a correction, the cheapest improvement available.

8.3.4 Learning from corrections

When a user picks a category that differs from the prediction, the application retrains on the seed list, the user's history and the correction. Two experiments measure what that buys. In the first, k random tier-C items were added as corrections (30 draws per k) and accuracy was

measured on the rest of tier C: it moved from 18.2 % with no corrections to 21.5 % with 48, so teaching the model some merchants hardly helps with others. In the second, one unseen merchant was corrected and then narrated three other ways (`UPI-X-4412`, `POS X`, `X 2`; 198 tests over 66 merchants): accuracy rose from 18.2 % to **100.0 %**. A correction fixes that merchant for good but does not generalise; the second test is easy by construction, because the variants share the corrected words.

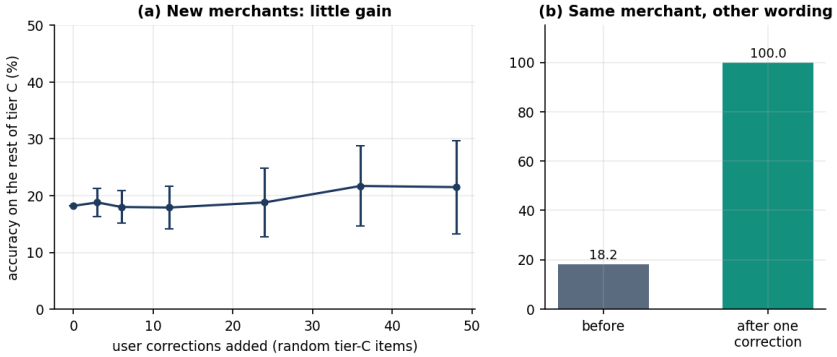


Figure 33: (a) Tier-C accuracy against the number of corrections; (b) accuracy on the same merchant written three other ways, before and after one correction.

One prediction takes 0.84 ms in process and retraining from the seed list 5.2 ms. The pickled model in `ml/` was trained under scikit-learn 1.5.0 and loads under 1.9.1 with an `InconsistentVersionWarning`; retraining removes it.

8.4 Spending forecast

8.4.1 Backtest on the demo account

The forecast fits a straight line to the monthly expense totals and extends it one month. It was tested by rolling origin on the demo account's six monthly totals (26,579, 26,878, 28,384, 36,118, 43,579, 30,080 INR for May to October 2026): months 4, 5 and 6 were each predicted from the months before them, so there are **only three test points**. The linear trend followed the August to September rise but overshot October, when spending fell from 43,579 to 30,080; its MAPE is **28.4 %** (MAE INR 9,597). The mean of all history was better (21.4 %, MAE INR 8,385), and so was the mean of the last three months (24.8 %). Three points cannot rank methods, but the trend model shows no demonstrated advantage here.

Table 37: One-step-ahead forecasts for August, September and October 2026 (INR) and their errors.

Method	Aug forecast	Sep forecast	Oct forecast	MAE	MAPE
Linear trend (app)	29,085	37,020	45,280	9,597	28.4 %
Mean of all history	27,280	29,490	32,308	8,385	21.4 %
Mean of last 3 months	27,280	30,460	36,027	9,301	24.8 %
Last value	28,384	36,118	43,579	9,565	27.8 %

The actual values were 36,118, 43,579, 30,080 INR. On the full six months the application predicts INR **39,471** for November (recent three-month average INR 36,592, trend increasing, 6 months used), as the Analytics page shows (Section 8.2.4). Per-category forecasts use the same fit: Rent 18,000, Investments 7,000, Travel 3,967, Shopping 3,314, Food 2,666, Bills 1,953, Entertainment 1,201, Healthcare 729, Transport 641 (INR); Rent equals its fixed INR 18,000, the one category where a straight line is exact.

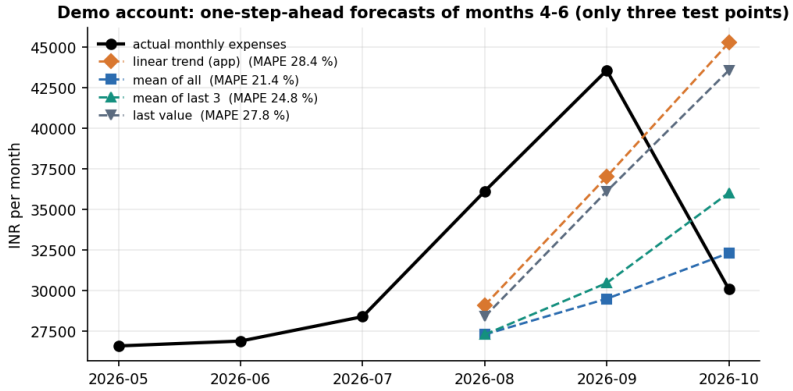


Figure 34: Actual monthly expenses against the four one-step-ahead forecasts for months 4 to 6 of the demo account.

8.4.2 Simulation: when does the trend help?

To learn more than three points can tell, the team simulated 2,000 users per setting (seed 7): a level drawn between 20,000 and 60,000, a linear drift of N(0, 3 % a month), Gaussian noise and a 10 % chance of a 50 % spike. **These are synthetic data and say nothing about real households**; they show how the methods behave when the generating process is known. At 5 % noise the trend gives a MAPE of 14.0 % against 13.1 % for the three-month mean; at 30 % noise 54.0 % and 51.1 %. The three-month mean was at least as good as the trend in every noise setting.

Table 38: Simulated one-step-ahead MAPE (%) against noise (12-month histories).

Noise (sd / level)	Linear trend (app)	Mean of all	Mean of last 3	Last value
0.05	13.99	16.77	13.11	13.19
0.10	18.62	20.72	17.07	18.84
0.20	30.84	31.06	28.23	32.44
0.30	53.99	54.26	51.11	56.11

History length matters more than noise. At 10 % noise the trend needs about seven months before it overtakes a plain average: with 3 months its MAPE is 22.5 % against 14.9 % for the mean, with 7 months 16.9 % against 17.8 %, and with 11 months 17.5 % against 33.6 %, because the mean of all history ignores the drift. The three-month mean stays at least as good as the trend at every length (20.2 % at 11 months against 17.5 %). The application's minimum of 3 months is therefore too low: at that length the trend is the worst of the four.

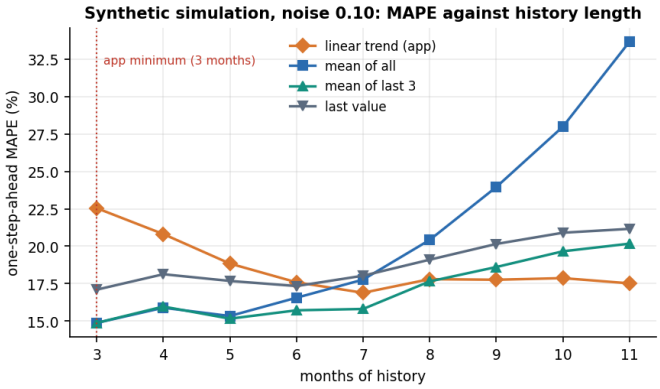


Figure 35: Simulated MAPE against months of history at 10 % noise; the dotted line marks the application's three-month minimum.

The base paper's LSTM reports RMSE 137.2, MAE 89.5 and R-squared 0.63 on a synthetic savings series of unstated units [1]; these cannot be compared with the errors above, since the series, target (savings, not expenses), units and validation differ.

8.5 Financial health score

The score is a weighted sum of six components, each scaled to 0 to 100 (Chapter 6). It has no training data, so the results are about its behaviour, not its accuracy. The demo account scores **90.4 out of 100, Excellent**; its weakest component is goal progress (24).

Table 39: Components of the demo account's score (October 2026).

Component	Weight	Component score	Contribution
Savings rate	0.30	100	30.0
Expense ratio	0.20	100	20.0
Budget adherence	0.15	100	15.0
Emergency fund	0.15	100	15.0
Investment diversification	0.10	80	8.0
Goal progress	0.10	24	2.4
Total	1.00		**90.4**

8.5.1 Sensitivity

From a baseline profile (income INR 65,000, expenses INR 32,500, a savings rate of 20 %, a balance of INR 100,000, two budgets at 60 % and 90 %, two asset types and goals at 40 %) the score is 80.7. Raising the savings rate from 0 % to 20 % lifts it from 50.7 to 80.7, and no further (the component saturates at 20 %). Raising monthly expenses from INR 20,000 to INR 75,000 lowers it from 85.5 to 56.3. A balance covering six months of expenses (INR 195,000 and above) lifts it to 88.0, where it stops. Goal progress from 0 % to 100 % adds 10.0 points and each extra asset type 2 points up to five types. The curves are monotone and saturate where the formulas cap.

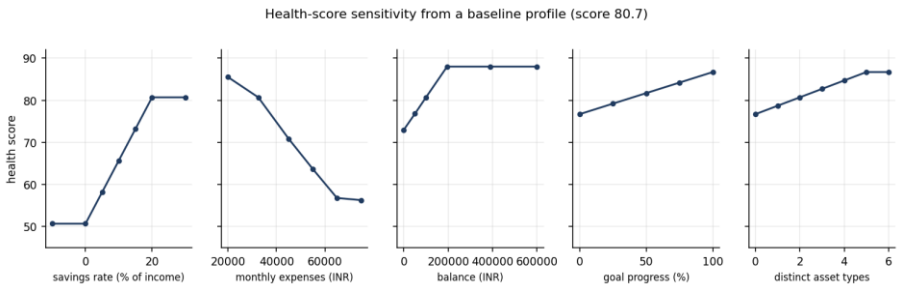


Figure 36: Health-score sensitivity to each input, changed one at a time from the baseline profile.

8.5.2 Behaviour over 5,000 random profiles

Over 5,000 random profiles the score ranged from 8.7 to 98.2 with a mean of 60.28; the ratings were Critical 1,425, Needs Improvement 680, Fair 1,083, Good 1,594 and Excellent 218. Raising the savings rate by five points never lowered the score (0 violations in 5,000 profiles). Two things limit what the score means. The savings rate and the expense ratio carry half of the weight and are strongly correlated (both derive from income minus expenses), so one habit is counted twice; and the weights are chosen by the team, not fitted to any outcome, with no user study. The score is a transparent summary, not a validated predictor of well-being.

8.6 AI assistant

The assistant routes a question through five rule handlers (can I save, reduce expenses, financial health, where am I spending, spending on a named category) and, only if none matches and an `AI_API_KEY` is set, to an optional language model; otherwise it replies with a fixed help text. **The language-model layer was not exercised in any measurement.** The team wrote 26 questions, 20 answerable from the user's data and 6 off-topic, and asked them of the demo account. The rule layer answered **14 of 20 (70 %)** answerable questions and correctly declined 6 of 6 off-topic ones, wrongly answering 0. The 6 misses all fell to the fixed help text: "How much did I spend on groceries?"; "What is my biggest expense?"; "How much do I spend on eating out?"; "What was my largest purchase this month?"; "Will I run out of money this month?"; "Which category should I cut first?": phrasings the keyword routing does not recognise.

Table 40: Sample questions and the assistant's answers (demo account).

Question	Answered by	Answer (shortened)
How much did I spend on Food?	analytics	You've spent INR 17,365.00 on Food across your recorded transactions.
Can I save 5000 this month?	analytics	Based on this month so far, your income minus expenses leaves you INR 34,920.00, which comfortably covers a INR 5...
How can I reduce my expenses?	analytics	Your highest spending categories are: - Rent: INR 108,000.00 - Food: INR 17,365.00 - Investments: INR 15,000.00 A c...
What is my financial health?	analytics	Your financial health score is 90.4/100 (Excellent). You're managing your finances very effectively. The ar...

Question	Answered by	Answer (shortened)
How much did I spend on groceries?	fixed help text	I can answer questions like "How much did I spend on Food?", "Where am I spending the most?", "Can I save INR ...
What is my biggest expense?	fixed help text	I can answer questions like "How much did I spend on Food?", "Where am I spending the most?", "Can I save INR ...
Tell me a joke	fixed help text	I can answer questions like "How much did I spend on Food?", "Where am I spending the most?", "Can I save INR ...

The numbers in the answers are correct: the Food answer is INR 17,365.00, equal to a direct SUM over the same records, and the top category is Rent at INR 108,000.00, as a group-by maximum gives. One quirk remains: Am I overspending on shopping? is answered with the Shopping total (INR 13,165), not a comparison with the budget.

8.7 Import pipeline, API latency and tests

8.7.1 Statement import

A 1,000-row CSV was prepared with all categories blank and 84 injected bad rows (28 unparsable dates, 17 negative amounts, 23 blank descriptions and 16 non-numeric amounts). The pipeline skipped exactly 84 rows, each with a Row n warning, and imported 916, auto-labelling all of them (916). Cleaning and categorising took 6.6 s and committing 26.04 s, **35.2 rows per second**: the commit opens one database session per row, the slow part and the first thing to change for large statements. The auto-labels inherit the accuracy of Section 8.3; their correct share is unknown, as the generated file has no ground truth.

8.7.2 API latency

Reads are fast, with medians between 7.3 and 72.1 ms. Login takes 305 ms because bcrypt is deliberately slow; the heaviest calls are the Excel report (209 ms) and the 100-row import preview (667 ms). Single user, single process: no load or concurrency test was run.

Table 41: API latency in milliseconds (n = repetitions).

Endpoint	n	Median	p95	Max
login (bcrypt verify)	30	304.9	367.4	509.0
GET /health	30	4.3	19.0	21.3
GET /dashboard	30	67.2	129.3	194.6
GET /transactions	30	12.7	38.9	96.7
GET /analytics	30	72.1	128.5	170.0
GET /budgets	30	19.7	51.6	63.7
GET /goals	30	7.3	11.6	14.0
GET /investments	30	7.0	12.8	17.8
GET /assistant/insights	30	35.9	47.9	55.0
GET /notifications	30	18.1	33.2	49.3
GET /meta	30	7.2	11.8	12.8
POST /transactions/predict-category	30	7.9	20.5	30.6
POST /assistant/ask (analytics)	30	21.6	47.9	68.7
POST /assistant/ask (health)	30	28.4	47.8	86.4
POST /transactions (auto-categorise)	30	15.2	20.2	26.8

Endpoint	n	Median	p95	Max
GET /reports/csv	10	17.7	30.5	59.1
GET /reports/excel	10	209.4	341.6	1165.2
GET /reports/pdf	10	135.5	201.8	211.9
POST /transactions/import/preview (100 rows)	10	667.0	933.3	941.0

8.7.3 Tests

`pytest` ran 38 tests and all 38 passed (authentication 9, budgets 6, financial health 6, machine learning 9, transactions 8) in 11.5 s, with 16 warnings from the scikit-learn pickle version. The end-to-end smoke script, which starts its own backend and a throw-away database, passed 49 of 49 checks (rejected weak passwords and tampered tokens, 401 on protected routes, per-user isolation, transactions, import, budgets, goals, investments, analytics, the assistant, the three downloads, profile and the demo-data loader). Not tested: browser-level UI tests (screens were captured by script and inspected by eye), concurrency, a security scan, accessibility, the language-model layer and the live price refresh.

8.8 Comparison with the base paper

The registered base paper [1] (Kharat et al.) builds four models in a Streamlit assistant: a BERT expense categoriser over 50 categories, a Q-learning budget recommender, an LSTM savings forecaster and an Isolation Forest anomaly detector. FINJARVIS implements simpler versions of the first and third modules, a rule-based substitute for the second and nothing for the fourth. The reported numbers belong to different data, label sets and models, so **they are shown side by side but must not be ranked**.

Table 42: Base paper against this work.

Module	Base paper [1]	FINJARVIS (this work)	Comparable?
Categorisation	BERT (bert-base-uncased, 10 epochs), 50 categories, SMOTE, 80/20 split of own data: accuracy 92.3 %, precision 90.8 %, recall 91.5 %, F1 91.1 %	TF-IDF + logistic regression, 11 categories, 51 training examples; team-written probe set: accuracy 71.7 %, macro F1 76.1 %; unseen merchants 18.2 %	No: different data, 50 against 11 labels, model size, split
Budget recommendation	Q-learning; convergence after about 150-200 of 300 episodes; no numeric accuracy	Budgets are set by the user; alerts at 75, 90 and 100 %; rule-based advice (top categories, trim 10-15 %)	No: nothing is learned here
Forecasting	LSTM on a synthetic savings series: RMSE 137.2, MAE 89.5, R-squared 0.63 (units not stated)	Linear trend on monthly expense totals: MAPE 28.4 % (MAE INR 9,597) on 3 backtest points; synthetic simulation in Section 8.4	No: different target, units and series
Anomaly detection	Isolation Forest: precision 93 %, recall 94 %, F1 94 %, on data with synthetic outliers	**Not implemented**; the nearest feature is the largest expense in a date range	Not applicable
Interface and delivery	Streamlit dashboard; Flask API mentioned	React 18 + FastAPI web app with 13 pages, 33 routes, SQLite; Streamlit original kept	Qualitative only

This work goes beyond the base paper in the product, not the models: a runnable multi-user web application with accounts, imports, budgets with alerts, goals, a portfolio, reports and an

explainable score, plus a measurement of where its simple models fail. It falls short in modelling: no deep categoriser, learned budget policy, sequence forecaster or anomaly detector. The batch abstract names Flask and MySQL; the shipped system uses FastAPI and SQLite, and the loan tracking in the abstract and periodic digest alerts were not built.

8.9 Sample inputs and outputs

The samples are taken from the stored experiment files; the demo account is fictional.

8.9.1 Forecast

`predict_next_month_expense` on the demo account's six months returns predicted_next_month_expense 39,470.53, recent_average 36,592.33, trend increasing and months_used 6 (Section 8.4).

8.9.2 Categoriser

Table 43: Misclassified probe descriptions (first four tier-C errors and both tier-B errors).

Description	True category	Predicted	Tier
BigBasket delivery	Food	Transport	C
Dominos pizza	Food	Transport	C
Blinkit groceries	Food	Transport	C
Chaayos tea	Food	Transport	C
UPI-AMAZON-PAY-8821	Shopping	Entertainment	B
UPI-HOSPITAL-BILL	Healthcare	Bills	B

```
# text with no known word (all-zero feature vector)
input:     xyzabc123 qqq
predicted: Transport
probabilities (top 3): Transport 0.1210, Food 0.1183, Entertainment 0.1178
```

8.9.3 Assistant

```
Q: How much did I spend on Food?
A: You've spent INR 17,365.00 on Food across your recorded transactions.


Q: What is my financial health?
A: Your financial health score is 90.4/100 (Excellent). You're managing your finances very effectively. The
area most holding you back right now is goal progress,


Q: What is my biggest expense?   (not matched)
A: I can answer questions like "How much did I spend on Food?", "Where am I spending the most?", "Can I save
INR 5000 this month?", "...
```

8.10 Success and failure analysis

8.10.1 Where the product succeeds

- Every screen of Section 8.2 works end to end on the demo data: sign-up, login, transactions with import, budgets with alerts, goals, a portfolio, analytics, the assistant, three report formats and notifications.

- The rules are exact and tested: 38 unit tests and 49 smoke checks pass, the assistant's numbers equal direct SUMs, the import skips exactly the 84 bad rows, and the health score is monotone in the savings rate over 5,000 random profiles.
- The categoriser is dependable inside its vocabulary (100 % on seed-like wording, 97 % on statement strings with a known keyword), and one correction fixes a repeat merchant in all 198 test cases.
- Reads answer in under 72 ms at the median and a categorisation takes 0.84 ms.

8.10.2 Where it falls short

- **Unseen merchants are silently mislabelled.** 48 of the 56 probe errors are labelled Transport, tier-C accuracy is 18.2 % and the model never answers Other or asks.
- **Corrections do not generalise.** After 48 corrections tier-C accuracy is 21.5 %; only repeats of a corrected merchant are fixed.
- **The seed list is tiny** (51 pairs, 3 to 6 per class), so leave-one-out accuracy is 11.8 %.
- **The forecast is not better than an average with fewer than seven months of history**, and the three-month minimum lets it run in exactly that regime; the demo backtest has only three points.
- **The assistant misses natural phrasings** (6 of 20 answerable questions) and answers a budget question with a category total.
- **Anomaly detection, reinforcement-learning budgeting, a sequence forecaster and a language-model categoriser were not built**, so objective 4 is not met and objectives 3 and 5 only in simple form.
- **Import commits one row at a time** (35.2 rows per second).
- **Budget alerts are de-duplicated by message text**, so a budget that crosses 75 % again in a later month raises no new alert; the screenshot in Section 8.2.6 shows a 75 % message while Entertainment stands at 88 %, because the 90 % message appears only above 90 %.
- **Tokens are home-made signed tokens** (HMAC-SHA256, 7 days, no revocation) with a default development secret when `APP_SECRET_KEY` is unset, and the database is one SQLite file without migrations.

8.11 Limitations of the evaluation

- All application data are fictional; no real person's transactions or bank API were involved.

- The categoriser was scored on a 198-item probe set written by the team that wrote the seed list: a diagnostic, not a benchmark, whose tier proportions (one third unseen merchants) drive the headline accuracy.
- The forecast was backtested on three points of one fictional account; the simulation uses a generating process of the team's own choosing.
- The health-score weights are expert-chosen and not validated; no user study was run. The assistant was tested with 26 team-written questions and its language-model layer never ran.
- Latency was measured with one client over loopback on one machine; import speed shifts between runs when the machine is busy. Not exercised: live price refresh, concurrent users, mobile browsers, a security scan and accessibility testing.

8.12 Objectives versus outcomes

Table 44: The six project objectives against what was measured.

Objective	Status	Evidence
1. Secure user accounts with transaction import and manual entry	Achieved	Sign-up and login with bcrypt hashes, password rules and signed tokens; per-user data scoping; manual entry and CSV or Excel import with preview (916 of 1,000 rows imported, 84 bad rows skipped); 38 tests and 49 smoke checks pass
2. Categorise expenses automatically with text classification	Partly achieved	TF-IDF + logistic regression labels with 71.7 % probe accuracy and 76.1 % macro F1, but only 18.2 % on unseen merchants; 11 categories from 51 examples, no BERT
3. Forecast upcoming expenses and savings with time-series models	Partly achieved	A linear trend forecasts next month's expenses and per-category spend (INR 39,471 for November on the demo); MAPE 28.4 % on three points, no better than a mean under seven months; savings are not forecast directly; no ARIMA or LSTM
4. Detect anomalous transactions	Not achieved	No anomaly detector exists; only the largest expense in a date range is reported
5. Personalised budget recommendations and alerts on threshold breaches	Partly achieved	Budgets are set by the user; alerts fire at 75, 90 and 100 % of a budget; advice is rule-based (top categories, a 10 to 15 % trim) and uses the user's own income and spending; no learned policy
6. Visualise spending, category breakdown and forecasts on an interactive dashboard	Achieved	React dashboard, analytics, budgets, goals and investment pages with charts and a forecast card (Section 8.2); health score with six explained components; reports in PDF, Excel and CSV

8.13 Summary of findings

- FINJARVIS runs as a complete personal finance web application (13 pages, 33 routes, 6 tables, 38 tests and 49 smoke checks passing); reads answer in under 72 ms at the median.
- The categoriser works on familiar wording (100 % and 97 % on tiers A and B) but fails on unseen merchants (18.2 %), mostly by answering Transport; a confidence threshold of 0.15 would keep 62 % of items at 94 % accuracy but is not applied.

- A correction fixes a repeated merchant completely, but 48 corrections lift accuracy on new merchants only from 18 % to 22 %.
- The linear-trend forecast overshot October on the demo account (MAPE 28.4 % against 21.4 % for a plain mean) and, in simulation, needs about seven months of history to beat an average; the three-month minimum is too low.
- The health score is explainable and monotone (demo account 90.4, Excellent); its weights are not validated. The assistant answers 14 of 20 answerable questions from the user's data and declines all off-topic ones; its language-model layer is untested.
- Anomaly detection, reinforcement-learning budgeting, an LSTM forecaster, a BERT categoriser, loan tracking and scheduled digests were not built; the base paper's four-model design is matched only partly, and its reported numbers are not comparable with these.
- Objectives 1 and 6 are met; objectives 2, 3 and 5 are met in simple form; objective 4 is not met.

CHAPTER 9: CONCLUSION

9.1 Summary

The project set out to build a personal finance platform that records spending, categorises it, forecasts what is coming and gives advice grounded in the user's own data. FINJARVIS delivers the platform: a React and FastAPI web application over a SQLite database, with secure accounts, transaction entry and import, budgets, goals, investments, an assistant, reports and notifications. Its machine-learning parts are a TF-IDF and logistic-regression categoriser, a least-squares spending forecast and a six-component health score.

The work was verified by 38 pytest tests, a 49-check end-to-end smoke test, a 198-description probe set, a rolling backtest, a synthetic forecasting simulation, 5,000 random health profiles, a 26-question assistant test, a 1,000-row import test and API latency measurements. The measurements are reported as they came out, including those that contradict the first draft of the design.

9.2 Objectives achieved

Table 45: Objectives against outcomes

Objective	Outcome	Evidence
1 Secure accounts with transaction import and manual entry	Achieved	bcrypt, signed tokens, per-user scoping; manual entry and CSV/Excel import (916 of 1,000 rows imported, 84 bad rows skipped); 49 of 49 smoke checks
2 Categorise expenses automatically from descriptions	Partly achieved	71.7 percent on the probe set but 18.2 percent on unseen merchants; 51 seed examples; 11 categories
3 Forecast expenses and savings with time-series models	Partly achieved	Linear trend on monthly expenses only; no ARIMA or LSTM; MAPE 28.4 percent on the backtest, not better than a mean
4 Detect anomalous transactions	Not achieved	No anomaly detector exists; only the largest expense in a window is shown
5 Personalised budget recommendations and threshold alerts	Partly achieved	Alerts at 75, 90 and 100 percent and rule-based advice from the user's data; budgets are set by the user, no learned recommender
6 Interactive dashboard of spending, categories and forecasts	Achieved	13 pages in dark and light themes; dashboard 67 ms and analytics 72 ms median

9.3 Key findings

1. The categoriser is accurate where its vocabulary overlaps the seed list (100 percent on seed-like text, 97 percent on statement strings) and weak elsewhere (18.2 percent on unseen merchants), against 51.5 percent for the keyword table over the whole probe set.
2. Its mistakes are not random: 48 of 56 errors are labelled Transport (precision 0.273), because the model has no way to say it does not know.

3. A confidence threshold of 0.15 would keep 62.1 percent of the probe items at 94.3 percent accuracy, so abstaining is a practical improvement the app does not yet make.
4. One correction fixes a repeat merchant (18.2 to 100 percent) but forty-eight corrections move unseen-merchant accuracy only to 21.5 percent.
5. The linear-trend forecast is not better than a plain average for short histories: MAPE 28.4 against 21.4 percent on the demo backtest, and in simulation it overtakes the mean of all months at 7 months of history and the mean of the last three months only at 9 months, so the three-month minimum is too low.
6. The health score behaves sensibly (no decrease when the savings rate rises in 5,000 random profiles) but saturates: the savings-rate component stops changing at 20 percent, and its weights are not fitted.
7. The rule-based assistant is correct when it answers and silent about much of what users might ask: 14 of 20 answerable questions handled, 6 of 6 off-topic questions declined.
8. The application is fast for one user: every dashboard, list and analytics endpoint answers in under 82 ms median, reports in 136 to 209 ms, with login (305 ms) the slowest by design.

9.4 Module-by-module conclusions

Accounts and data handling. Every query is scoped by user, weak passwords, wrong passwords and tampered tokens are rejected, every import row is validated, and a keyword fallback keeps categorisation working if the model file is unreadable; the 49 smoke checks and 38 unit tests cover these paths.

Categorisation. The model is a good lookup table for descriptions that resemble its seed list and a poor guesser for anything else. Its overall probe accuracy of 71.7 percent hides a split of 100, 97 and 18.2 percent across the three tiers, which is why the tier breakdown, the confusion matrix and the Transport dump class are reported with it. Learning from corrections is worth keeping, because a merchant tagged once stays tagged, but it is no substitute for a larger training set.

Forecasting. The straight line follows a steady rise and over-shoots a sudden fall, as in the demo account's last month; with little history a mean is as good or better. It is useful as a prompt to look at a category, but should be presented with its limits.

Health score and assistant. The score gives one number and one reason, and the assistant repeats those figures in words; neither adds information that is not in the data, a strength for trust and a limit for insight.

9.5 Practical applications

- A student or salaried person can keep all transactions in one place, import a bank statement as CSV, and see where the money goes without a subscription fee.
- Budget alerts and the health score show whether spending is on track and which area is weakest, with the reason.
- The assistant answers common questions (spend on a category, can I save a given amount, where can I cut) from the user's data and says plainly when it cannot answer.
- A teaching team can reuse the code and experiment script as an honest example of measuring simple models before trusting them.

9.6 Closing remark

The value of the project lies in a platform that works end to end and a measured account of how its simple models behave. The experiments do not show that TF-IDF and a line can replace the base paper's BERT, LSTM and Isolation Forest; they show where the simple versions fail and the first improvements: an abstain threshold and a mean fallback for short histories.

9.7 Known limitations

The categoriser fails silently on unseen merchants (18.2 percent), the forecast is not better than a mean on short histories, the assistant answered 14 of 20 answerable questions, the health weights are not fitted, and tokens use a home-made scheme with a default secret. The data are fictional or simulated, no user study was carried out, and the language-model layer, live prices and concurrency were not tested.

CHAPTER 10: FUTURE SCOPE

10.1 Improvements to the existing system

- **Abstain instead of guessing.** A threshold of 0.15 on the top class probability is the cheapest repair for the silent Transport errors; it should be judged by coverage and accuracy on a larger probe set that includes unseen merchants.
- **Grow the seed list.** Fifty-one examples, three to six per class, give a leave-one-out accuracy of 11.8 percent; several hundred labelled descriptions per class from consenting users would change the picture and allow a proper train and test split.
- **Make the forecast honest for short histories.** Use the mean of the recent months below about seven months, and report an interval rather than a single number.
- **Strengthen authentication.** Standard signed tokens with refresh and revocation, an enforced secret and rate-limited login.
- **Alert logic.** Key alert de-duplication by budget, month and threshold, not by message text.

10.2 Additional features

- **Anomaly detection** for objective 4: a robust statistical rule first, then an Isolation Forest, evaluated against injected and labelled outliers.
- **Savings and income forecasting** with exponential smoothing or ARIMA, compared by rolling-origin backtest, for objective 3.
- **Loan tracking** with amortisation and prepayment advice, and **scheduled digests** for the abstract's loan tracking and task scheduling.
- **Statement parsers** for common Indian bank formats and UPI exports, so that import needs no preparation.

10.3 Scalability and deployment

FINJARVIS runs as one process with one SQLite file, which suits a single person or a demonstration; no load test was run. A shared deployment needs a server database with migrations, several workers behind a reverse proxy, HTTPS and a secret kept outside the code.

- Replace the per-call database sessions with a connection pool and batch writes.
- Serve the built React bundle from a static host and keep the API behind HTTPS.
- Add monitoring of error rates and latency per endpoint, and nightly database backups.

10.4 Evaluation plan for the next version

- **A held-out merchant split.** Divide the labelled descriptions by merchant, not by row, so that the test set contains only merchants the model has never seen; report accuracy and macro F1 for that split and for a random split.
- **Selective accuracy.** Report coverage and accuracy at several confidence thresholds.
- **Rolling-origin forecasting.** Evaluate every method on many months of many users, not three points of one account, with MAE, MAPE and a naive baseline.
- **Injected-anomaly benchmark.** Generate outliers of known size in real histories and measure precision, recall and the number of false alarms per user per month.
- **A small user study.** Ask a few users to categorise a week of their own transactions with and without the assistant and the score, and record time taken and corrections made.

10.5 Order of work

First the cheap repairs (abstain threshold, short-history fallback, alert de-duplication, token secret, batched import), each re-measured with the existing scripts. Then the missing models, anomaly detection and a savings forecast. Last loan tracking, scheduling and statement parsers, which need more design and real sample files.

10.6 Risks and dependencies

- **Data and privacy.** A larger corpus and statement parsing mean handling real financial data; consent, minimisation and secure storage must come first.
- **Evaluation.** A bigger model that scores better on the team's probe set may only have learned its wording; any new model should be tested on merchants and months it has not seen.
- **Advice.** The assistant and health score are informational and must keep their disclaimer; a language-model layer needs checks that figures in its answers match the database.

10.7 Further research

- How much labelled data does a TF-IDF model need before unseen-merchant accuracy becomes useful, and does a character n-gram or transformer model close the gap at a similar cost?
- How should a forecast combine a trend with seasonal effects such as festivals and rent resets when only a few months exist? The base paper's LSTM result (R2 0.63 on synthetic savings, [1]) does not answer this for real users.

REFERENCES

[1] R. Kharat, S. Solapurkar, A. Tikkas, M. Uttarwar and A. Yadav, "A smart AI-based personal finance assistant: A multi-model approach for expense categorization, budgeting, forecasting and anomaly detection," Int. J. Research in Applied Science and Engineering Technology (IJRASET), vol. 13, no. XI, pp. 167-180, Nov. 2025, doi: 10.22214/ijraset.2025.75019.

[2] P. L. To, "Enhancing financial literacy and management through goal-directed design and gamification in personal finance application," arXiv:2601.08640, Jan. 2026.

[3] A. Theerthala, "Synthesizing behaviorally-grounded reasoning chains: A data-generation framework for personal finance LLMs," arXiv:2509.14180 [cs.CL], Sep. 2025.

[4] V. Dilip and R. S. Janaki, "FinSenseAI: An explainable AI-based personal finance recommendation and expense prediction system using hybrid machine learning models," Int. J. Innovative Science and Research Technology, vol. 11, no. 7, pp. 1637-1650, Jul. 2026, doi: 10.38124/ijisrt/26jul842.

[5] F. R. Abiya, S. Parvathy, M. R. Devende, S. R. Ajmalsha and C. Viji, "Personal expense leakage detection and budget optimization using machine learning," Int. J. Research in Applied Science and Engineering Technology (IJRASET), Apr. 2026, doi: 10.22214/ijraset.2026.79467.

[6] "AI-driven financial insights for personal budget planning: A smart approach to future expense prediction," in Proc. 2025 3rd Int. Conf. on Intelligent Data Communication Technologies and Internet of Things (IDCIoT), IEEE, 2025 (IEEE Xplore document 10914721; as listed in the batch survey, authors not verified).

[7] "AI-driven finance management platform for intelligent expense tracking and budget automation," Int. Research J. of Modernization in Engineering Technology and Science, 2026 (as listed in the batch survey; not located, not verified).

[8] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, 2019, pp. 4171-4186.

[9] D. Araci, "FinBERT: Financial sentiment analysis with pre-trained language models," arXiv:1908.10063, 2019.

[10] F. T. Liu, K. M. Ting and Z.-H. Zhou, "Isolation forest," in Proc. 8th IEEE Int. Conf. on Data Mining (ICDM), 2008, pp. 413-422.

[11] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735-1780, 1997.

[12] C. J. C. H. Watkins and P. Dayan, "Q-learning," Machine Learning, vol. 8, no. 3-4, pp. 279-292, 1992.

[13] N. V. Chawla, K. W. Bowyer, L. O. Hall and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," J. Artificial Intelligence Research, vol. 16, pp. 321-357, 2002.

[14] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[15] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing & Management, vol. 24, no. 5, pp. 513-523, 1988.

[16] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," Int. J. Forecasting, vol. 22, no. 4, pp. 679-688, 2006.

[17] S. Makridakis, E. Spiliotis and V. Assimakopoulos, "Statistical and machine learning forecasting methods: Concerns and ways forward," PLoS ONE, vol. 13, no. 3, e0194889, 2018.

[18] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. Melbourne, Australia: OTexts, 2021.

[19] A. Lusardi and O. S. Mitchell, "The economic importance of financial literacy: Theory and evidence," J. Economic Literature, vol. 52, no. 1, pp. 5-44, 2014.

[20] N. Provos and D. Mazieres, "A future-adaptable password scheme," in Proc. USENIX Annual Technical Conf., FREENIX Track, 1999, pp. 81-91.

[21] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000.

[22] M. Bayer, "SQLAlchemy," in The Architecture of Open Source Applications, Volume II, A. Brown and G. Wilson, Eds. 2012.

[23] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in Proc. 14th Int. Joint Conf. on Artificial Intelligence (IJCAI), 1995, pp. 1137-1143.

APPENDIX A: IMPORTANT CODE

The key functions are copied from `8.ProjectCode` as they stood when the results were measured; docstrings are removed to save space (the full files are in the repository). The categoriser and the forecast in Sections A.1 and A.2 and the health score in Section A.3 are explained with formulas in Chapter 6; Sections A.4 to A.6 show how alerts, tokens and the assistant's routing are written.

A.1 Expense categoriser (ml/expense_classifier.py)

```
def _train_pipeline(descriptions: list[str], labels: list[str]) -> Pipeline:
    pipeline = Pipeline([
        ("tfidf", TfidfVectorizer(lowercase=True, ngram_range=(1, 2), min_df=1)),
        ("clf", LogisticRegression(max_iter=1000)),
    ])
    pipeline.fit(descriptions, labels)
    return pipeline


def predict_category(description: str) -> str:
    description = (description or "").strip()
    if not description:
        return "Other"


    pipeline = _load_pipeline()
    if pipeline is not None:
        try:
            return pipeline.predict([description])[0]
        except Exception:
            pass


    lowered = description.lower()
    for keyword, category in KEYWORD_FALLBACK.items():
        if keyword in lowered:
            return category
    return "Other"


def learn_from_correction(description: str, correct_category: str, user_history: list[tuple[str, str]])
        -> None:
    examples = list(user_history) + [(description, correct_category)]
    train_and_save(extra_examples=examples)
```

A.2 Spending forecast (ml/spending_prediction.py)

```
def predict_next_month_expense(df: pd.DataFrame) -> dict:
    totals = _monthly_totals(df, "expense")
    if len(totals) < MIN_MONTHS_REQUIRED:
        return {
            "available": False,
            "message": f"Need at least {MIN_MONTHS_REQUIRED} months of transaction "
                       f"history for a reliable prediction (currently have {len(totals)}).",
        }


    y = totals.values.astype(float)
    x = np.arange(len(y))
    # Simple least-squares linear fit; clip prediction at 0 (can't spend negative money)
    slope, intercept = np.polyfit(x, y, 1)
    next_x = len(y)
    predicted = max(0.0, slope * next_x + intercept)


    return {
        "available": True,
        "predicted_next_month_expense": round(predicted, 2),
        "recent_average": round(float(np.mean(y[-3:])), 2),
        "trend": "increasing" if slope > 1 else ("decreasing" if slope < -1 else "stable"),
        "months_used": len(y),
    }
```

A.3 Financial health score (ml/financial_health.py)

```
WEIGHTS = {
    "savings_rate": 0.30,
    "expense_ratio": 0.20,
    "budget_adherence": 0.15,
    "emergency_fund": 0.15,
    "investment_diversification": 0.10,
    "goal_progress": 0.10,
}


def score_savings_rate(savings_rate: float) -> float:
    # 20%+ savings rate = full marks; scales linearly below that; negative = 0
    return _clip((savings_rate / 20.0) * 100)


def score_emergency_fund(total_balance: float, monthly_expenses: float) -> float:
    if monthly_expenses <= 0:
        return 50.0
    months_covered = total_balance / monthly_expenses
    # 6 months of expenses covered = full marks
    return _clip((months_covered / 6.0) * 100)


def score_goal_progress(goals: list[dict]) -> float:
    if not goals:
        return 50.0  # neutral if no goals set
    avg_progress = sum(g["percent_complete"] for g in goals) / len(goals)
    return _clip(avg_progress)
```

A.4 Budget alerts (services/budget_service.py)

```
def check_and_raise_alerts(user_id: int, month: int, year: int) -> list[str]:
    created = []
    statuses = get_budget_status(user_id, month, year)
    with get_session() as session:
        for s in statuses:
            pct_used = s["percent_used"]
            if pct_used >= 100:
                msg = f"You have exceeded your {s['category']} budget."
                ntype = "danger"
            elif pct_used >= 90:
                msg = f"Your {s['category']} budget is 90% used."
                ntype = "warning"
            elif pct_used >= 75:
                msg = f"Your {s['category']} budget is 75% used."
                ntype = "warning"
            else:
                continue


            exists = session.query(Notification).filter(
                and_(Notification.user_id == user_id, Notification.message == msg)
            ).first()
            if not exists:
                session.add(Notification(user_id=user_id, message=msg, notification_type=ntype))
                created.append(msg)
    return created
```

A.5 Tokens and the authentication dependency (backend/main.py)

```
def make_token(user_id: int) -> str:
    payload = _b64(json.dumps({"uid": user_id, "exp": int(time.time()) + TOKEN_TTL}).encode())
    sig = _b64(hmac.new(SECRET, payload.encode(), hashlib.sha256).digest())
    return f"{payload}.{sig}"


def read_token(token: str) -> int | None:
    try:
        payload, sig = token.split(".", 1)
        good = _b64(hmac.new(SECRET, payload.encode(), hashlib.sha256).digest())
        if not hmac.compare_digest(sig, good):
            return None
        data = json.loads(base64.urlsafe_b64decode(payload + "=" * (-len(payload) % 4)))
        return int(data["uid"]) if data["exp"] > time.time() else None
    except Exception:
        return None


def current_user(authorization: str | None = Header(default=None)) -> SimpleNamespace:
    token = (authorization or "").removeprefix("Bearer ").strip()
    uid = read_token(token) if token else None
```

```
    if uid is None:
        raise HTTPException(401, "Your session has expired. Please log in again.")
    with get_session() as session:
        u = session.query(User).filter(User.id == uid).first()
        if not u:
            raise HTTPException(401, "This account no longer exists. Please log in again.")
        return SimpleNamespace(**user_dict(u))
```

APPENDIX B: INSTALLATION AND USER GUIDE

B.1 Requirements

Python 3.11 or newer, Node.js 20 or newer and the packages in `backend/requirements.txt` (bcrypt is pinned to 4.0.1). No account or key is required; `AI_API_KEY` is optional and enables free-form assistant answers.

B.2 Installation and start with run.bat

1. Copy or clone the project so that `8.ProjectCode` is on a local disk.
2. Open the folder and double-click `run.bat` (or run it from a command prompt). It checks that Python and Node are installed, creates `.venv` if absent (and rebuilds one that was copied from another PC), installs `backend\requirements.txt` and runs `npm install` in `frontend` the first time (a few minutes).
3. It starts the FastAPI backend on port 8411 and the web app on port 5411 in two minimised windows and opens the browser at `http://localhost:5411`. Keep both windows open.
4. Log in with the built-in demo account `demo` / `Demo@1234` (or press Try the demo account) or create your own account. To stop, close the two windows.
5. For the documentation run the servers were started by `scripts/capture_screens.py` and the measurement scripts, which start their own backend on a throw-away database; a fresh-machine installation through `run.bat` was not repeated.

B.3 Manual start and settings

```
# backend (from 8.ProjectCode)
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r backend\requirements.txt
.venv\Scripts\python.exe -m uvicorn backend.main:app --port 8411


# frontend (from 8.ProjectCode\frontend)
npm install
npm run dev          # http://localhost:5411, proxies /api to 8411
npm run build        # production build into frontend\dist


# the original Streamlit app over the same database
streamlit run app.py   # http://localhost:8501
```

Settings are read from `.env` (copy `.env.example`); every one is optional. `.env` is git-ignored and must never be committed.

Table 46: Environment variables

Variable	Meaning
BACKEND_PORT / FRONTEND_PORT	Ports used by `run.bat` (default 8411 / 5411)

Variable	Meaning
APP_SECRET_KEY	Signs the login tokens; set a long random string (a development default is used otherwise)
AI_API_KEY	Optional Anthropic key; enables free-form answers in the AI assistant

B.4 Using the web app

Table 47: Steps for each page

Page	What to do	What you see
Landing /	Read the product summary; press Create free account, Log in or Try the demo account	A demo card with the health score and a six-month curve computed from the demo account
Login and sign-up	Enter a username or e-mail and a password, or create an account with a name, e-mail, username and a password with 8 or more characters, an upper-case and a lower-case letter, a digit and a special character	Password-rule chips that tick as you type; a welcome toast
Dashboard	Read the eight summary cards, the health score and the budget alerts; press Add transaction	Charts of income against expenses, spending by category, the monthly trend and the investment allocation
Transactions	Add: fill the form and leave the category on Auto-detect or choose one to teach the model. View / manage: filter, search, sort, edit or delete. Import: drop a CSV or Excel file with columns date, description, amount, category and type	A preview with skipped-row warnings before anything is saved
Analytics	Choose a date range and what the category chart shows	Totals, savings rate, daily average, largest expense, charts, the next-month prediction and any predicted budget overrun
Budgets	Pick a month, a category and an amount; saving again for the same category updates it	Progress bars with On track, Getting close or Over budget badges; alerts at 75, 90 and 100 %
Goals and Investments	Add a goal with a target amount and date, update the saved amount; add holdings and press Price to refresh a live price (needs internet)	Progress, the monthly saving needed, portfolio value, profit and loss
AI Assistant	Type a question or press a suggestion	An answer calculated from your data, with a disclaimer; the insights panel on the right
Reports	Choose month and year; press Generate PDF, Generate Excel or Export CSV	A downloaded file
Notifications and Profile	Mark alerts as read; edit name, income, savings target, currency and risk preference; Load demo data adds fictional records	The unread count in the sidebar and the bell

The assistant understands questions of these kinds: how much did I spend on a category, where am I spending the most, can I save an amount this month, how can I reduce my expenses and what is my financial health. Other phrasings receive a fixed help text (Section 8.6). The application gives information, not financial advice, and says so on the assistant page.

B.5 Tests and experiments

```
python -m pytest
python scripts/smoke_test.py
python scripts/run_experiments.py
python scripts/measure_api.py
python scripts/capture_screens.py
```

B.6 Troubleshooting

Table 48: Common problems

Symptom	Cause and fix
Server offline in the top bar	The API is not running. Start `run.bat` or read the FINJARVIS backend window for the error
Port already in use	Change BACKEND_PORT or FRONTEND_PORT in `.env`, or close the program that holds the port
Sign-up says the password cannot be longer than 72 bytes	bcrypt 5 is installed and breaks passlib; delete `.venv` and run `run.bat` again so that bcrypt 4.0.1 is installed
Live investment price fails	It needs internet and a ticker such as `NIFTYBEES.NS`; the last known price is kept
The assistant says free-form chat is unavailable	Set `AI_API_KEY` in `.env` and restart; the built-in analytics answers work without it

APPENDIX C: ADDITIONAL SCREENS

The screens below are captures that are not shown in Chapter 8: the light theme of the landing page and of the dashboard, and the sign-up page. All were taken from the running application at 1440 x 900 pixels, logged in as the fictional demo account where a login is needed, on 22 October 2026.

C.1 Light theme and sign-up pages

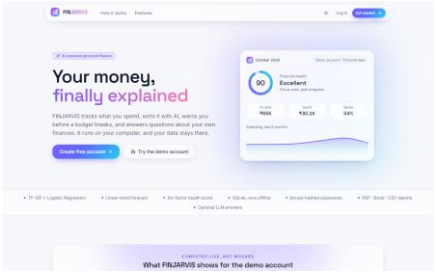


Figure 37: Additional screen: landing page, light theme, with the same demo card (health score 90, income INR 65K, spent INR 30.1K, saved 54 %).

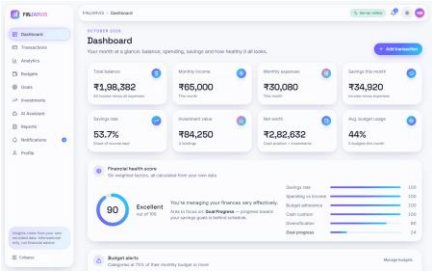


Figure 38: Additional screen: dashboard, light theme: the same eight cards and the health score card as in the dark theme.

Both themes use the same colour tokens; the toggle sits in the top bar. The light dashboard repeats the figures of the dark one exactly (total balance INR 1,98,382, net worth INR 2,82,632, health score 90), which confirms that the theme changes only the presentation.

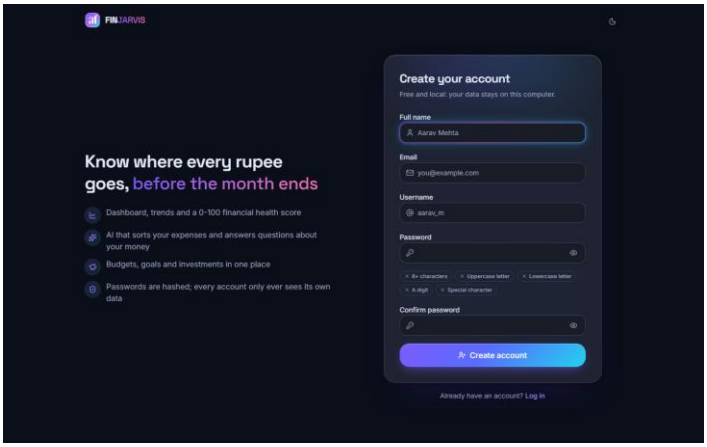


Figure 39: Additional screen: sign-up page with the password-rule chips (8+ characters, upper-case, lower-case, a digit, a special character) that turn on as the password satisfies each rule.

The page validates the rules in the browser and the server validates them again (username of 3 to 30 characters, a valid e-mail address, a matching confirmation and a password that meets all five rules). The name shown in the field is a placeholder, not an account.

C.2 Additional result charts

The charts below belong to the measurements of Chapter 8 and are kept here to save space in the chapter.

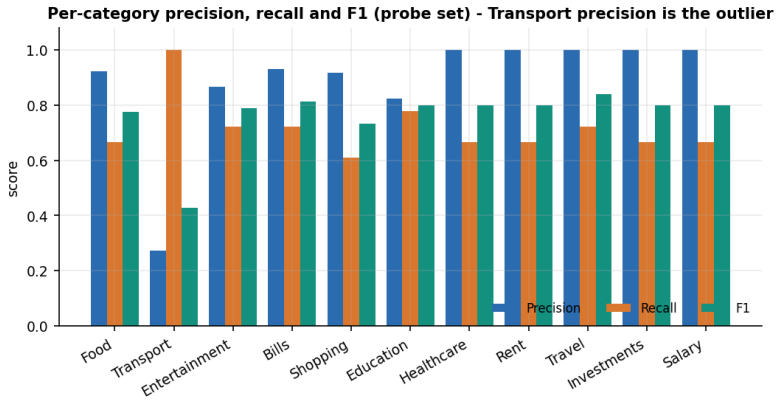


Figure 40: Precision, recall and F1 per category on the probe set; Transport recalls everything but is right on only 27 % of what it labels.

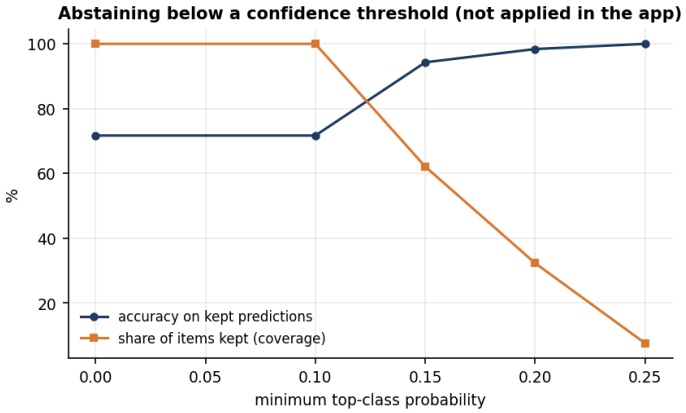


Figure 41: Accuracy on kept predictions and share of items kept as the minimum probability rises (computed offline; the application applies no threshold).

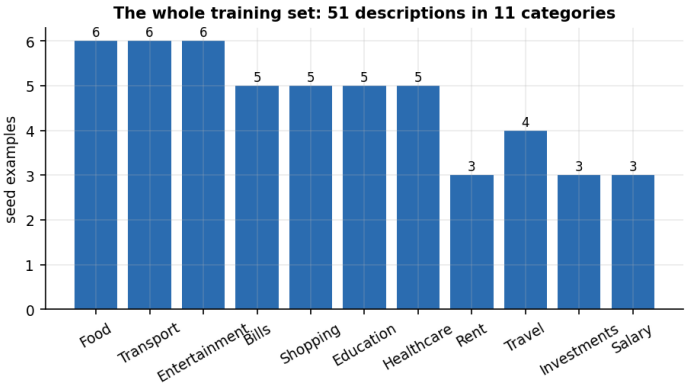


Figure 42: The complete training set of the categoriser: 51 descriptions, 3 to 6 per category.

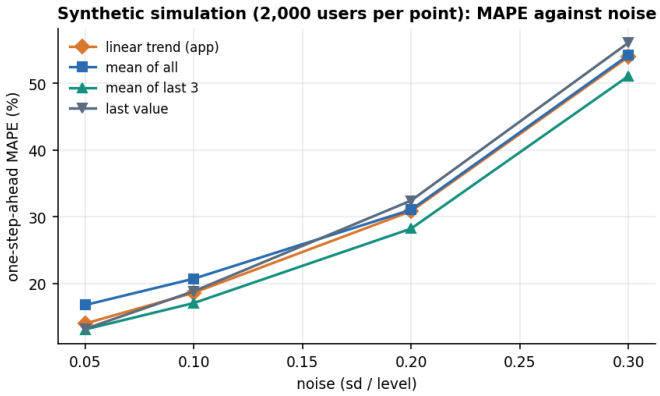


Figure 43: Simulated MAPE of the four forecasting methods against noise (2,000 synthetic users per point).

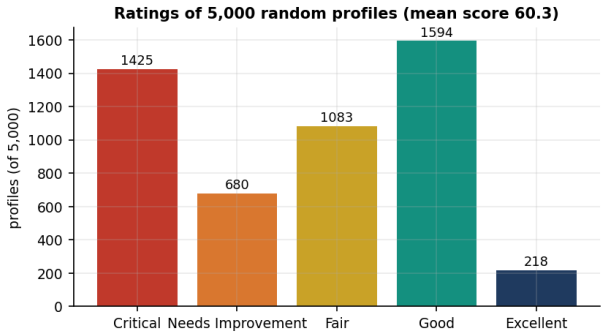


Figure 44: Ratings of 5,000 random profiles under the health score.

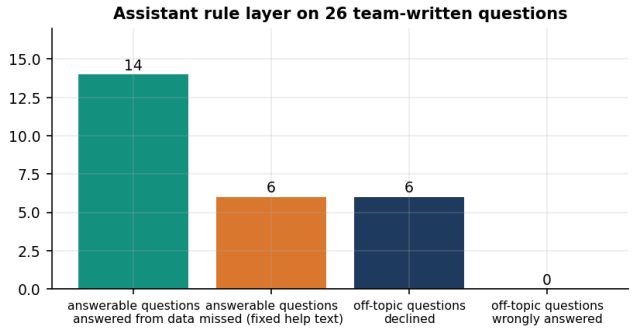


Figure 45: Outcome of the 26 test questions for the assistant's rule layer.

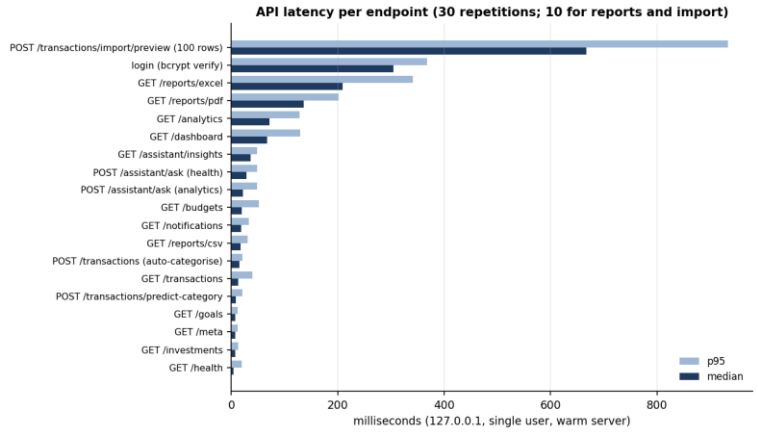


Figure 46: Median and 95th-percentile latency of every measured endpoint.